

laSalle

UNIVERSITAT RAMON LLULL

Escola Tècnica Superior d'Enginyeria La Salle

Treball Final de Grau

Grau en Enginyeria Informàtica

Intelligent Matchmaking for e-Service-Learning:
A Platform for Enhanced Student Engagement
and Community Impact.

Alumne

Adrián-Jorge Sánchez López

Professor Ponent

Joan Navarro Martín

ACTA DE L'EXAMEN DEL TREBALL FI DE CARRERA

Reunit el Tribunal qualificador en el dia de la data, l'alumne

D. Adrián-Jorge Sánchez López

va exposar el seu Treball de Fi de Carrera, el qual va tractar sobre el tema següent:

Intelligent Matchmaking for e-Service-Learning:

A Platform for Enhanced Student Engagement
and Community Impact.

Acabada l'exposició i contestades per part de l'alumne les objeccions formulades pels Srs. membres del tribunal, aquest valorà l'esmentat Treball amb la qualificació de

--

Barcelona,

VOCAL DEL TRIBUNAL

VOCAL DEL TRIBUNAL

PRESIDENT DEL TRIBUNAL

Abstract

This thesis presents the development of a web-based matchmaking platform designed to foster civic engagement through e-Service-Learning (e-SL) initiatives. The platform addresses the need for enhanced collaboration between universities, teachers, students, and rural community organizations, particularly in underserved communities. The core objectives include facilitating the connection of these stakeholders, providing recommendations for e-SL activities, and streamlining the management of e-SL projects. The platform's development followed a software development approach that prioritized user-friendly design, accessibility, and multilingual support. Key features include user management with role-based access, activity management functionalities, an intelligent matching algorithm, an AI-powered chatbot recommender, and user dashboards. The successful implementation of this platform is expected to enhance the efficiency and effectiveness of e-SL initiatives, promote social inclusion, and strengthen the collaboration between academic institutions and rural communities.

Keywords: e-Service-Learning, Matchmaking Platform, Civic Engagement, Community Impact, Intelligent Matching, Chatbot Recommender, Rural Communities.

Resumen

Esta tesis presenta el desarrollo de una plataforma web de emparejamiento diseñada para fomentar la participación cívica a través de iniciativas *e-Service-Learning* (e-SL). La plataforma aborda la necesidad de una mayor colaboración entre universidades, profesores, estudiantes y organizaciones comunitarias rurales, particularmente en comunidades desatendidas. Los objetivos principales incluyen facilitar la conexión de estos actores, proporcionar recomendaciones para actividades de e-SL y optimizar la gestión de proyectos de e-SL. El desarrollo de la plataforma siguió un enfoque de desarrollo de software que priorizó el diseño fácil de usar, la accesibilidad y el soporte multilingüe. Las características clave incluyen gestión de usuarios con acceso basado en roles, funcionalidades de gestión de actividades, un algoritmo de emparejamiento inteligente, un *chatbot especializado en* recomendaciones e impulsado por IA y paneles de control para usuarios. Se espera que la implementación exitosa de esta plataforma mejore la eficiencia y efectividad de las iniciativas de e-SL, promueva la inclusión social y fortalezca la colaboración entre instituciones académicas y comunidades rurales.

Palabras Clave: Aprendizaje-Servicio electrónico, Plataforma de Emparejamiento, Participación Cívica, Impacto Comunitario, Emparejamiento Inteligente, *Chatbot* de recomendaciones, Comunidades Rurales.

Resum

Aquesta tesi presenta el desenvolupament d'una plataforma web d'aparellament dissenyada per fomentar la participació cívica mitjançant iniciatives d'*e-Service-Learning* (e-SL). La plataforma aborda la necessitat d'una major col·laboració entre universitats, professors, estudiants i organitzacions comunitàries rurals, particularment en comunitats desateses. Els objectius principals inclouen facilitar la connexió d'aquests actors, proporcionar recomanacions per a activitats d'e-SL i optimitzar la gestió de projectes d'e-SL. El desenvolupament de la plataforma va seguir un enfocament de desenvolupament de software que va prioritzar el disseny fàcil d'utilitzar, l'accessibilitat i el suport multilingüe. Les característiques clau inclouen la gestió d'usuaris amb accés basat en rols, funcionalitats de gestió d'activitats, un algorisme de d'aparellament intel·ligent, un *chatbot* especialitzat en recomanacions impulsat per IA i taulers de control per a usuaris. S'espera que la implementació exitosa d'aquesta plataforma millori l'eficiència i l'efectivitat de les iniciatives d'e-SL, promogui la inclusió social i enforteixi la col·laboració entre institucions acadèmiques i comunitats rurals.

Paraules Clau: Aprenentatge-Servei electrònic, Plataforma de Connexió, Participació Cívica, Impacte Comunitari, Aparellament Intel·ligent, *Chatbot* de Recomanacions, Comunitats Rurals.

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Dr. Joan Navarro Martín and Xavier Solé Beteta, who mentored and offered invaluable guidance throughout the entire project. Their insights and support were essential in the development of this thesis. In addition, I would also like to extend my thanks to Yuri González Azín, whose expertise and advice on the available web technologies, their respective strengths, and the most suitable options for this thesis project provided crucial direction during the initial phases of development.

Contents

1	Introduction	1
1.1	Background and Context	1
1.1.1	The Importance of Civic Engagement and Service-Learning.....	1
1.1.2	Challenges in Connecting Stakeholders	1
1.1.3	The Role of Technology in Facilitating Engagement.....	2
1.2	Project Overview.....	3
1.3	Problem Statement.....	3
1.4	Thesis Objectives	4
1.5	Research Questions	5
1.6	Limitations	5
1.7	Thesis Structure	6
2	State of the Art.....	7
2.1	Introduction to Service-Learning and the Emergence of e-Service-Learning.....	7
2.2	The Role of Digital Platforms in Facilitating e-Service-Learning	7
2.3	Intelligent Systems in e-Service-Learning: Recommendation Engines.....	8
2.4	AI-Powered Support: Chatbots in the e-Service-Learning Context	9
2.5	Bridging Gaps in Rural and Underserved Communities through e-Service-Learning Technology	10
2.6	Synthesis and Positioning of the CIVENHANCE Platform	10
3	Requirements Analysis	13
3.1	Introduction	13
3.2	Requirements Elicitation.....	13
3.3	Initial Project Understanding.....	13
3.4	Stakeholder Consultation	13
3.4.1	Question Formulation	13
3.4.2	Partner Feedback Collection	14
3.5	Requirement Consolidation and Decision Making	14
3.5.1	Analysis of Partner Responses.....	14
3.5.2	Tutor Discussions and Internal Decisions.....	14
3.5.3	Final Agreement with Partners.....	15
3.6	The SRS.....	15
3.6.1	Purpose.....	15
3.6.2	Product Scope.....	16

3.6.3	Product Overview	17
3.6.4	Functional Requirements (FR)	20
3.6.5	Performance Requirements	25
3.6.6	Usability Requirements	25
3.6.7	Interfaces Requirements	25
3.6.8	Security Requirements	27
3.6.9	Design Constraints.....	27
3.6.10	Software Quality Attributes	27
3.6.11	Verification	27
3.6.12	Assumptions and Dependencies	28
4	Platform Design and Development	29
4.1	Introduction	29
4.2	Technology Stack Selection	29
4.2.1	Guiding Principles for Selection.....	29
4.2.2	Front-End Development	30
4.2.3	Back-End Development	34
4.2.4	Database.....	38
4.2.5	Deployment Environment	39
4.3	System Architecture.....	40
4.3.1	Architectural Overview.....	40
4.3.2	Key Components and Their Interactions:.....	43
4.4	Database Design	46
4.4.1	Conceptual Data Model.....	46
4.4.2	Logical Schema Design (ERD).....	47
4.4.3	Rationale for Schema Design:.....	50
4.5	User Interface and User Experience Design	51
4.5.1	Core Design Philosophy	52
4.5.2	Platform Interfaces and Design Rationale.....	53
4.5.3	Accessibility Implementation Details	68
4.5.4	Multilingual Feature Implementation:	69
4.6	Core Functionality Implementation Details.....	70
4.6.1	User Management (FR.1, FR.2, FR.3, FR.4).....	70
4.6.2	Activity Lifecycle Management (FR.6, FR.7, FR.8, FR.9, FR.10)	73
4.6.3	Activity Interaction (FR.11, FR.12, FR.13, FR.14)	75

4.6.4	Group Management for Teachers (FR.5).....	77
4.6.5	Recommendation System and Chatbot (FR.15, FR.16)	78
4.6.6	User Dashboards (FR.17)	80
4.7	Security Implementation	81
5	Preliminary Evaluation and Testing.....	85
5.1	Introduction	85
5.2	Testing Setup	85
5.3	Unit Testing.....	85
5.4	Integration Testing.....	86
5.5	System Testing	86
5.6	Simulated Data Tests	89
5.6.1	Recommendation System Evaluation.....	89
5.6.2	Chatbot Evaluation	90
5.7	Limitations of the Evaluation	90
6	Discussion.....	91
7	Conclusions and Future Work	95
7.1	Summary of Key Findings and Conclusions	95
7.2	Contributions of the Thesis.....	96
7.3	Limitations of the Work	97
7.4	Temporal Costs	97
7.5	Future Work and Recommendations.....	99
7.6	Concluding Remarks	100
8	References.....	101
9	Appendices	103
9.1	Appendix A: Guide to set-up the environment and initialize the project.	103
9.1.1	Introduction.....	103
9.1.2	Prerequisites.....	103
9.1.3	Environment Setup.....	104
9.1.4	Running the Platform	106
9.1.5	Managing Translations	106
9.1.6	Managing Core Data Entities	107
9.1.7	Recommendation Service Configuration.....	108
9.1.8	Chatbot Configuration	109

Figures

Figure 1. Deployment diagram.....	18
Figure 2. Use case diagram.	20
Figure 3. Proposed wireframe of an student’s dashboard.	26
Figure 4. Component Diagram Illustrating Key Components and their Interactions.....	41
Figure 5. Database ERD.	48
Figure 6. Landing Page - Top Fold.	54
Figure 7. Landing Page - Bottom Section.	54
Figure 8. Documentation Page.....	55
Figure 9. Registration Page - Part 1.....	56
Figure 10. Registration Page - Part 2.....	56
Figure 11. Login Page.	57
Figure 12. Student Dashboard – Recommendations.	58
Figure 13. Student Dashboard - Applications & Enrolments.	59
Figure 14. Teacher Dashboard – Recommendations.	60
Figure 15. Teacher Dashboard – Approvals.	60
Figure 16. Teacher Dashboard - Student Engagement.	60
Figure 17. Community Partner Dashboard.	61
Figure 18. Activity Creation/Editing Page - Top Part.....	62
Figure 19. Activity Creation/Editing Page - Bottom Part.....	62
Figure 20. Activity Browser Page.....	64
Figure 21. Activity Details Page.....	65
Figure 22. Group Management Page.	66
Figure 23. Group Details Page.	67
Figure 24. Teachers Dashboard with the Chatbot Pop Up.....	68
Figure 25. Results of Student Cypress Test.	87
Figure 26. Results of Partner and Teacher Cypress Test.....	88
Figure 27. Temporal Cost Distribution Pie Chart.....	99
Figure 28. Contents of the .env file.	105
Figure 29. Translation.js structure.	106
Figure 30. Recommendation Service Execution on Initialization.....	108
Figure 31. Recommendation Service Scheduling.	108
Figure 32. Structure of faqData.js file.	110

Tables

Table 1. Frontend Stack Options Summary.....	33
Table 2. Backend Stack Options Summary.....	36
Table 3. Estimated Temporal Cost Distribution by Category.	98

Acronyms

SL: Service-Learning.

e-SL: e-Service-Learning.

HE: Higher Education.

SRS: Software Requirements Specification.

FR: Functional Requirements.

UI: User Interface.

UX: User Experience.

SPA: Single-Page Application.

TTI: Time to Interactive.

FCP : First Contentful Paint.

DOM: Document Object Model.

SSR: Server-Side Rendering.

SSG: Static Site Generation.

I/O: Input/Output.

NPM: Node Package Manager.

ORM: Object-Relational Mapper.

ASGI: Asynchronous Server Gateway Interface.

API: Application Programming Interface.

RESTful: Representational State Transfer.

URI: Uniform Resource Identifier.

JSON: JavaScript Object Notation.

RDBMS: Database Management System.

CTE: Common Table Expressions.

LLM: Large Language Model.

ERD: Entity-Relationship Diagram.

WCAG: Web Content Accessibility Guidelines.

JWT: JSON Web Token.

FAQ: Frequently Asked Questions.

XSS: Cross-Site Scripting.

1 Introduction

1.1 Background and Context

This section provides an overview of the context and foundational concepts driving this work, starting with the importance of civic engagement and service-learning, delving then into the challenges faced in connecting stakeholders within service-learning initiatives, and finally, it highlights the key role technology has in facilitating a greater engagement and overcoming these challenges.

1.1.1 The Importance of Civic Engagement and Service-Learning

Civic engagement, which is the active participation of individuals in their communities and societies, is widely recognized as a great indicator of a healthy society and a key driver of social progress. It encompasses a range of activities, from simple activities like voting and volunteering to more complex ones such as collaborating to address community needs or advocacy.

Within the realm of education, Service-Learning (SL) has emerged as a powerful pedagogical approach to foster civic engagement among students. Service-learning integrates community service with academic instruction and reflection, enriching the learning experience while addressing real-world issues.

This methodology offers numerous benefits for students, educational institutions and communities alike. Firstly, for students, it provides opportunities to apply classroom knowledge in practical settings, develop useful skills, like critical thinking or problem-solving and cultivate a sense of social responsibility. Educational institutions benefit from it by strengthening their ties with the communities, enhancing their reputation and developing more socially conscious graduates. Lastly, communities gain valuable support in addressing their needs, benefiting from the skills and efforts of both the students and the educational institution in question.

The rise of technology has significantly impacted various sectors, including education, and service-learning is no exception. The integration of digital tools and platforms with the traditional service-learning has led to the emergence of *e-Service-Learning* (e-SL), which expands the scope and accessibility of traditional service-learning, overcoming geographical barriers and enhancing inclusivity. This is particularly valuable for rural or underserved communities, which may have limited access to traditional service-learning opportunities. Through e-SL, students can engage with community partners remotely, allowing for collaboration on digital projects, online tutoring or virtual support for community initiatives.

1.1.2 Challenges in Connecting Stakeholders

Despite the recognized benefits of service-learning and the potential of e-SL, significant challenges remain in effectively coordinating and connecting the key stakeholders: students, teachers, and community organizations. One major obstacle is the lack of awareness of available opportunities. Community organizations may struggle to communicate their needs and projects

to educational institutions, while students and teachers may be unaware of relevant service-learning initiatives.

Moreover, difficulties in matching student skills and interests with community needs can burden the success of service-learning programs. Ensuring that students are placed in projects that align with their academic background and personal interests is crucial for maximizing their engagement and the value of their contributions. However, manually matching students with appropriate projects can be a complex and time-intensive task for teachers and program administrators.

1.1.3 The Role of Technology in Facilitating Engagement

Technology can help overcome these barriers by:

- **Centralizing communication and project coordination:** For instance, a dedicated platform can offer integrated group management in order to ease the process of organizing the people enrolled into a civic activity. This ensures all stakeholders involved into the same activity can be easily managed, thereby easing the whole process until the activity is finished.
- **Automating the matching of students to opportunities through recommender systems:** As an example, a recommendation engine can analyze student profiles, gathering their specific skills and interest to suggest projects that require the corresponding competencies or align with those interests. This targeted matching can significantly improve the relevance of opportunities presented to students and increase the likelihood of a successful and engaging experience. While reducing the time students will spend looking for a suitable and attractive activity.
- **Promoting accessibility and multilingual interaction:** By designing platforms in accordance with Web Content Accessibility Guidelines (WCAG), ensuring features like keyboard navigation, screen reader compatibility, and appropriate colour contrasts, and by incorporating multilingual support. Allowing users to interact with the platform and view content in their most comfortable language, broadening participation and fostering inclusivity among a diverse European student and partner base.
- **Enabling digital collaboration tools to sustain partnerships remotely:** For example, platforms can integrate project management dashboards and tools that allow all stakeholders to track progress, which is vital for e-SL models involving rural or geographically dispersed partners, making collaborations more feasible and sustainable.

To sum up, a well-designed digital platform that, through the use of technology, implements the previously mentioned features, like an intelligent matching algorithm, multilingual support, adherence to WCAG guidelines or group management tools, has the potential to significantly enhance the quality, reach, and impact of service-learning initiatives across educational institutions.

1.2 Project Overview

The CIVENHANCE project is a European initiative aimed at strengthening the capacity of Higher Education (HE) institutions to deliver effective e-SL programs, particularly in collaboration with rural communities. The project's primary objectives are to:

- Enhance the capacity of HE teachers to effectively guide students in e-SL projects with rural community partners.
- Foster civic engagement among students by providing them with meaningful e-SL experiences.
- Promote e-SL initiatives in rural communities, empowering students to actively participate in addressing community needs.
- Facilitate collaboration between universities, teachers and rural partners through the use of digital tools and platforms.
- Ensure the long-term sustainability of e-SL outcomes to support social inclusion, civic engagement, resilience, prosperity and sustainability in rural communities.

The project involves a consortium of university partners across Europe, including Univerzita Mateja Bela v Banskej Bystrici (UMB), Sveuciliste U Zagrebu (UNIZG), Libera Università Maria SS. Assunta Di Roma (LUMSA), University Ramon Llull – La Salle Campus Barcelona (URL), Pädagogische Hochschule Wien (PHW) and Universitat de les Illes Balears (UIB).

Central to the CIVENHANCE project is the development of a web-based e-SL matchmaking platform, the main focus of this thesis, designed to bridge gaps between HE institutions, students, teachers and rural community organizations by simplifying the discovery, management and participation in e-SL activities.

1.3 Problem Statement

Despite the recognized potential of e-SL, the adoption and sustained engagement of students in e-SL activities across European institutions remains limited due to key challenges such as the lack of centralized platforms for discovering or coordinating e-SL activities and the difficulty in matching students with projects that align with their skills, interests and academic goals.

Existing digital solutions for managing service-learning tend to be either institution-specific or lacking personalization features that support student engagement. In particular, recommendation systems, widely used in e-commerce and entertainment, have not yet been effectively applied to promote meaningful matches between students and service-learning initiatives in an educational context.

Therefore, there is a clear need for a technology-driven, scalable platform that:

- Centralizes e-SL activity management, streamlining the process of connecting community organizations, teachers and students, making it easier to discover, organize and participate in e-SL activities.
- Utilizes intelligent recommendation algorithms to improve the matching of students to activities, increasing student engagement and satisfaction, and ensuring that their skills and interests are effectively used.
- Focuses on accessibility and multilingual support to promote inclusivity, ensuring that a wider range of students can benefit from e-SL opportunities.

This thesis addresses the design, development and early evaluation of a web-based platform that aims to function as a marketplace for e-SL activities. The system will include an integrated recommender system to personalize activity suggestions for students and coordinators. However, since the platform is being developed as part of a broader European project with a longer timeline, its full deployment and empirical assessment will occur after the completion of this thesis. Consequently, there is a need to define a research approach that validates the system's design and functionality through simulated data, usability testing and feedback, providing a rigorous foundation for future deployment and impact evaluation.

1.4 Thesis Objectives

The primary objective of this thesis is to design and develop a prototype web platform that facilitates the discovery, recommendation and coordination of e-SL activities for students across European HE institutions. The platform is intended to act as a centralized marketplace for e-SL opportunities and make use of a personalized recommender systems to enhance student engagement and alignment with relevant projects.

Given the timeline of the broader European project within which this platform will be deployed, the thesis focuses on the pre-implementation phase, aiming to establish a solid foundation for the later deployment of system.

The specific objectives of this thesis are to:

1. **Identify and analyze the functional and user requirements** for an e-SL matchmaking platform through engagement with relevant stakeholders, culminating in the creation of a Software Requirements Specification (SRS).
2. **Design the architecture and user interface** of the web-based platform, ensuring usability, accessibility and alignment with the needs of students, teachers, and rural community partners.
3. **Design and implement a recommendation engine** capable of suggesting personalized e-SL activities based on user preferences and activity characteristics.
4. **Implement an AI Powered chatbot** capable of gathering user preferences and suggest e-SL activities based on them.

5. **Develop a functional prototype** that includes the core functionalities of the matchmaking platform, such as the activity marketplace, user management, integrated recommender system, and AI-powered chatbot.
6. **Conduct preliminary evaluations** of the platform's usability, functionality and recommendation logic using simulated data and scenario-based testing.
7. **Document the development process**, challenges and technical considerations, setting the groundwork for future research and deployment.

1.5 Research Questions

To guide the development and evaluation of the e-SL platform, the following research questions have been formulated:

- *RQ1*: What are the key functional, technical and user experience requirements for a web-based platform aimed at promoting e-SL engagement among European students?
- *RQ2*: How can user-centered design principles be applied to create an intuitive and accessible interface for browsing, managing and participating in e-SL activities?
- *RQ3*: What recommendation strategies are most suitable for matching students with appropriate e-SL opportunities based on their preferences, skills, and academic goals?
- *RQ4*: How effective is the prototype in terms of its designed usability, inferred perceived usefulness, and system logic, based on preliminary evaluations using synthetic data and scenario-based testing?
- *RQ5*: What technical, ethical, and practical challenges arise in the design of a recommender-based educational platform, and how can they inform future iterations and full deployment?

1.6 Limitations

Due to the development timeline of the overall European initiative, several limitations must be acknowledged:

- The platform will not be publicly launched during the timeframe of the thesis. Therefore, no live user data or large-scale empirical validation will be available.
- The recommender system will be evaluated through simulated scenarios and synthetic datasets, rather than real-world interactions and longitudinal user studies.
- The impact on student engagement and learning outcomes cannot be fully assessed at this stage and remains outside the scope of the current research.

- The evaluation focuses primarily on usability, perceived usefulness and system logic as judged by experts and simulated users, rather than on educational outcomes.

These limitations are clearly acknowledged to ensure transparency and to define realistic expectations for the contribution of this research within the ongoing development process.

1.7 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2, State of the Art**, reviews existing literature and practices in SL, the role of digital platforms in facilitating these initiatives, and the application of intelligent systems like recommendation engines and chatbots in educational contexts, particularly for connecting stakeholders and supporting rural or underserved communities. It also positions the CIVENHANCE platform within this landscape.
- **Chapter 3, Requirements Analysis**, details the process of analyzing the requirements for the e-SL matchmaking platform. This chapter focuses on the elicitation, documentation, and validation of the platform's functional and non-functional requirements, culminating in the development of the SRS.
- **Chapter 4, Platform Design and Development**, describes the design and development of the e-SL matchmaking platform. This includes the architectural design, the selection of the technology stack, API design, database schema design, user interface and user experience considerations, and the implementation details of core functionalities as defined in the SRS.
- **Chapter 5, Preliminary Evaluation and Testing**, presents the testing process and results from the preliminary evaluation of the platform, covering unit, integration, and system tests conducted using simulated data and scenario-based testing to ensure the platform's quality, reliability, and performance in a controlled development environment.
- **Chapter 6, Discussion**, evaluates and discusses the results obtained from the testing phase and the development process, analyzing the platform's effectiveness in meeting the research objectives, its usability, and its potential impact on facilitating connections and promoting e-SL engagement. It specifically addresses how the project outcomes align with the posed research questions.
- **Chapter 7, Conclusions and Future Work**, summarizes the key findings of the thesis, highlights the contributions and their significance, discusses the limitations of the current work, outlines the temporal costs involved in the project, and suggests potential directions for future research and development.

2 State of the Art

2.1 Introduction to Service-Learning and the Emergence of e-Service-Learning

Service-Learning is a pedagogical approach that integrates meaningful community service with academic instruction and reflection to enrich the learning experience, teach civic responsibility, and strengthen communities [1]. It is recognized as a high-impact practice in higher education, fostering academic achievement, personal growth, and civic engagement [2] [3]. Traditional SL involves students engaging in activities that meet identified community needs, allowing them to apply theoretical knowledge in real-world contexts and reflect upon their experiences to gain a deeper understanding of course content and their role in society.

The proliferation of digital technologies and online education has led to the evolution of SL into e-SL. E-SL combines the core principles of service-learning with online learning methodologies, allowing the service provided to occur partially or fully online [1]. This adaptation has expanded the reach and accessibility of SL, allowing students to participate regardless of geographical location, physical constraints, or scheduling conflicts [4].

E-SL manifests in various models, ranging from courses where instruction is online and service is on-site, to those where instruction is on-site and service is online, and "extreme" e-SL where both instruction and service are conducted entirely online [1] [5]. The benefits of e-SL are numerous, including increased accessibility, the potential to connect with a diverse range of community partners globally, and the development of digital citizenship skills [5] [6]. However, e-SL also presents challenges, such as ensuring fair access to technology, fostering effective three-way communication between instructors, students, and community partners, managing digital project complexities, and maintaining student engagement in virtual environments. Nevertheless, despite these challenges, research suggests that e-SL can be as effective, and in some cases more so, than traditional SL in achieving learning outcomes, although some studies indicate potential differences in areas like civic learning when compared to traditional SL [7] [2].

2.2 The Role of Digital Platforms in Facilitating e-Service-Learning

The effective implementation and scaling of e-SL initiatives highly depends on robust digital infrastructures. Therefore, digital platforms play a crucial role in bridging the gap between education institutions, students, and community organizations. These platforms can serve multiple functions, including:

- **Information Hubs:** Centralizing information about available e-SL opportunities, community needs, and resources.

- **Connection and Matchmaking:** Facilitating the discovery and connection of students with relevant projects, teachers with community partners, and organizations with academic support.
- **Project Management:** Providing tools for organizing, managing, and tracking the progress of e-SL projects.
- **Communication and Collaboration:** Offering channels for communication and collaboration between stakeholders.
- **Impact Tracking:** Enabling the documentation and assessment of learning outcomes and community impact.

Some platforms already exist to support community engagement and volunteer management. For instance, “GivePulse” is a platform that supports e-SL by enabling organizations to list opportunities, manage volunteers, track hours and impacts, and allowing users to search and filter for projects based on various criteria like skills and causes [4] . Other general community engagement platforms like Hivebrite [12] , Bettermode [13] , and Engagement Hub [14] offer tools for building online communities and managing engagement, though they may not be specifically tailored for the specific requirements of academic e-SL.

Despite the availability of such tools, a specific challenge highlighted in the problem statement of this thesis is the lack of centralized, dedicated matchmaking platforms that effectively connect HE institutions with rural community organizations for e-SL, and that incorporate intelligent matching based on a wide array of student and project attributes. The CIVENHANCE platform aims to address this gap by providing a specialized environment for e-SL matchmaking with a focus on rural communities.

2.3 Intelligent Systems in e-Service-Learning: Recommendation Engines

To address the complexity of matching students with suitable e-SL opportunities from a potentially vast pool of options, intelligent systems, particularly recommendation engines, hold great potential. Recommendation systems are widely used in many online domains, designed to predict user preferences and suggest items, like products, content or services, that are likely to be of interest. In the educational context, recommender systems are being explored to personalize learning pathways, suggest relevant learning resources, and guide students towards academic opportunities [8] .

Applying recommendation engines to e-SL involves matching student profiles, which includes their skills, academic interests, learning goals, availability, and potentially past experiences, with the characteristics and requirements of e-SL projects. In this context, common approaches to recommendation include:

- **Content-Based Filtering:** Recommends items similar to those a user liked in the past or based on a match between item attributes and user profile attributes. This method is particularly useful when detailed descriptions of users and items are available, as is the case with student profiles and e-SL project descriptions.

- **Collaborative Filtering:** Recommends items based on the preferences of similar users. This approach requires a substantial amount of user interaction data to be effective and can suffer from the "cold start" problem (the difficulty of making recommendations for new users or new items).
- **Hybrid Approaches:** Combine elements of content-based and collaborative filtering (and potentially other methods) to take advantage of their respective strengths and mitigate their weaknesses.

As noted in the problem statement of this thesis, the application of sophisticated recommendation systems to specifically promote meaningful matches between students and service-learning initiatives, particularly in a cross-institutional and multilingual e-SL context, is an area that needs further development. The CIVENHANCE platform's inclusion of an AI-driven matching algorithm represents an effort to innovate in this space by moving beyond simple keyword matching to a more sophisticated, attribute-based recommendation process.

2.4 AI-Powered Support: Chatbots in the e-Service-Learning Context

Artificial intelligence is increasingly being integrated into educational technologies to provide personalized support and enhance learning experiences. Chatbots are one of those applications, designed to simulate human conversation and assist users with various tasks. In educational settings, chatbots are used to:

- Answer frequently asked questions (FAQs) about courses, administrative processes, and campus services.
- Provide full time academic support and guidance.
- Offer personalized learning experiences and real-time feedback.
- Facilitate student engagement and reduce instructor workload by handling routine inquiries [9] [10] .

Within the e-SL context, chatbots can play a valuable role in supporting students throughout their engagement journey. They can provide instant information about e-SL principles, platform navigation, project application processes, and ethical considerations. Furthermore, as envisioned by the CIVENHANCE project, chatbots can extend to offering personalized recommendations for e-SL activities by interacting with students to understand their preferences and needs.

The benefits of using chatbots include their constant availability and ability to provide immediate responses. However, challenges also exist, such as ensuring the accuracy and relevance of information provided, developing robust natural language processing capabilities to understand diverse user queries, maintaining user privacy and data security, and addressing ethical concerns related to AI in education [9] . The integration of a chatbot in the CIVENHANCE platform aims to take advantage of these benefits to enhance student support and guidance in discovering e-SL opportunities.

2.5 Bridging Gaps in Rural and Underserved Communities through e-Service-Learning Technology

A significant advantage of e-SL is its potential to extend educational and civic engagement opportunities to rural and underserved communities that may face barriers to participating in traditional SL. In such cases, technology can act as a powerful enabler by:

- **Overcoming Geographical Distance:** Allowing students from urban HE institutions to collaborate with remote rural organizations without the need for extensive travel.
- **Increasing Accessibility:** Providing flexible learning and service options for individuals in communities with limited local partnerships or for students with mobility or time constraints.
- **Facilitating Resource Sharing:** Allowing rural organizations to take advantage of the skills, knowledge, and resources of universities and their students for community development projects.
- **Promoting Digital Literacy:** Engaging community members and students in technology-mediated projects can enhance digital skills for all participants, especially for rural community partners.

For e-SL technology to be truly effective in these contexts, platforms must be designed with inclusivity in mind. This includes ensuring multilingual support to meet the needs of diverse linguistic backgrounds within Europe and beyond, and adhering to web accessibility standards to make the platform usable by people with disabilities. The CIVENHANCE project's emphasis on these aspects is crucial for achieving the full potential of e-SL to foster social inclusion and strengthen collaborations between academic institutions and rural communities.

2.6 Synthesis and Positioning of the CIVENHANCE Platform

The state of the art in e-SL reveals a growing field that makes use of technology to expand the reach and impact of hands-on learning and civic engagement. Key trends include the development of various e-SL models, the emergence of digital platforms to support these initiatives, and the newly emerging exploration of AI-powered tools like recommendation systems and chatbots to enhance personalization and support.

However, gaps remain. While general community engagement platforms exist, there is a need for specialized platforms tailored to the unique requirements of e-SL, particularly those that facilitate connections between HE institutions and specific community partners, such as rural organizations. Furthermore, the application of intelligent matching algorithms and AI-driven support within the e-SL domain is still relatively underdeveloped compared to other online sectors.

The CIVENHANCE platform, as described in this thesis, is strategically positioned to address these gaps and contribute to the advancement of e-SL. It builds upon the existing understanding of e-SL pedagogy and technology by:

- **Providing a Centralized Matchmaking Hub:** Specifically designed to connect students, teachers, and rural community organizations for e-SL projects, addressing the need for a dedicated platform.
- **Incorporating an AI-Driven Recommendation System:** Aiming to improve the quality and relevance of recommendations between students and e-SL activities, moving beyond basic search and filter functionalities.
- **Integrating an AI-Powered Chatbot Recommender:** Offering personalized guidance and activity recommendations through a conversational interface, utilizing a LLM API for intelligent, context-aware matching.
- **Prioritizing Multilingual Support and Accessibility:** Ensuring the platform is inclusive and usable by a diverse European audience, with a particular focus on fostering engagement with rural communities.

By combining these features, the CIVENHANCE platform seeks to facilitate the coordination of e-SL initiatives, enhance student engagement, and foster more impactful collaborations between HE institutions and rural communities, thereby making a valuable contribution to the evolving landscape of technology-enhanced service-learning.

3 Requirements Analysis

3.1 Introduction

This chapter details the process followed to elicit, analyze and document the requirements for the e-SL matchmaking platform. The culmination of this process is the SRS, which serves as the foundational agreement for the platform's development.

3.2 Requirements Elicitation

The requirements elicitation phase is one of the most critical components of any software development project. As it involves gathering information from stakeholders to understand their needs and expectations, which are then translated into a detailed list of requirements. Therefore, a thorough and well-documented requirements analysis ensures that the final product aligns with the project's objectives and meets the user's needs. For this project, multiple steps were performed to ensure all stakeholder perspectives are properly comprehended and considered, in order to avoid any kind of error or mistake that might lead to delays in the development of the platform or to deliver a platform that does not align perfectly with the stakeholder's vision.

3.3 Initial Project Understanding

The first step performed was an initial and deep review of the CIVENHANCE project description. In order to get a general overview of the project's aims, which, as previously mentioned, include strengthening the capacity of HE institutions to deliver effective e-SL programs in collaboration with rural communities and the development of a web-based e-SL matchmaking platform. However, the initial description was broad and general, without specifying the functionalities or detailing operational aspects that the proposed platform should have. Therefore, a more in-depth information-gathering process was needed to accurately define the requirements.

3.4 Stakeholder Consultation

To bridge the gap between the general project vision and the specific platform requirements, a direct consultation process with project partners was initiated. These partners, representing various universities involved in the CIVENHANCE project, were key stakeholders directly involved in the project whose input was vital for shaping the platform and define key functionalities.

3.4.1 Question Formulation

A set of approximately 15 questions was formulated for the partners to answer. These questions were designed to address key aspects of the platform, including:

- **User Types and Roles:** Identifying all potential users (e.g., students, teachers, rural community organizations, administrators...) and defining their specific permissions and tasks.
- **Activities and Content Creation:** Determining who would be responsible for creating and uploading activities, the nature of these activities and any review or approval processes.
- **Matching Algorithm:** Understanding the desired criteria for the matching algorithm for students, and whether it should extend to matching teachers or community partners.
- **Platform Features:** Exploring the need for communication tools, user dashboards, language support and search functionalities.

3.4.2 Partner Feedback Collection

The formulated questions were then sent to each of the project partners. Partners provided their responses, often reflecting diverse perspectives and priorities based on their preferences, vision of the project and cultural differences. For example, regarding user types, most partners agreed on the three main categories: students, teachers and community organizations. However, for functionalities like feedback mechanisms or student autonomy in selecting activities, a range of opinions was expressed. This variety in responses underscored the importance of a subsequent consolidation phase.

3.5 Requirement Consolidation and Decision Making

The collected feedback from partners formed the basis for the next stage: consolidating the requirements and making informed decisions.

3.5.1 Analysis of Partner Responses

Each set of answers was carefully reviewed. Identifying the answers they agreed on, the questions where there were different opinions and unique suggestions worth considering. For instance, while there was general agreement on the types of users as previously said, the specifics of what each user could do, such as activity creation responsibilities or the necessity of an approval process for activities, showed varied opinions that needed reconciliation.

3.5.2 Tutor Discussions and Internal Decisions

The analyzed responses were then discussed in detail with my thesis tutors, who are also involved in the CIVENHANCE project. These discussions were crucial for evaluating the feasibility of different suggestions, ensuring their alignment with the project goals and technical constraints, and resolving stakeholder's conflicting viewpoints. Decisions regarding the

approach to each question were made aiming for a balanced solution that incorporated the most critical and widely supported features. For example, the decision to allow both teachers and community partners to propose activities, but with teachers having an approval role for activities created by community partners, was a result of such deliberations. Ultimately, the responses of the different stakeholders were carefully considered and evaluated, even in questions where there were no conflicts between the partners answers, like the question regarding the types of users to be in the platform. In which we realized the necessity for an additional user role: the administrator. This role was deemed essential to manage a verification process for teacher and community partner registrations, performed in order to mitigate the risk of malicious users attempting to register and disrupt the platform.

3.5.3 Final Agreement with Partners

After the internal deliberations and decisions with the tutors, the proposed final answers for each question were communicated back to the project partners. In order to have a final agreement in how to address each question and reaching a consensus among all stakeholders before proceeding with the formal SRS documentation, which can be seen in the following section: “3.6 The SRS”.

3.6 The SRS

3.6.1 Purpose

The purpose of this Software Requirements Specification document is to define the functional and non-functional requirements for the Civic Engagement Matchmaking Platform. The platform aims to connect students, teachers and rural organizations to promote civic engagement through e-Service-Learning initiatives. It will include features like activity creation, an AI-powered recommendation system and collaborative tools to enhance user interaction and project outcomes. AI-powered recommendations in contrast with simple matching or recommendation algorithms, instead of static keyword-based matching or simple attribute-based matching, the AI chatbot interacts with students, learning about their preferences, inferred interests and past participation. Introducing them to new opportunities they might not have considered.

This document adheres to the ISO/IEC/IEEE 29148:2018 standard, ensuring a clear, structured approach to defining requirements for all stakeholders involved.

3.6.1.1 Document Conventions

The document is based on the standard “ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering”. Achieving the goal that both partners and the developer team can understand and analyse what is expected to achieve.

3.6.1.2 Intended Audience and Reading Suggestions

This document is intended to be read by the developing team and the project stakeholders to ensure we follow the same path we agreed on before, that is the reason to follow the IEEE Standard so both parts can correctly interpret all the content.

The SRS document consists of five parts: in the first place, the Introduction, where it is presented the objective of the document and the software is going to be developed, among other general specifications. Therefore, the Overall Description, where more details about the main functionalities, users' characteristics and a general preparatory of the product are exposed. Further on, all the interface types, categorizing them into software, hardware, or user interfaces. Finally, Section 2 of the Specific Requirements Specification that encompasses both functional and non-functional requirements in written form or with diagrams.

3.6.2 Product Scope

In many rural and underserved communities, there is a lack of access to higher education initiatives aimed at fostering civic engagement and social inclusion. This gap results in missed opportunities for collaboration between universities and local organizations, leaving students with limited exposure to real-world, impactful learning experiences.

To address this, a software development approach is proposed to create a Civic Engagement Matchmaking Platform. This platform will leverage digital tools to facilitate service-learning initiatives by:

- Enabling rural community organizations to outline their needs and challenges.
- Allowing teachers to align their courses and student skills with these challenges.
- Empowering students to engage in meaningful projects tailored to their academic and personal growth.

The platform will use an AI-powered recommendation system to provide personalized suggestions, improving efficiency and satisfaction for all users. By centralizing these efforts, the platform will enhance the frequency and quality of collaborations, seeking to improve participation rates, project outcomes and the overall user experience in e-SL initiatives. The success of the project will be measured according to the quantitative indicators set on the project report.

3.6.2.1 Goals and objectives

The goals of this software platform are as follows:

- To connect students, teachers and rural community organizations in a seamless and collaborative environment for civic engagement.
- To provide an AI-powered recommendation system to match students and teachers with activities that align with their skills, interests and academic goals.

- To streamline the process for rural organizations and teachers to identify partners and initiate service-learning projects efficiently.
- To promote long-term social inclusion and resilience in rural communities through digital service-learning initiatives.
- To create an accessible, user-friendly interface that encourages active participation and collaboration.

This platform aims to set a benchmark for modern civic engagement by enabling impactful, technology-driven collaborations that benefit students, teachers and communities alike.

3.6.2.2 Partnership

The software module specified by this SRS will be piloted and tested by, at least, all the university partners belonging to the CIVENHANCE consortium:

- Univerzita Mateja Bela v Banskej Bystrici (UMB) – SK.
- Sveuciliste U Zagrebu (UNIZG) – HR.
- Libera Università Maria SS. Assunta Di Roma (LUMSA) – IT.
- University Ramon Llull – La Salle Campus Barcelona – ES.
- Pädagogische Hochschule Wien (PHW) – AT.
- Universitat de les Illes Balears (UIB) – ES.

3.6.3 Product Overview

3.6.3.1 Product Perspective

The Civic Engagement Matchmaking Platform will serve as a central hub for connecting students, teachers and rural community organizations through e-Service-Learning (e-SL) initiatives. This platform is designed to bridge the gap between academic goals and community needs by leveraging digital tools and an AI-driven recommendation system.

The platform is a web-based solution with three primary user groups: students, teachers and rural community partners. Each user group will have tailored access and features to ensure their specific needs are met. The system integrates a robust backend for data processing and matching, as well as a user-friendly frontend for seamless interaction.

The following UML deployment diagram, which can be seen in Figure 1, depicts the general architecture of the system, a standpoint about the project and the responsibility of each part of the product.

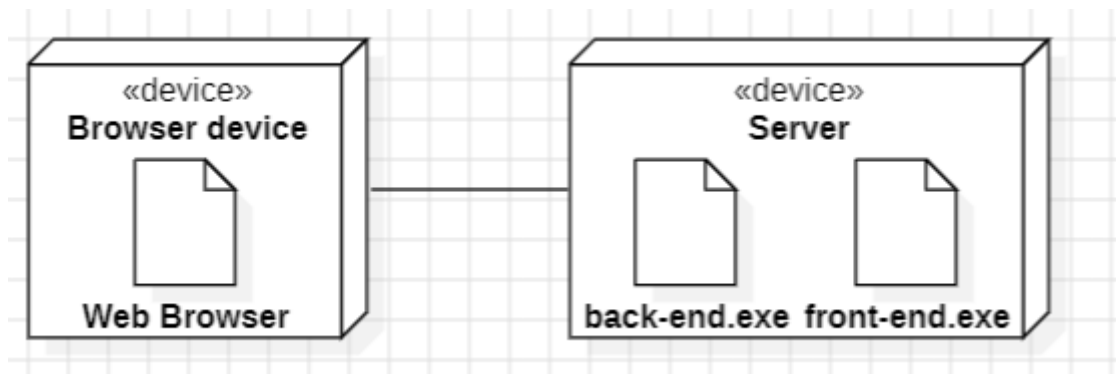


Figure 1. Deployment diagram.

The diagram shows the two devices involved to interact with the program, the left one for the browser (Browser device) and the right one (Server) where it will have installed both the front-end and back-end.

The following sections will explain thoroughly how this architecture pretends to work. Briefly say that the software is thought to stay only on a computer device and then a second device without the software installed, just a web browser (Web Browser) to interact with the web application that gets the information with a communication between the two devices.

3.6.3.2 Product Functions

The primary functionalities of the platform include:

- **User Management:** Separate registration and roles for students, teachers and community partners to ensure secure and appropriate access.
- **Activity Management:** Teachers and community partners can create, propose, edit and publish activities. And teachers and students can browse activities matching their skills and interests.
- **Chatbot Integration:** Conversational interface for collecting data and activity discovery.
- **Matching Algorithm:** Intelligent recommendation system to suggest activities based on user profiles, skills and preferences. Suggestions for teachers and community partners based on project needs and interests.
- **Dashboard:** Dashboards summarizing activities, preferences and engagement history.
- **Accessibility and Usability:** Multilingual support and compliance with W3C Web Accessibility standards.

3.6.3.3 User Classes and Characteristics

Students: Browse activities, receive recommendations that match their academic and personal interest and check information about their enrolled activity.

Teachers: Create, edit and manage activities; approve student participation or enrol students in activities. Align activities with course objectives, manage student involvement and connect with community partners.

Rural Community Partners: Publish activities or needs, connect with teachers to fulfil project requirements.

Administrator: Manage user registrations, specifically teachers and community partners, and verify them.

3.6.3.4 Operating Environment

The software will be a web-based application accessible via modern browsers (e.g., Chrome, Firefox, Safari). Therefore, being compatible with different platforms (e.g., Windows, macOS, Linux, Android...).

3.6.3.5 Design and Implementation Constraints

There are two major design and implementation constraints:

- The platform must support multiple languages and enable dynamic translation: Slovak, Croatian, Italian, Spanish, German and English.
- Accessibility must meet W3C Web Accessibility Initiative standards.

3.6.3.6 User Documentation

The platform will include a user guide, step-by-step instructions for students, teachers and community partners. All documentation will be available in the supported languages of the platform.

3.6.4 Functional Requirements (FR)

The details and responsibilities of each of the use cases, which can be seen in Figure 2, are explained in the following sections.

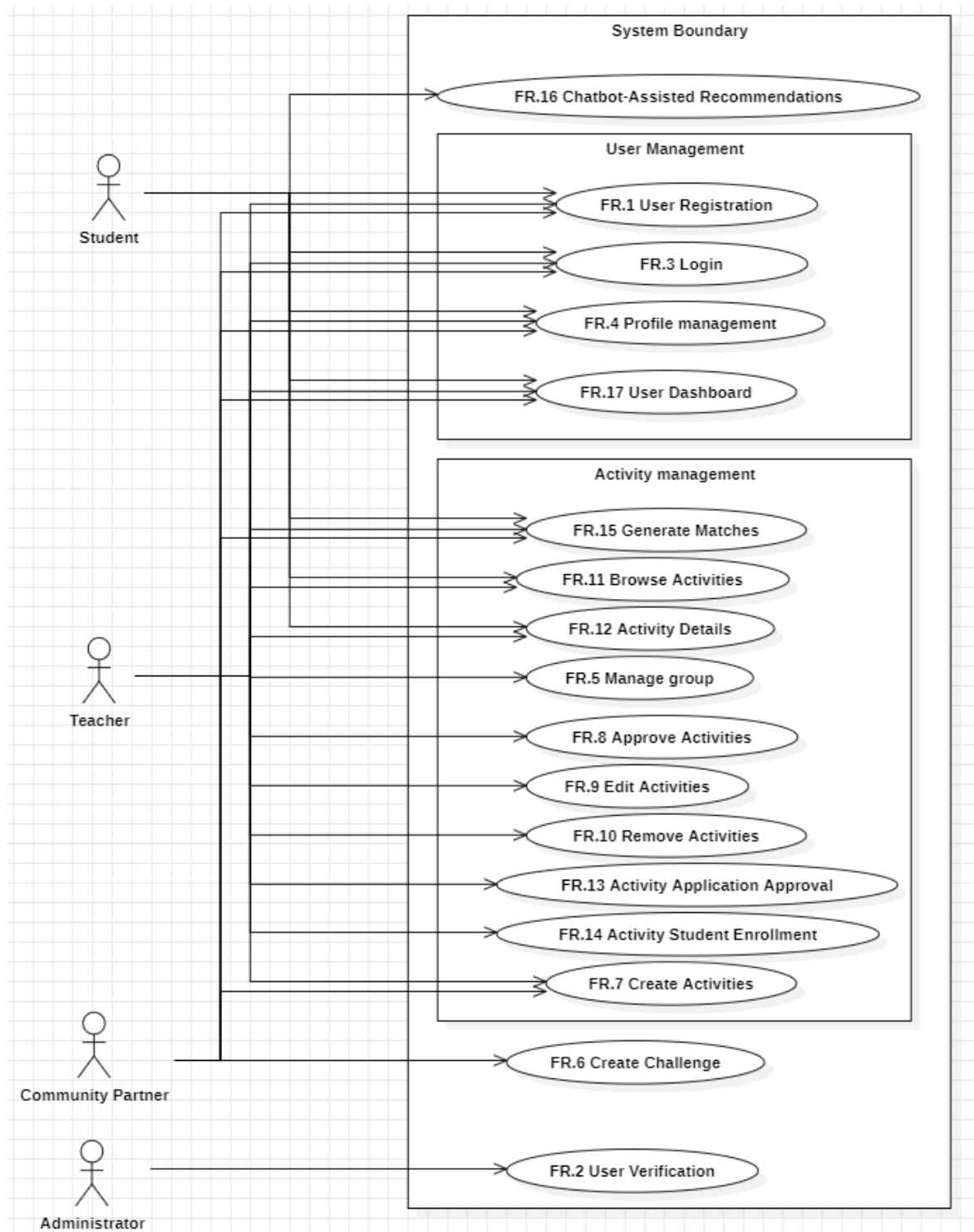


Figure 2. Use case diagram.

As previously seen and explained, the system will have different types of roles for the different users and each role will have different permissions and functionalities.

- **Students:**
 - Browse activities.
 - Interact with the chatbot for recommendations.
 - Pre-enrol in activities waiting for teacher approval.
- **Teachers:**
 - Upload, review, transform challenge into activities and manage activities.
 - Approve or reject student activity preferences.
 - Manage his students.
 - Connect through mail with community partners.
- **Community Partners:**
 - Upload challenges and needs.
 - Upload an activity or a draft of it.
 - Connect through mail with community partners.
- **Administrator:**
 - Approve register of teachers and community partners.

3.6.4.1 (FR.1) User Registration

Actor: All users

Application: Civic Engagement Matchmaking Platform

This functionality enables users to register on the platform. Students, teachers and community partners must provide details such as name, email, password, role and location. In addition, teachers and students must provide the university they belong to and add a set of tags regarding their interest and preferences. Finally, students will also have to add a set of tags regarding the skills they have.

3.6.4.2 (FR.2) User Verification

Actor: Administrator

Application: Civic Engagement Matchmaking Platform

The administrator will verify the different registrations of either teachers or community partners and will either approve or reject them.

3.6.4.3 (FR.3) Login

Actor: All users

Application: Civic Engagement Matchmaking Platform

Users log in with their credentials, email and password.

3.6.4.4 (FR.4) Profile management

Actor: All users

Application: Civic Engagement Matchmaking Platform

Allow users to update their profiles, by modifying the data entered during registration.

3.6.4.5 (FR.5) Manage group

Actor: Teachers

Application: Civic Engagement Matchmaking Platform

Teachers can create new groups. Specifying the name of the group, amount of student hours for the SL and finally they can add their students to the class. To add students, teachers will have a search bar where they can enter the email or name of the students and the system will show the registered users that coincide with the entered data, which the user will be able to select in order to add them to the class.

3.6.4.6 (FR.6) Create Challenge

Actor: Community partners

Application: Civic Engagement Matchmaking Platform

Community partners can create a challenge instead of an activity in case they don't fully know how to create the activity and fill all fields, tags... By creating a challenge, they only need to write what the civic activity they want to create is, the challenges it involves, the needs, skills, interests... Then a teacher can review the challenge and create the proper activity for it.

3.6.4.7 (FR.7) Create Activities

Actor: Teachers & Community partners

Application: Civic Engagement Matchmaking Platform

Teachers and community partners can create and propose activities by specifying details such as title, description, skills required, expected outcomes, duration, location and responsible contact information. Activities created by teachers do not need to be approved and teachers can create activities from a challenge proposed by community partners.

3.6.4.8 (FR.8) Approve Activities

Actor: Teachers

Application: Civic Engagement Matchmaking Platform

All activities created by community partners require approval by a teacher to ensure alignment with educational objectives and quality standards.

3.6.4.9 (FR.9) Edit Activities

Actor: Teachers

Application: Civic Engagement Matchmaking Platform

Allows teachers to update existing or proposed activities by modifying its contents or adding more content. If it is a proposed activity any teacher can edit it and approve it, but if it is an already approved activity, only the teacher that has approved the activity can modify its contents.

3.6.4.10 (FR.10) Remove Activities

Actor: Teachers

Application: Civic Engagement Matchmaking Platform

Allows teachers to remove existing or proposed activities.

3.6.4.11 (FR.11) Browse Activities

Actor: Students & Teachers

Application: Civic Engagement Matchmaking Platform

Students can browse approved activities by searching by name, filtering by skills, interests, hours to do and other relevant parameters. While teachers can browse both for approved and proposed activities.

3.6.4.12 (FR. 12) Activity Details

Actor: Students & Teachers

Application: Civic Engagement Matchmaking Platform

Upon selecting an activity while browsing, both students and teachers will be able to check all its information. In particular students will also be able to apply for an activity they would like to do and wait for their teacher approval. While teachers will be able to approve proposed activities and edit or remove both proposed and approved activities.

3.6.4.13 (FR. 13) Activity Application Approval

Actor: Teachers

Application: Civic Engagement Matchmaking Platform

Teachers can check the activity applications of the students in their class/subject and can either approve or deny the application.

3.6.4.14 (FR. 14) Activity Student Enrolment

Actor: Teachers

Application: Civic Engagement Matchmaking Platform

Teachers can enrol the students in their class/subject into an approved activity.

3.6.4.15 (FR. 15) Generate Matches

Actor: All users

Application: Civic Engagement Matchmaking Platform

The algorithm matches students to activities based on specified attributes, such as skills, interests, hours to do, location... Teachers are also matched with activities that might be interesting for them based on their specified attributes, the attributes of one of their classes and activities recommended to their students. Finally, community partners are matched with universities or teachers based on their attributes.

3.6.4.16 (FR. 16) Chatbot-Assisted Recommendations

Actor: Students

Application: Civic Engagement Matchmaking Platform

A chatbot interacts with students in order to retrieve information about them and recommend activities based on their profiles, skills, interests, hobbies...

3.6.4.17 (FR. 17) User Dashboard

Actor: Students, Teachers & Community partners

Application: Civic Engagement Matchmaking Platform

Users will have a dashboard page which will show them relevant information related to them in a concise and easy way.

The student dashboard displays the following:

- List of recommended activities based on the matching algorithm.
- Status of submitted activity applications (pending teacher approval or enrolled).
- History of enrolled activities with brief summaries.
- Personal profile overview, including skills and interests.

The teacher dashboard displays the following:

- List of proposed activities waiting for their approval.
- List of activities where at least one of their students is enrolled
- Analytics and insights, such as the number of students engaged in an activity in their class/subject.

The community partner dashboard displays the following:

- List of proposed activities and challenges and their statuses (pending for approval or approved).
- Suggested teachers or universities related to the activity/es they proposed or their interests.

3.6.5 Performance Requirements

The web application should be able to support up to 150 concurrent users. Critical operations will be carefully implemented so they are as optimal as possible and are performed as fast as they can.

3.6.6 Usability Requirements

This requirement is related to the different software interfaces that the user will interact with. The Civic Engagement Matchmaking Platform webpage will be designed and built to accomplish a good User Experience and Usability, so the software is going to be easy to use for any person, following W3C Web Accessibility standards.

3.6.7 Interfaces Requirements

3.6.7.1 User Interfaces

There will be one user interface for each type of role in the system, the students, the teachers and the community partners. Each role has different functionalities and permissions, though different roles can have common functionalities. Below, in Figure 3, you can see a proposed example of how a student dashboard would be.

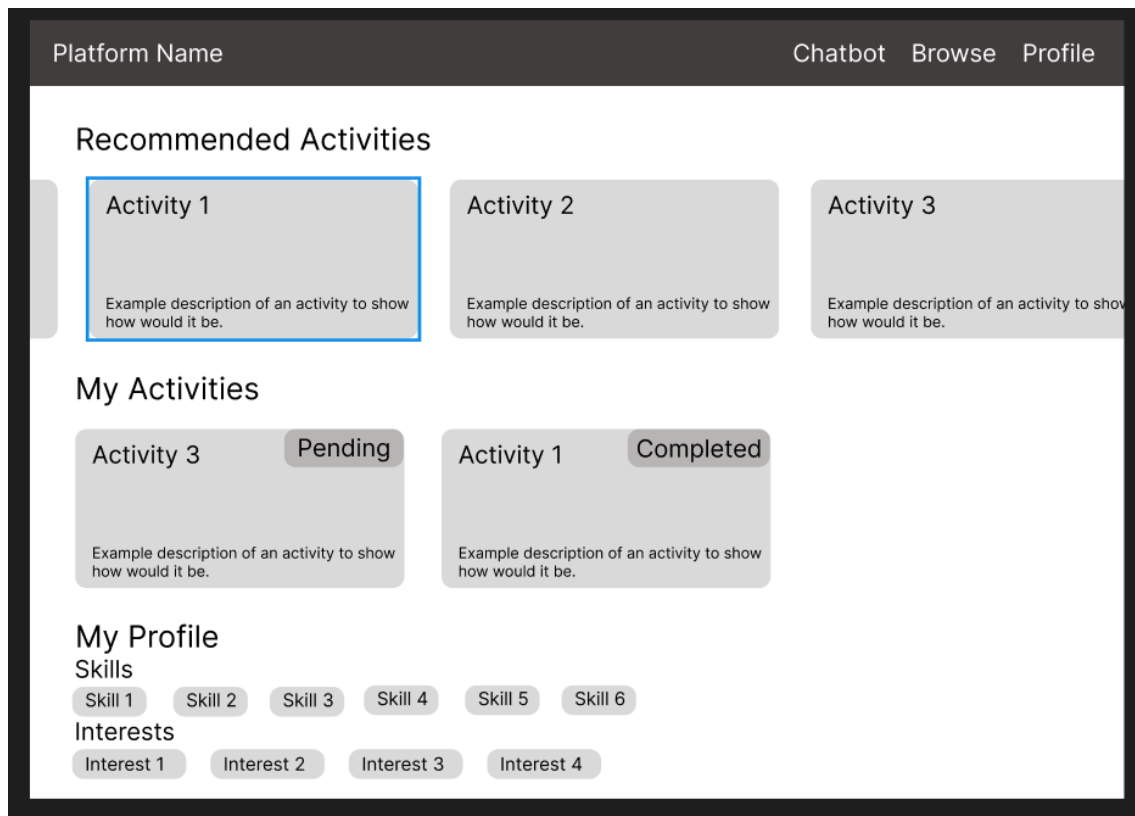


Figure 3. Proposed wireframe of an student's dashboard.

3.6.7.2 Hardware Interfaces

- **Device 1:** Teachers, students, and community partners using desktops or mobile devices to interact with the platform.
- **Device 2:** Server infrastructure managing data processing and storage.

3.6.7.3 Software Interfaces

The software is web-based and interacts via RESTful (Representational State Transfer) APIs to enable secure, real-time data exchanges between users and the system.

3.6.7.4 Communications Interfaces

The platform relies on secure HTTPS protocols for all data transmissions. In addition, the chatbot and matchmaking engine will integrate via an external API (Application Programming Interface) for enhanced recommendations.

3.6.7.5 Memory Constraints

All important data like passwords will be securely stored and encrypted.

3.6.8 Security Requirements

Encryption for sensitive communications will be implemented.

3.6.9 Design Constraints

The platform must comply with W3C Web Accessibility standards and the platform must implement all the languages of the different partners involved in the project and English. Finally, the platform must be easily maintainable.

3.6.10 Software Quality Attributes

- **Reliability:** The recommendations given by the system will be powered by AI and it might not be accurate for each user. Though machine learning and other methods used are not 100 % reliable. In addition, as recommendations are something subjective, even if the recommendations are totally accurate, they might not feel accurate to the user due to the subjective nature of human likes, dislikes, interest... It will be assessed that the recommendations are fair and accurate by checking how often students select recommended activities.
- **Maintainability:** Once the software is deployed to the public, the developer team is no longer in charge of solving problems or keep updating the software. The code will be commented and documented. Code convention will be taken into consideration while developing the platform according to the accepted standard for the technologies used.
- **Portability:** The software should be compatible with modern desktop and mobile browsers.

3.6.11 Verification

The software will meet the good practices in software development and the following types of tests will be performed:

- Unit tests. Every functionality defined in Section 2 will be tested and documented in an isolated way.
- Integration tests: Specific tests will be designed, implemented, and documented for all those functionalities that require interactions with other components.
- System tests: After running all the unit and integration tests, an end-to-end system test will be run to verify that the software meets the specifications of this document.
- Acceptance tests: A set of beta tests will be conducted over the partners of the project consortium to validate the acceptance of the software tool.

3.6.12 Assumptions and Dependencies

This document presents all the requirements that the partners have agreed and validated about all the information described. Any change requested by the partners will be evaluated separately unless it is a new functionality that will not be considered as the software will stay only with the functionalities described in section 2.6.4. So proposed changes can only affect the existing functionalities, always considering there is a previous agreement between partners.

4 Platform Design and Development

4.1 Introduction

This chapter presents a detailed exposition of the design and development processes undertaken for the Civic Engagement Matchmaking Platform. Conceived as an integral component of the CIVENHANCE project, the platform's fundamental purpose is to facilitate effective connections between students, teachers and rural community organizations. This is aimed at promoting and managing e-SL initiatives in HE, fostering a mutual relationship between educational pursuits and tangible community engagement.

The following sections of this chapter will provide a comprehensive understanding of the platform's technical realization. This includes an in-depth discussion of the architectural design decisions that support the system's structure and the arguments behind the selection of the specific technology stack for both front-end and back-end development. Furthermore, critical aspects of user interface (UI) and user experience (UX) design will be examined, with a particular emphasis on achieving usability, accessibility and a tailored experience for the distinct user roles. Finally, a significant portion of this chapter will be dedicated to articulating the implementation details of the core functionalities, as delineated in the SRS document exposed in the previous chapter.

4.2 Technology Stack Selection

The selection of an appropriate technology stack is a foundational step in software development, critically influencing the whole project's trajectory, maintainability and overall success. For the CIVENHANCE Matchmaking Platform, the choice of technologies was not arbitrary but was guided by a carefully considered set of principles. These principles aimed to ensure alignment with the project's specific requirements, while also taking into account broader software engineering considerations and learning objectives. In the end, for the front-end, the combination of Astro and React was chosen. Regarding the back-end, it was developed using Node.js together with Express.js, while PostgreSQL was selected for the database.

4.2.1 Guiding Principles for Selection

The primary drivers for technology selection were derived directly from the functional and non-functional requirements stipulated in the SRS and the general goals of the CIVENHANCE project.

One of the key considerations was the alignment with the project requirements. Given the platform's specification as a web-based application, the chosen technologies needed to be inherently suitable for a client-server architecture and facilitate seamless browser interaction. The SRS further indicated an API-driven architecture, requiring technologies that support robust RESTful API development and consumption for the communication between the front-end and back-end components. Performance was another key requirement, as the platform is expected to efficiently support up to 150 concurrent users, this particularly influenced the selection of the

back-end and database technologies. Furthermore, the requirement for multilingual support, encompassing Slovak, Croatian, Italian, Spanish, German, and English, required the use of technologies and design patterns that facilitate effective internationalization and localization. In addition, adherence to W3C Web Accessibility Initiative standards was a non-negotiable constraint, directly influencing front-end technology choices and development practices. Finally, the SRS highlighted maintainability as a crucial software quality attribute, leading to a preference for technologies that promote clean code, modularity, and comprehensive documentation to facilitate future updates and the resolution of future issues.

Beyond these specific project requirements, broader software development considerations also played a significant role. For example, scalability was a key factor, favouring technologies that could easily handle a potential future growth in the platform's user base. Development speed and efficiency were also valued, making technologies with robust frameworks and extensive libraries advantageous for accelerating the development lifecycle. Lastly, the strength of community support and the richness of the surrounding ecosystem were additional criteria considered. Therefore, technologies with active developer communities, ample documentation and a wide array of third-party resources were preferred for their utility in bug resolution, troubleshooting and extending functionality.

Finally, the developer familiarity with some technologies and its learning objectives were also carefully pondered. Existing proficiency in languages such as Python and Java provided a strong foundation for evaluating back-end development options and for implementing the recommendation logic. Nevertheless, this project was also viewed as a great opportunity to acquire new skills in widely adopted web technologies, even if they represented a newer area of exploration for the developer.

Based on those principles, the technologies considered for the CIVENHANCE Matchmaking Platform were selected. For the front-end, the Astro, React, Next.js, Angular and Vue.js were selected to then be carefully assessed before making the final decision. In the back-end, Node.js (with Express.js), Python (with its Django or Flask) and Java (with Spring Boot) were strong contenders and underwent detailed consideration. Finally, for the crucial database layer, PostgreSQL, MySQL and the NoSQL database MongoDB, were explored and evaluated against the platform's specific data storage needs.

4.2.2 Front-End Development

4.2.2.1 Chosen Technologies: Astro and React

The front-end architecture of the Civic Engagement Matchmaking Platform was strategically built using a combination of Astro and React. Astro served as the primary build tool and static site generator, responsible for the overall structure and delivering highly optimized, content-focused pages. React was then integrated for creating dynamic and interactive user interface components within the Astro framework. This dual-technology approach was chosen to take advantage of the strengths of each technology, addressing both performance requirements and the need for complex interactivity, while also aligning with technical and personal development goals.

More specifically, Astro was key in providing the high-performance, content-focused foundation, ensuring that largely static informational pages (e.g., landing page, lists of approved activities before filtering or individual activity detail pages) were delivered to the user with maximum speed and SEO-friendliness. React components were then selectively embedded as interactive "islands" on these Astro-generated pages precisely where dynamic functionality was necessary. For instance, a page displaying activity details might be mostly static HTML served by Astro, but the section for a student to apply for that activity, or for a teacher to edit it, would be a React component, loading its JavaScript only when needed. This development workflow involved defining these interactive regions and letting Astro manage the build process to integrate them seamlessly. This hybrid approach allowed the platform to benefit from Astro's superior static-site generation performance for content dense pages, while still making use of React's powerful data-binding, state management, and component model for the interactive user tasks. Consequently, the platform could properly meet its objectives of being highly performant and accessible for content consumption, while still being interactive and dynamic for user engagement and management tasks, without the typical trade-offs associated with using a single framework for all purposes.

4.2.2.2 Astro Rationale

Astro was selected primarily for its innovative "islands architecture" and its profound commitment to performance through a content-first approach. This architecture facilitates the creation of websites that are exceptionally fast by default, as it minimizes the amount of JavaScript sent to the client's browser for it to render. Unlike traditional single-page application (SPA) frameworks that often load a large JavaScript bundle upfront to render the entire page, Astro renders pages to static HTML at build time. Interactive elements, termed "islands," are then selectively hydrated with JavaScript only when they become visible or are interacted with by the user (a concept known as partial hydration). This "zero-JavaScript-by-default" paradigm for static content drastically reduces initial page load times, Time to Interactive (TTI), and First Contentful Paint (FCP), which are critical performance metrics directly impacting user experience and accessibility. For a platform targeting diverse users, including those in rural communities with potentially limited or unstable internet connectivity, such performance gains, even if small, are crucial, leading to lower bounce rates and improved user engagement. The content-focused nature of Astro also proves to be beneficial for a platform centered around presenting detailed information about e-SL activities, community challenges, user profiles, and more, as it ensures that this core information is delivered swiftly and efficiently.

From the developer personal development viewpoint, the adoption of Astro, despite the developer having no prior experience with it, was a deliberate choice. As Astro represents a significant paradigm shift in modern web development, moving away from JavaScript-heavy clients for many use cases. Engaging with this cutting-edge technology offered an invaluable opportunity for the developer to acquire expertise in a tool that is rapidly gaining traction and recognition for its performance benefits and developer experience. Its growing user base and bright future highlighted its importance even more. Moreover, Astro is framework-agnostic, meaning that it has no problem in allowing the integration of components from various UI libraries or frameworks (including React, Vue, and Svelte), providing a lot of flexibility for

developers to integrate different frameworks into one project, making it possible to take advantage of the different benefits of each framework. Overall, from a personal development point of view, this learning experience was deemed essential for keeping up with the latest and most effective methods in web performance and architecture.

4.2.2.3 React Rationale

Regarding React, it was chosen for its robust and well-established capabilities in building complex, stateful, and interactive user interfaces. Its core strength lies in its component-based architecture, which allows developers to break down complex UIs into smaller, isolated, and reusable pieces of code. For this platform, this meant that distinct UI elements such as cards for the activities or challenges, tag selectors for the interest and skill tags or forms could be developed independently and then composed to build up the different user interface where they are needed. This modularity not only enhances code organization and maintainability but also significantly speeds up the development process through the reuse of the components. React's use of a Virtual DOM (Document Object Model) is another key advantage, as it acts as an in-memory representation of the actual DOM. Therefore, when data changes, React first updates the Virtual DOM instead of the actual one, which enables react to then efficiently calculates the minimal changes needed to update the real DOM, thereby reducing costly direct manipulations and leading to smoother rendering of dynamic content, such as lists of activities or challenges that can be filtered or sorted, or dashboards that update based on user interactions. On top of that, React has a large and well-established ecosystem, along with strong community support. This combination results in a wide range of third-party libraries and tools. For instance, developers can use powerful state management solutions (e.g., Redux, Zustand, or React Context API itself for simpler cases), sophisticated routing libraries (like React Router for managing navigation within the interactive parts of the application), and numerous UI component libraries that can further accelerate development and ensure consistency.

In addition, from the developer point of view, this decision is further supported, as he already possessed some foundational familiarity with React and this project served as an ideal context to substantially deepen that knowledge and gain practical expertise. Moreover, given React's status as one of the most widely adopted front-end libraries globally, particularly in Western societies, enhancing proficiency in it was a significant personal development objective. The challenges presented by the platform, such as managing complex application state across different user roles, handling intricate forms with dynamic fields and displaying data in an engaging and interactive manner, provided a rich learning ground for mastering advanced React concepts and patterns, thereby aligning personal skill enhancement with the project's technical requirements for a sophisticated interactive layer.

4.2.2.4 Alternative Technologies Considered

Ultimately, while the Astro and React combination was deemed optimal and became the final choice, there were other front-end approaches which were carefully considered and could have become the final choice.

One such alternative was a pure React approach, for instance, using Create React App or Next.js in SPA mode. Although powerful for highly dynamic SPAs, this alternative would potentially lead to larger initial JavaScript bundles and slower first-load times compared to Astro's content-first methodology, which was a critical consideration for accessibility and performance. While Next.js, with its server-side rendering (SSR) and static site generation (SSG) capabilities, presented a closer alternative. Nevertheless, Astro's specific focus on its "islands architecture" and optimization for content-heavy sites offered a more targeted solution for achieving the desired performance characteristics.

Another framework evaluated was Angular, which as previously noted, is a comprehensive framework. However, its steeper learning curve and more opinionated structure were considered less suitable for a single developer aiming to integrate it with a newer technology like Astro, particularly given the project timelines. It was determined that the combined complexity would have outweighed the benefits within the context of this specific project.

Finally, Vue.js was the last alternative to be considered. Known for its gentle learning curve and flexibility, Vue.js could have been integrated with Astro much like React. Not to mention that, as with React, the developer already had some familiarity with Vue.js. Nevertheless, React's larger ecosystem, coupled with the developer's interest in React and its will to deepen its React expertise, rendered the React-Astro combination a more direct path to achieving the project's goals as well as the personal development objectives of the developer.

In conclusion, the strategic combination of Astro for its performance-oriented static generation and React for its dynamic component capabilities provided a robust, modern, and efficient solution for the platform's front-end, aligning with technical requirements and fostering valuable skill development.

To provide a better overview, Table 1 summarizes the key aspects of the chosen technology and the alternatives discussed.

Table 1. Frontend Stack Options Summary

Feature	Astro + React	Pure React / Next.js	Angular	Vue.js
Primary Suitability	Content-focused sites needing high performance & selective interactivity	Dynamic SPAs, applications requiring rich client-side logic	Large-scale, complex enterprise applications	Versatile, from small projects to SPAs
Performance (Initial Load)	Excellent (Astro's "zero-JS-by-default" and partial hydration)	Good (Next.js SSG/SSR helps); SPA mode can be heavier	Moderate (Can have larger bundle sizes)	Good (Generally lightweight)
Interactivity	High (Achieved via React "islands" on Astro pages)	Very High	Very High	High
Ecosystem & Community	Growing (Astro) + Very Large & Mature (React)	Very Large & Mature	Comprehensive, but more self-contained	Large and active
Developer Experience	Good (Astro's simplicity + React's power)	Good	Steeper learning curve, more opinionated	Gentle learning curve, flexible

Content-Heavy Sites	Excellent (Astro's core design focus)	Good (Especially with Next.js SSG)	Capable	Capable
UI Framework Agnostic	Yes	N/A	N/A	Yes
Developer Familiarity	Astro (New learning goal), React (Familiar)	Familiar	No familiarity	Some familiarity, but React preferred for deeper learning
Chosen Rationale Summary	Best balance of performance for content, dynamic interactivity, and developer learning goals for this project.	Powerful, but Astro offered better targeted performance for content-first aspects.	Considered too heavyweight and complex for a single developer and project timeline.	A strong contender, but React's larger ecosystem and specific learning goals made it more aligned.

4.2.3 Back-End Development

4.2.3.1 Chosen Technology: Node.js with Express.js

The back-end infrastructure, serving as the engine for business logic, data processing, API provision, and overall system coordination, was developed utilizing the Node.js runtime environment in conjunction with the Express.js framework. This critical decision was not made lightly, it derived from a careful evaluation of the technical features offered by the Node.js ecosystem for constructing contemporary web services, balanced with careful considerations pertinent to developer skill enhancement and project velocity.

4.2.3.2 Rationale

Node.js, distinguishes itself with its event-driven, non-blocking I/O (Input/Output) model. This architectural paradigm is exceptionally well-suited for building scalable network applications, particularly those that involve numerous concurrent connections and frequent, short-lived I/O operations, such as the RESTful APIs which is to be used for this platform (as specified in SRS Section 7.3, "Software Interfaces"). Unlike traditional blocking I/O models where a thread might wait idly for an I/O operation (like a database query or an external API call) to complete, Node.js uses an event loop. This allows it to handle many connections simultaneously with a single thread or a small pool of threads, significantly reducing the overhead associated with thread management and context switching. Consequently, its performance characteristics are excellent for I/O-heavy operations, which are typical in web applications that constantly interact with databases, file systems, and external services. This efficiency translates to lower latency and higher throughput for the platform's API, ensuring a responsive experience for front-end interactions even under moderate traffic.

Regarding Express.js, which is a minimal, unopinionated, and highly flexible Node.js web application framework. It was chosen to complement Node.js, as it provides a robust yet lightweight set of features for building web and mobile applications without covering up the underlying Node.js features or imposing a rigid structure. Particularly, Express.js excels at

simplifying the development of API routes, enabling clear and concise definitions for handling various HTTP requests. On top of that, its powerful middleware architecture is fundamental to its utility, allowing middleware functions to be easily chained to handle tasks such as request parsing (e.g., for JSON or URL-encoded data), user authentication and authorization (e.g., using JWTs or session management), logging, error handling, and security measures like setting HTTP headers. This modular approach allows for a clean separation of concerns and promotes reusable code. In addition, the extensive ecosystem of Express-compatible middleware available via npm (Node Package Manager) further accelerates development by providing already made and tested solutions for common back-end tasks.

Finally, a significant advantage of selecting Node.js and Express.js was the ability to use JavaScript across the entire technology stack. With Astro and React already chosen for the front-end, employing JavaScript on the back-end promoted a unified development environment. This "full-stack JavaScript" approach offered key advantages in terms of code consistency, the possibility of sharing utility functions or validation logic between client and server (with appropriate considerations), and a unified development mindset for the developer. This approach will accelerate the development process by streamlining workflows, reducing context-switching, and simplifying the overall learning curve when moving between front-end and back-end tasks.

From the developer's personal perspective, the choice of Express.js was particularly compelling and aligned with the learning objectives outlined earlier. While the developer possessed prior experience with other back-end programming languages and frameworks, Express.js represented an opportunity to delve into a new, highly relevant, and widely adopted back-end technology. Given the increasing global popularity of Node.js for building fast, scalable, and data-intensive web services, gaining practical proficiency in Express.js was identified as a valuable objective. This project provided an ideal, hands-on context to acquire practical experience in designing and implementing RESTful APIs following industry best practices while using Express.js. It also allowed for exploration of effective techniques for handling asynchronous operations (a core aspect of Node.js development using callbacks, Promises, and `async/await`), managing application configuration, structuring a larger Node.js application, and understanding the intricacies of the Node.js runtime environment and its module system.

4.2.3.3 Alternative Technologies Considered

Nevertheless, as with the front-end, before committing to the Node.js and Express.js combination for the back-end, several other prominent back-end technologies were carefully evaluated.

Python, with its frameworks Django and Flask, was a significant contender. Renowned for its elegant syntax, extensive standard library, and a remarkable ecosystem, especially in data science, machine learning, and artificial intelligence, presented an attractive option. This was particularly relevant given the platform's planned integration of AI. Django, as a high-level framework, offers an Object-Relational Mapper (ORM), an admin panel, and numerous built-in features that can speed up development for certain applications. Flask, in contrast, as a micro-framework, provides core routing and request handling, offering developers greater flexibility

to select their own tools and libraries. The developer's existing familiarity with Python and its rich ecosystem in data science and artificial intelligence further strengthened its candidacy. However, the project's nature as a primarily API-driven service, where efficiently handling many concurrent I/O-bound requests is crucial, led to a preference for Node.js. As, as previously said, its non-blocking, event-driven architecture often provides a more specialized and potentially more performant solution for such scenarios compared to traditional synchronous Python web frameworks, even though asynchronous capabilities in Python with ASGI (Asynchronous Server Gateway Interface) servers like Uvicorn are becoming increasingly mature. In addition, the appeal of a unified JavaScript stack also played a crucial role in this decision.

Another solid alternative considered was using Java together with the Spring Boot framework. This combination is well-regarded for building enterprise-grade, highly scalable, and resilient applications. Spring Boot, in particular, makes it much easier and faster to build stand-alone and production-ready Spring-based applications with very little initial setup. Additionally, it benefits from a wide range of helpful tools and resources, Java's strong typing, excellent support for microservices, and a focus on dependency injection and aspect-oriented programming. On top of that, the developer also had a high proficiency using Java and some foundational knowledge of the Spring framework. Nevertheless, Java applications, and Spring Boot in particular, generally have a larger memory footprint. Moreover, they can also involve a more verbose development process and longer build times compared to lighter alternatives like Node.js/Express.js, especially for small to medium-sized projects like the matchmaking platform which is to be developed. Finally, there was a realistic concern that the inherent overhead and potentially slower development iteration cycles associated with the Spring Boot ecosystem might negatively impact the project's timeline and agility. This made it a less ideal fit for this specific academic project, despite its undeniable strengths in enterprise environments.

To sum up, after carefully assessing the specific requirements for efficient API development, the inherent performance advantages of Node.js for I/O-bound operations, the benefits of a unified JavaScript front-end and back-end, and the valuable learning opportunity presented by Express.js, this combination emerged as the most appropriate and balanced choice for the platform's back-end architecture.

The following Table 2 offers a summary of the chosen backend stack together with the considered alternatives, highlighting the key factors that influenced the final decision.

Table 2. Backend Stack Options Summary

Feature	Node.js with Express.js	Python with Django/Flask	Java with Spring Boot
Primary Suitability	API-driven services, I/O-bound operations	General web dev, AI/ML integration	Enterprise-grade, highly scalable applications
Performance (I/O Focus)	Excellent (Event-driven, non-blocking I/O)	Good (Synchronous by default, ASGI maturing for async)	Good (Robust, but can have larger overhead)
Scalability	High (Efficiently handles concurrent connections)	Good	Very High (Designed for large, complex systems)
Ecosystem & Libraries	Very Large (npm, extensive middleware)	Very Large (PyPI, strong in data science/ML)	Extensive (Spring framework, many tools)

Development Speed	High (Flexible, lightweight framework)	High (Django for RAD, Flask for flexibility)	Moderate (Can be more verbose, longer build times)
Developer Familiarity	New learning opportunity (aligned with project goals)	Familiar	Very Familiar
Memory Footprint	Generally Lower	Generally Lower-Moderate	Potentially Higher
Unified JavaScript Stack	Yes	No	No
Chosen Rationale Summary	Best fit for API, I/O performance, unified JS stack, and learning objectives for this project.	Strong alternative, especially if AI/ML was the primary objective.	Powerful but considered overkill with potential for slower iteration for project scope.

4.2.3.4 API Design

The design of the API was carefully developed, following established RESTful principles to ensure a standardized, scalable, and maintainable communication layer between the front-end client, the Astro/React application, and the back-end server, the Node.js/Express.js application. One of the central ideas behind REST is to treat server-side data as resources that can be created, read, updated, and deleted. Consequently, all key entities within the platform, including users (students, teachers, community partners, administrators), e-SL activities, community-proposed challenges, and student groups, were clearly identified as resources, each addressable via a unique Uniform Resource Identifier (URI). For example, `/api/users`, `/api/activities`, `/api/challenges`, etc. Also, a key aspect of the RESTful design was maintaining statelessness for all server requests. This means that each request from the client to the server must contain all the information needed for the server to understand and process that request, without the server having to rely on any prior session state stored on its side.

In addition, in building the API, a crucial aspect was the consistent and semantic application of standard HTTP methods to interact with resources. Each method was employed precisely for its intended purpose, ensuring predictable behaviour and clear communication between the client and server. For instance, the GET method was consistently used for retrieving representations of resources. For example, a request like `GET /api/activities` would fetch a complete list of all activities available in the system, while `GET /api/activities/{activityId}` would retrieve all the details for a singular, specific activity. When introducing new resources, the POST method came into play, where sending a `POST /api/activities` request, accompanied by a JSON payload in the request body with the data of an activity, would create a new activity entry within the system.

For updates, two methods were employed: PUT and PATCH. PUT was used to entirely replace an existing resource. For example, `PUT /api/activities/{activityId}` would update the specified activity by replacing its entire data with the new information provided. However, for scenarios where only specific attributes of a resource needed modification, PATCH proved invaluable. A `PATCH /api/activities/{activityId}` request, with a JSON payload containing only the fields intended for change, allowed for precise, partial updates without requiring the full resource representation. Finally, to remove a resource from the system, the DELETE method was

employed. A straightforward request like `DELETE /api/activities/{activityId}` would efficiently delete the specified activity, ensuring data integrity.

Furthermore, data is exchanged primarily in JSON format, a lightweight, human-readable, and widely adopted standard for web APIs, making it easy for both client and server to parse and generate. Finally, HTTP status codes were used appropriately to indicate the outcome of API requests (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error), providing clear feedback to the client.

4.2.4 Database

4.2.4.1 Chosen Database System: PostgreSQL

For the persistent storage layer of the Civic Engagement Matchmaking Platform, PostgreSQL was selected as the relational database management system (RDBMS). This choice was supported by its reputation for reliability, robust feature set, and strong adherence to SQL standards, which were deemed critical for managing the structured and interrelated data in the platform.

4.2.4.2 Rationale

The decision to use PostgreSQL was primarily driven by its strong capabilities in ensuring relational integrity. The platform's data model involves numerous entities (users, activities, groups, challenges, applications, etc.) with complex relationships and dependencies. For example, a student belongs to a university and can apply for multiple activities; an activity is created by a teacher or community partner and requires specific skills. PostgreSQL's support for foreign keys, constraints (e.g., NOT NULL, UNIQUE, CHECK), transactions (ensuring ACID properties: Atomicity, Consistency, Isolation, Durability), and triggers allows for the enforcement of these relationships and business rules at the database level. All these features mentioned significantly contribute to data consistency and accuracy, reducing the likelihood of orphaned records or invalid data states.

PostgreSQL is also renowned for its extensibility and advanced features. It supports a wide range of data types, including arrays, JSON/JSONB (for storing semi-structured data like user preferences or tags), and geometric types, offering flexibility in data modelling. Moreover, its support for advanced indexing techniques (e.g., B-tree, Hash, GiST, GIN) is key for optimizing query performance, especially as the volume of data grows. Additionally, features like Common Table Expressions (CTEs) and window functions can simplify complex queries, making it easier to retrieve and analyze data for features like the recommendation engine or dashboard analytics. Furthermore, PostgreSQL's robust support for full-text search can be utilized for the activity browsing and search functionalities.

Finally, scalability and reliability are other core strengths of PostgreSQL. While the initial requirement is for approximately 150 concurrent users, PostgreSQL is known for its ability to handle significant workloads and large datasets, offering a clear path for future scaling if the platform's adoption grows. Its mature architecture, active development community, and proven

track record in production environments provide confidence in its long-term stability and reliability.

4.2.4.3 Alternative Database Systems Considered

In considering database systems for the project, other alternatives were evaluated. MySQL, another widely used open-source RDBMS, was the strongest contender. It is well-regarded for its ease of use, robust performance, particularly in read-heavy workloads, and its extensive community support. However, for applications demanding complex queries, strict data integrity, and support for advanced data types, PostgreSQL usually has an advantage. For this specific project, PostgreSQL's richer feature set, including advanced indexing capabilities, native JSONB support, and inherent extensibility, was considered more beneficial. These features were crucial for handling the diverse data requirements and the potential future needs of the matchmaking platform. While MySQL would have certainly been a viable choice, PostgreSQL ultimately proved more advantageous due to its advanced capabilities and stricter adherence to industry standards.

MongoDB, a popular NoSQL document database, was also briefly explored. Its schema-less nature provides considerable flexibility, which can accelerate initial development, especially for applications with rapidly evolving data structures. However, the core data for this platform is inherently relational, characterized by clearly defined entities and their interconnections. Managing these relationships and ensuring consistent data across various "documents" in MongoDB would necessitate significant application-level logic. This approach could potentially increase complexity compared to the inherent data integrity enforcement provided by an RDBMS at the database level. Furthermore, the developer lacked prior experience with MongoDB. Given the critical role of the database, opting for a more familiar RDBMS paradigm, with its strong emphasis on data integrity, was considered a more prudent approach for this project. In conclusion, a document-oriented database wasn't perceived to be as strong a fit as a relational one, particularly given the structured nature of the platform's core information.

4.2.5 Deployment Environment

The deployment strategy for the Civic Engagement Matchmaking Platform is centered around taking advantage of the existing university infrastructure, with a strong emphasis on containerization to ensure consistency and ease of management.

The platform operates on a client-server model, as outlined in the SRS, where user browsers interact with a central server. This server will reside within the university's IT system, providing a controlled environment and utilizing existing institutional resources. This machine will host the platform's distinct components: the Astro and React-based front-end application, the Node.js with Express.js back-end API service, and the PostgreSQL database.

To streamline the deployment process and guarantee a consistent environment from development to testing and ultimately to the university's production machine, Docker is the chosen containerization technology. Each primary component: the Astro/React front-end, the

Node.js/Express.js back-end, and the PostgreSQL database, will be packaged into separate Docker containers. This approach offers several significant advantages. Firstly, it effectively encapsulates each application and its dependencies, virtually eliminating the common problem of the platform working in the developer's machine but not in the deployment infrastructure. In addition, Docker containers are inherently lightweight and portable, which greatly simplifies deployment onto the university's server infrastructure, regardless of its underlying operating system details, as long as Docker is supported, which is the case. Furthermore, containerization facilitates easier scaling of individual components if future growth demands it, and it simplifies version management and rollbacks.

While the Node.js/Express.js application can directly serve HTTP requests, best practice in a production-like environment dictates placing a dedicated web server or reverse proxy, such as Nginx or Apache, in front of it. Nginx, in particular, can handle crucial tasks like SSL/TLS termination for HTTPS, manage load balancing if multiple back-end instances are required, serve static assets (like the HTML, CSS, and JavaScript generated by Astro) more efficiently, and implement robust request logging and security headers. For this platform, Nginx would likely be configured to route API requests (e.g., /api/*) directly to the Node.js/Express.js container while serving the static front-end assets.

The chosen technology stack is particularly well-suited for this deployment strategy. The Astro/React front-end, which builds down to static HTML, CSS, and JavaScript files, produces highly portable assets that can be easily served by any static web server like Nginx, or even packaged into a lightweight Docker container using a simple Nginx or http-server base image. Node.js applications, forming the back-end, are inherently ideal for containerization, as creating Dockerfiles for them is straightforward, and there is a rich ecosystem of official and community-supported base images available. The Express.js application can run as a standalone process within its container, listening on a specific port, with Nginx or another reverse proxy managing external access. Lastly, PostgreSQL also benefits from official Docker images, making it exceptionally easy to deploy and manage as a container. Docker Compose can then be used to define and orchestrate this multi-container application, linking the front-end, back-end, and database services and establishing persistent data volumes for the database to ensure data is not lost when containers are stopped or restarted.

This deployment approach, utilizing university hardware and Docker for containerization, aims to provide a stable, manageable, and scalable environment for the Civic Engagement Matchmaking Platform. While subject to final confirmation based on university IT policies and available resources, it represents a robust and modern strategy.

4.3 System Architecture

4.3.1 Architectural Overview

4.3.1.1 Architecture Description

The Civic Engagement Matchmaking Platform is fundamentally designed based on a Client-Server architectural model. This model, as depicted in the SRS (Figure 1), involves client

applications (web browsers on user devices) making requests to a central server that hosts the application logic and data. This server, in turn, processes these requests and returns responses to the clients. This architecture naturally aligns with the web-based nature of the platform and the distinct roles of the front-end and back-end technologies chosen.

More specifically, the architecture can be viewed as a three-tier structure:

- **1. Presentation Tier (Client-Side):** This is handled by the user's web browser, running the front-end application built with Astro and React. Its primary responsibilities include rendering the user interface, capturing user input, performing client-side validation, and initiating API requests to the application tier.
- **2. Application Tier (Server-Side Logic):** This tier is implemented by the Node.js/Express.js back-end application. It houses the core business logic of the platform, processes requests from the presentation tier, enforces business rules, interacts with the data tier, and manages user sessions and authentication. The recommendation engine and chatbot integration would also logically reside or be orchestrated from this tier.
- **3. Data Tier (Server-Side Storage):** This tier consists of the PostgreSQL database, responsible for the persistent storage and retrieval of all platform data, including user profiles, activity details, group information, and application data.

While the previously mentioned deployment diagram in Figure 1 shows the physical arrangement of devices, the logical organization and interaction of the key components within the described three-tier structure are illustrated in more detail in Figure 4. This component diagram details the relationships and communication pathways between the frontend browser application, the web server, the backend API server (which includes the recommendation and chatbot service logic), the PostgreSQL database, and external services such as the LLM API.

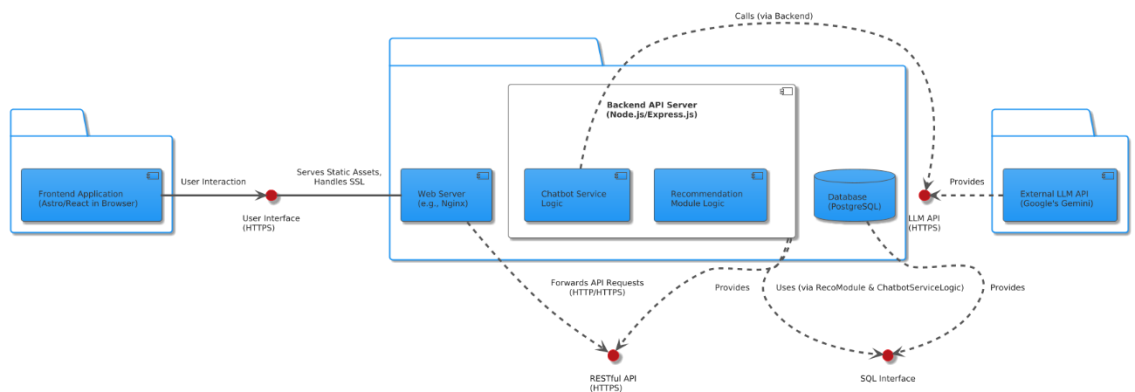


Figure 4. Component Diagram Illustrating Key Components and their Interactions

4.3.1.2 Justification for Chosen Architectural Style

The selection of a Client-Server, three-tier architecture for the Civic Engagement Matchmaking Platform was a deliberate choice, driven by several key project needs and the benefits it offers.

This model provides a clear and logical separation of concerns across the user interface (presentation), the business logic (application), and data storage (data). This separation is crucial easing the complexity of managing it, as each tier can be developed, modified, and maintained with a significant degree of independence. For example, a change to the UI on the front-end does not necessarily require modifications to the back-end business logic, and vice-versa. This aligns well with the distinct technology choices of Astro/React for the front-end and Node.js/Express.js for the back-end.

The distinct tiers also facilitate independent development and deployment. This means that development efforts can proceed in parallel if there are more than one developer, allowing the front-end developer/s to concentrate on UI/UX aspects, while the back-end developer/s can focus on the server-side logic and database interactions. While the RESTful API acts as a clearly defined contract, ensuring seamless communication between the two. In addition, even with a single developer on this project, this conceptual separation greatly aids in organizing the workload and thinking process. Furthermore, the ability to potentially deploy and update the front-end and back-end components independently offers high flexibility.

Regarding scalability, while the initial back-end is developed as a single Node.js/Express.js application, a modular monolith, the API-driven nature of this architecture lays a solid foundation for future growth. The application tier can be scaled by running multiple instances of the Node.js server behind a load balancer. The data tier, powered by PostgreSQL, also offers its own advanced scaling strategies, such as read replicas or sharding, though these are considerations for much larger loads that will probably not happen. Also, it's important to note that a full microservices architecture was consciously not adopted for initial development due to the added complexity for a single-developer project and the defined scope. However, this N-tier design doesn't prevent refactoring specific, computationally intensive, or independently evolving services, like the recommendation engine, into separate microservices in the future if the need arises.

Moreover, the modularity nature of this architecture also significantly enhances maintainability and testability. Issues can often be isolated to a specific tier, making debugging and updates far more manageable than in other architectures. It also improves testability, as each tier can be tested independently. For instance, unit tests can be run for back-end logic, UI tests for the front-end, and integration tests for API endpoints. In addition, this architecture allows for technology specialization, ensuring that the most appropriate tools are used for each tier's specific requirements. In this project case, as previously explained: Astro/React for rich user interfaces, Node.js/Express.js for efficient I/O and API development, and PostgreSQL for robust relational data management.

Finally, this architectural choice also aligns perfectly with the project's scope. For the currently expected user load of approximately 150 concurrent users and the defined set of functionalities, a well-structured Client-Server/N-tier architecture provides the necessary robustness and performance. Furthermore, it achieves this without introducing the initial overhead and complexity that would come with more distributed systems, such as a full microservices architecture.

In summary, the chosen architectural style offers a balanced approach, providing a clear structure, facilitating development and maintenance, and offering the possibility for future scalability, all while being appropriate for the current scale and requirements of the Civic Engagement Matchmaking Platform.

4.3.2 Key Components and Their Interactions:

- **Front-End Application:** Role in UI rendering, user interaction, API communication.
- **Back-End Application (API Server):** Role in business logic, data processing, database interaction, serving API requests.
- **Database:** Persistent storage of all platform data.
- **Recommendation Module:** Intelligent recommendation engine executed as a separate batch service within the backend.
- **AI Powered Chatbot Service:** Interacts with the front-end to gather user input. The back-end processes this input, first attempting to answer informational queries via a predefined FAQ. For activity recommendations or more complex queries not covered by the FAQ, it interfaces with a LLM API, providing contextual data to generate personalized suggestions or comprehensive answers.

The platform's architecture is composed of several key components, each with distinct responsibilities, interacting to deliver the overall functionalities of the platform. Obviously, these components map to the tiers previously described.

4.3.2.1 Front-End Application (Client/Presentation Tier)

The Front-End Application, serving as the client and presentation tier, is the user's primary window into the Civic Engagement Matchmaking Platform. Built using Astro for static site generation and React for dynamic interactivity, it's the point of contact for all users, students, teachers, community partners, and administrators alike.

This layer is responsible for rendering the entire user interface, ensuring all visual elements, information, forms, and dashboards are displayed accessibly and responsively across various devices. It actively manages user interaction, capturing inputs from forms (like those for registration, activity creation, or profile updates), handling button clicks, navigation, and other interactive events. In addition, to enhance efficiency and user experience, the front-end also performs client-side validation of user input, providing immediate feedback and reducing unnecessary trips to the server by checking for required fields or correct email formats.

Beyond just display and input, it handles state management, overseeing the local state of UI components and, for more complex scenarios, managing application-level state such as user authentication status or fetched data. A core function of this layer is the API communication, which makes asynchronous HTTP requests via RESTful APIs to the back-end application, sending user data (e.g., form submissions) and retrieving essential information (e.g., lists of activities,

user profiles, recommendations, etc.). It's also adept at processing API responses, including success and error states, and updating the UI accordingly. Finally, the front-end also houses and manages the chatbot interface, sending user messages to the back-end chatbot endpoint and displaying the AI's responses directly to the user.

4.3.2.2 Back-End Application (API Server/Application Tier)

The Back-End Application, built with Node.js and Express.js, acts as the central hub for all server-side operations, the true brain of the platform. Its core responsibilities are numerous and critical to the platform's functionality.

Primarily, it's responsible for API Provision, exposing a comprehensive suite of RESTful API endpoints that the front-end application consumes to send and retrieve data. These endpoints cover everything from user and activity management to group administration, challenge creation, application processing, and fetching recommendations. Inherent to this is the implementation of all business logic and workflows defined in the SRS, including the intricate details of user registration, secure login authentication and authorization, the activity approval processes, student enrolment in activities, and all necessary data validation that goes beyond what the client performs.

The back-end is also crucial for data processing, handling all manipulations, transformations, and calculations required by the platform's various features. Additionally, it serves as the sole intermediary for database interaction, constructing and executing SQL queries to create, read, update, and delete data from the PostgreSQL database, often taking advantage of an ORM or query builder for abstraction and enhanced security. On top of that, beyond its direct data handling, it plays a vital orchestration role, coordinating interactions with other essential services, such as initiating processes for the recommendation module or managing requests for the chatbot service. Finally, a critical responsibility is security enforcement, which involves implementing robust measures like input sanitization, protection against common web vulnerabilities (such as XSS, CSRF, and SQL injection), meticulous management of user sessions, and ensuring proper authorization for every API endpoint.

4.3.2.3 Database (Data Tier)

Finally, the PostgreSQL database stands as the authoritative source of truth, forming the platform's data tier and providing persistent storage for all its vital information. It meticulously houses user accounts and profiles for every type of user (students, teachers, community partners, and administrators), with their roles, permissions, contact information, skills, and interests. Beyond users, it stores the details of the different e-SL activities and community challenges, including their descriptions, required skills, locations, durations, and approval statuses. Information about teacher-managed groups and the students belonging to those groups is also diligently maintained here, alongside records of student applications to activities and their respective approval statuses. The database is also where generated recommendations for users are stored, and it keeps logs of critical interactions, such as chatbot conversations. Fundamentally, this tier ensures robust data integrity through its defined schemas, constraints,

and relationships, and it provides the necessary mechanisms for efficient data retrieval and updates across the entire platform.

4.3.2.4 Recommendation Module

The recommendation module, exemplified by the logic within the `recommendationService.js` file, operates as a distinct service orchestrated by the back-end. Its integration is designed for efficiency, rather than generating recommendations in real-time for every user request, which could be computationally expensive, it runs as a scheduled batch process, perhaps daily, to pre-calculate and store recommendations. The back-end application triggers this service, which then directly queries the PostgreSQL database to fetch essential data such as user profiles, activity details, skills, and interests. After processing this data and generating tailored recommendations, it writes these results back into a dedicated table in the database. Consequently, when the front-end requests recommendations, it retrieves these pre-computed suggestions via a standard API endpoint provided by the back-end. This asynchronous, batch-processing approach ensures that recommendation delivery to the user is remarkably fast, as it simply involves a quick database lookup. The regular scheduling, such as daily, guarantees that the recommendations remain reasonably up-to-date for users.

4.3.2.5 Chatbot Service

The Chatbot Service, managed by the back-end while users interact with it via the front-end, operates with a hybrid model approach. When a user engages with the chatbot interface, their messages are sent from the front-end to a specific API endpoint on the back-end. The back-end then, first attempts to find a relevant answer by searching a predefined FAQ dataset, generating rapid responses to common queries while reducing the monetary costs that making a call to a LLM (Large Language Model) API has. Then, if a satisfactory answer isn't found in the FAQ, or if the system detects the user is asking for activity suggestions or recommendations, the back-end then makes an API call to an external Generative AI service, specifically Google's Gemini. For informational queries, the prompt sent to Gemini is carefully contextualized to the CIVENHANCE platform and includes the user's message and preferred language. For recommendation requests, the prompt is enriched, it includes the user's full profile (containing their skills, interests, university, and other relevant data), their specific query, and the complete, up-to-date list of currently available activities in the platform. This allows Gemini to analyze the user's needs against the available opportunities and provide tailored activity suggestions.

The resulting response, whether from the FAQ or Gemini, is then relayed from the back-end to the front-end, where it's displayed in the chat interface. Moreover, as mentioned before, all interactions: the user's prompt, the bot's response, and the source of that response (FAQ or Gemini) are diligently logged to the database by the back-end for ongoing monitoring and future analysis. This hybrid approach efficiently handles common questions through the FAQ, while simultaneously taking advantage of the power of a LLM for more complex inquiries and personalized activity suggestions, all properly managed and mediated by the platform's robust back-end.

4.4 Database Design

The database is the foundation of the platform, responsible for the persistent storage and organized retrieval of all information. A well-designed database schema is crucial for data integrity, query performance, and the overall functionality of the application. This section details the conceptual data model, the logical schema design, and the rationale behind these design choices.

4.4.1 Conceptual Data Model

The conceptual data model defines the main entities involved in the Civic Engagement Matchmaking Platform, their attributes, and the relationships between them, based on the requirements outlined in the SRS.

At the heart of the system is the Users entity, designed to store information about all individuals interacting with the platform. Each user is uniquely identified and possesses attributes such as name, email (which must be unique across the system), a securely hashed password, and a defined role (student, teacher, partner, or admin) that governs their permissions and access. Additional user-specific information includes location and, for students and teachers, an affiliation with a specific university, which is an id referencing a universities entity that stores details about the participant academic institutions, including their name and location. Then, a crucial aspect for teachers and community partners is a verified status, ensuring legitimacy and that no malicious user registers with one of those roles that have elevated permissions. Furthermore, to capture the diverse capabilities and preferences of users, particularly students and teachers, the model incorporates Skills and Interests. These are implemented as separate reference entities (skills, interests), each containing a predefined list of terms. The many-to-many relationships between users and these attributes are managed through intermediary join tables: “user_skills” links users to their possessed skills, and “user_interests” links users to their declared areas of interest. This normalized approach allows for flexible tagging and efficient querying based on these criteria.

The core of the matchmaking functionality revolves around Activities and Challenges. The activities entity stores detailed information about e-SL projects, including title, description, expected outcomes, duration, location, and the user who created it. An important attribute is the approved status, indicating whether an activity is visible and available for student engage with or not. Additionally, community partners can initially propose challenges, which are stored in a dedicated challenges entity. These represent needs or project ideas that may later be transformed into full-fledged activities, a relationship indicated by the attribute “challenge_id”, which is a foreign key, in the activities table. Similar to users, activities are also associated with skills and interests through join tables (activity_skills, activity_interests) to facilitate matching.

For teachers managing students, the Groups entity allows for the creation of distinct classes or project teams. Each group is linked to a teacher and can have specific requirements, such as the minimal number of hours required. Also, the association of students with these groups is managed by the “group_students” join table, establishing a many-to-many relationship.

Regarding the student engagement with activities is captured through two primary entities: Activity Applications and Activity Enrolments. The Activity Applications entity records the expression of interest of a student in a specific activity, including its status (e.g., pending, approved, rejected). Then, when a student application is approved by their respective teacher, it is stored in the Activity Enrolments entity, recording which teacher enrolled the student, which student was enrolled, in which activity and when. Both tables link directly to the users and activities entities.

Moreover, to support the platform's multilingual requirement, several entities exist to store the translations of dynamic data within the system (e.g., Skill Translations, Interest Translations, University Translations, Challenge Translations, Activity Translations). These entities, as said, store the translated text for various fields (like names, titles, descriptions) for the different languages supported by the platform, linked back to the primary entity's ID. This design allows the platform to present information in the user's preferred language.

Finally, the system also includes entities for operational and analytical purposes: the Matches and Chatbot Logs entities. The Matches entity is designed to store the outputs of the recommendation algorithm, linking users to recommended entities (like activities or other users) along with a score and reason for the match. The Chatbot Logs entity records interactions with the chatbot, capturing user prompts, system responses, and the source of the response (FAQ or AI model), which is valuable for monitoring and improving the chatbot's performance.

These entities and their carefully defined relationships create a robust and normalized data structure. This structure is designed to prevent data redundancy, ensure consistency, and efficiently support the varied querying and data manipulation needs of the matchmaking platform's functionalities, from user registration and profile management to activity browsing, application processes, and the generation of personalized recommendations.

4.4.2 Logical Schema Design (ERD)

The conceptual data model described previously is translated into a logical schema, visually represented by the Entity-Relationship Diagram (ERD) which can be seen below in Figure 5. This ERD provides a detailed blueprint of the database structure, illustrating the tables, their columns (attributes), data types, primary keys, foreign keys, and the relationships between them. It directly reflects the structure of what will be the final schema that will define the actual database tables and relations.



Figure 5. Database ERD.

The ERD, as depicted in Figure 5, showcases the interconnected tables that form the platform's database. For example, the users table is linked to many other tables, reflecting its pivotal role. For instance, it has one-to-many relationships with groups (a teacher creates many groups), activities (a user creates many activities), and challenges (a user creates many challenges). Many-to-many relationships, such as those between users and skills, users and interests, activities and skills, activities and interests, and groups and students, are implemented using intermediary join tables (user_skills, user_interests, activity_skills, activity_interests, group_students). These join tables typically contain foreign keys referencing the primary keys of the two entities they connect.

4.4.2.1 Key Design Choices, Data Types, and Constraints

A fundamental design choice involved treating skills, interests, and universities as centralized reference points. Instead of duplicating this information across various records, dedicated tables were created for each: skills, interests, and universities. Each entry in these tables is given a unique id (a SERIAL PRIMARY KEY) and a name (a VARCHAR that's both UNIQUE and NOT NULL). This means, for example, "Project Management" as a skill exists only once in the skills table, and any user or activity possessing this skill simply references its unique ID. This approach not only prevents data redundancy and ensures consistency but also simplifies administration. Adding,

removing, or even modifying a skill, interest, or university becomes a straightforward operation within its respective table. Furthermore, to maintain data integrity, ON DELETE CASCADE constraints were applied to foreign keys in related "join" tables. This ensures that if a skill is removed from the skills table, all its associations with users (in, say, the user_skills table) are automatically deleted, preventing orphaned data.

Moreover, skills and interests associated with users and activities, as previously mentioned, are managed through many-to-many relationships, implemented using dedicated "join" tables like "user_skills", "user_interests", "activity_skills", and "activity_interests". Each of these tables utilizes a composite primary key, formed by combining the "user_id" (or activity_id) with the "skill_id" (or interest_id). This structure is highly efficient for querying and filtering users or activities based on multiple skill or interest criteria, which is absolutely crucial for the platform's core matching algorithm.

Another critical aspect of the platform's design is its multilingual capability, which for dynamic content like the one stored in the database, is achieved through the use of translation tables. For any entity whose content needs to be displayed in multiple languages, such as skills, interests, universities, challenges, and activities, a dedicated translation table was created. Each translation table links back to its parent entity via a foreign key. These tables include a column storing the language code (like 'de' for German or 'es' for Spanish) and columns for the translated text. A UNIQUE constraint on the combination of "entity_id" and "language_code" in each translation table ensures that there's only one translation per language for any given entity. This design effectively separates translatable content from the core entity data, keeping the main tables clean and organized. It also allows for the easy addition of new languages by simply inserting new rows into these translation tables without requiring any structural changes to the main entity tables.

Finally, the selection of data types and constraints played a significant role in ensuring data integrity and optimizing database performance.

- **Primary Keys:** Most tables use SERIAL for their id column, which automatically generates an incrementing integer, serving as the unique identifier for each record.
- **Foreign Keys:** Relationships between tables are enforced using FOREIGN KEY constraints. These are set with appropriate ON DELETE actions, such as CASCADE (to automatically delete related records if the referenced record is deleted) or SET NULL (if a related entity can exist even if the referenced one is removed).
- **Text Fields:** VARCHAR is used for text fields with a defined maximum length, while TEXT is reserved for longer, variable-length content like activity descriptions.
- **Boolean Values:** BOOLEAN is used for simple true/false flags like the verified column in the table users.
- **Timestamps:** TIMESTAMP is utilized to record creation and update times, often defaulting to CURRENT_TIMESTAMP.
- **Numerical Values:** INTEGER is used for numerical IDs and counts.

- **CHECK Constraints:** These are implemented to enforce specific value ranges or sets for certain columns, such as ensuring that the role column in the table users is one of the following: 'student', 'teacher', 'partner', or 'admin'.
- **UNIQUE Constraints:** These guarantee uniqueness for specific columns or combinations of columns, like the email attribute of users or the composite keys in join tables.
- **NOT NULL Constraints:** These ensure that essential fields always contain a value, preventing incomplete data.

In conclusion, these carefully chosen design elements contribute to a robust, normalized, and highly flexible database schema, well-equipped to support both the current functionalities and future additions or modifications of the Civic Engagement Matchmaking Platform.

4.4.3 Rationale for Schema Design:

The database schema, as detailed previously and visualized in the ERD of Figure 5, was meticulously designed not merely as a data repository but as an active enabler of the platform's diverse functionalities. The rationale behind its structure centered around robustly supporting core features, ensuring data integrity, and promoting operational efficiency. While some aspects of data integrity and specific design choices were touched upon in the previous section, this section further elaborates on how the overall schema serves the platform's objectives.

The schema's support for core platform functionalities is evident in its entity relationships. For instance, user management and role-based access are directly supported by the users table, where the role column dictates the user's capabilities and the verified flag adds a layer of trust, particularly for teachers and community partners. The entire lifecycle of e-SL opportunities, from ideation to completion, is managed through the challenges and activities tables. The link between them (via the challenge id) allows a seamless transition from a community-proposed need to a structured learning activity.

Furthermore, student engagement is properly tracked through the “activity_applications” table which captures a student's initial interest, while “activity_enrolments” signifies a confirmed participation, always facilitated by a teacher through the “groups” and “group_students” tables. This granular tracking is essential for both student and teacher dashboards. On top of that, the platform's ability to facilitate effective filtering and searching of activities is a direct result of the normalized structure for skills and interests, and their association with both users (via user_skills, user_interests) and activities (via activity_skills, activity_interests). This design allows for complex queries that can, for example, find activities matching a student's specific skill set, aligning with their interests, and located within their university's region.

Additionally, the generation of personalized matches and recommendations is heavily reliant on this structured data, which allows the recommendation service to efficiently query user profiles (including their skills, interests, and university affiliation from the users and related tables) and activity profiles (with their required skills and associated interests). The clear separation of these

attributes into linked tables allows the algorithm to compute accurate scores effectively. Moreover, storing these pre-calculated recommendations in the matches table, with a score and reason, is a deliberate design choice for performance, ensuring that users can quickly access their personalized suggestions without real-time computational delays. Furthermore, the design for multilingual support, through the dedicated translations tables for entities like skills, interests, universities, activities, and challenges, is essential for the platform's functionality. This structure allows the platform to retrieve and display content in a user's preferred language by joining the main entity table with its corresponding translation table based on a selected language code, ensuring a localized user experience.

Data integrity is one of the key parts of the schema design. The extensive use of primary keys for unique entity identification and foreign keys to define relationships between tables is fundamental. These foreign key constraints, often coupled with ON DELETE rules like CASCADE or SET NULL, automatically maintain referential integrity and prevent orphaned records. Also, UNIQUE constraints prevent duplicate entries, while NOT NULL constraints ensure that essential information is always present. CHECK constraints further enforce business rules at the database level, like validating a user role is within the allowed values. This rigorous application of constraints shifts much of the burden of data validation to the database itself, leading to a more robust and reliable system.

Efficiency considerations also guided the schema's development. While a high degree of normalization (evident in the separation of skills, interests, and translations into distinct tables) can sometimes lead to queries involving multiple joins, modern RDBMS like PostgreSQL are highly optimized for such operations, especially when appropriate indexing is in place. Primary keys and columns with UNIQUE constraints typically have indexes created automatically. For frequently queried columns involved in joins (foreign keys) or WHERE clauses, explicit indexing would be applied to significantly speed up data retrieval. In addition, the choice of appropriate data types (e.g., INTEGER for IDs, VARCHAR with sensible length limits, BOOLEAN for flags, etc.) minimizes storage overhead and improves processing speed. Finally, as previously noted, the batch processing of recommendations into the matches table is a key efficiency strategy for a more computationally intensive feature.

In essence, the database schema is designed to be a well-structured foundation that not only stores the platform's data but actively supports its complex functionalities, maintaining the quality and consistency of information, and allowing for efficient operations, thereby contributing directly to a positive user experience and the overall success of the CIVENHANCE project.

4.5 User Interface and User Experience Design

The design of the user interface and the overall user experience for the Civic Engagement Matchmaking Platform was approached with the understanding that a well-crafted interface is critical for user adoption, engagement, and the ultimate success of the platform in achieving its goals. The aim was to create an environment that is not only functional but also intuitive, accessible, and tailored to the diverse needs of its distinct user groups.

4.5.1 Core Design Philosophy

The design of the platform's user interface and experience was guided by a set of core principles, each integral to ensuring a seamless and effective user journey. These principles aimed to forge a user-centric platform that was not only robust in its capabilities but also welcoming and empowering for every individual who engaged with it.

4.5.1.1 User-Friendliness and Accessibility

At the forefront of the UI/UX design was a high commitment to user-friendliness and intuitiveness. In order to create a platform that is easy to learn and navigate for all users, regardless of their technical proficiency. This involved striving for clear information architecture, the consistent application of design patterns, and the creation of predictable interactions. Workflows for common tasks, such as registering, creating an activity, or applying for an opportunity, were designed to be as straightforward as possible, minimizing cognitive load and potential sources of confusion, thereby ensuring a straightforward experience.

Alongside with user-friendliness was a firm adherence to accessibility, specifically by adhering to the WCAG established by the W3C Web Accessibility Initiative, following guidelines such as implementing a robust keyboard navigability, ensuring ample colour contrast, and providing descriptive text alternatives for non-textual content when applicable. This was a non-negotiable aspect of the design, guaranteeing that the platform was usable by individuals across a broad spectrum of disabilities, including visual, auditory, physical, speech, cognitive, and neurological impairments.

4.5.1.2 Tailored Experiences and Multilingual Support

Given the variety of roles within the platform, a key focus was to offer experiences specifically designed for each role. Students, Teachers, Community Partners, and Administrators each possess distinct needs and responsibilities, and the platform was developed to address these differences. Upon logging in, users are presented with dashboards and navigation options precisely relevant to their specific functions. For instance, a student's dashboard intuitively highlights recommended activities and their application statuses, while a teacher's dashboard focuses on group management, approving student applications, and overseeing their own activities and new challenges proposed. This role-based approach optimizes the user journey by displaying the most pertinent information and tools for each user.

Furthermore, as a platform designed to serve a diverse European audience, multilingual support was a foundational design consideration. The interface was specifically engineered to effectively manage text expansion and contraction intrinsic to various languages, while also facilitating effortless language switching. Beyond technical considerations, the visual design was carefully developed to be culturally neutral and broadly appealing, ensuring that linguistic differences do not impede participation and that all users can interact with the platform comfortably in their preferred language, fostering inclusivity.

4.5.1.3 Ensuring Clarity, Consistency, and User Guidance

In addition, the principles of clarity and consistency were key in shaping the platform's visual identity and interactive elements. Visual uniformity in layout, typography, colour schemes, and interactive components was rigorously maintained across the entire platform, creating a cohesive and predictable user experience. Prioritization was given to the clear labelling of buttons, form fields, and navigation items to eliminate ambiguity and enhance user understanding.

An additional core principle guiding the UI/UX design was its alignment with the CIVENHANCE project's established visual identity. Significant efforts were made to ensure that the matchmaking platform's design, from its colour palettes and typography to its overall aesthetic, was harmonious with the main CIVENHANCE project website [11] . This deliberate visual coherence aims to provide users with a familiar experience as they navigate between various components of the broader CIVENHANCE initiative, reinforcing brand identity and fostering a sense of belonging to a unified project.

Finally, the system was designed to provide users with timely and relevant feedback and guidance. This included clear confirmation messages upon successful actions, such as form submissions, alongside explicit error messages when issues arose. Where necessary, additional guidance was incorporated through features like tooltips and instructional text, ensuring users were supported throughout their interactions with the platform.

This combination of principles aimed to create a user-centric platform that is not only powerful in its functionality but also welcoming and empowering for all its users.

4.5.2 Platform Interfaces and Design Rationale

This section delves into the specific design of key interfaces within the Civic Engagement Matchmaking Platform. For each critical view, the design choices, information hierarchy, and how these elements serve user needs will be discussed, supported by visual representations of the actual implemented interfaces.

4.5.2.1 Landing Page

The "**Landing**" page serves as the primary entry point for all users, whether new or returning. Its design, illustrated in Figure 6 and Figure 7, prioritizes immediate clarity regarding the platform's purpose and value proposition.

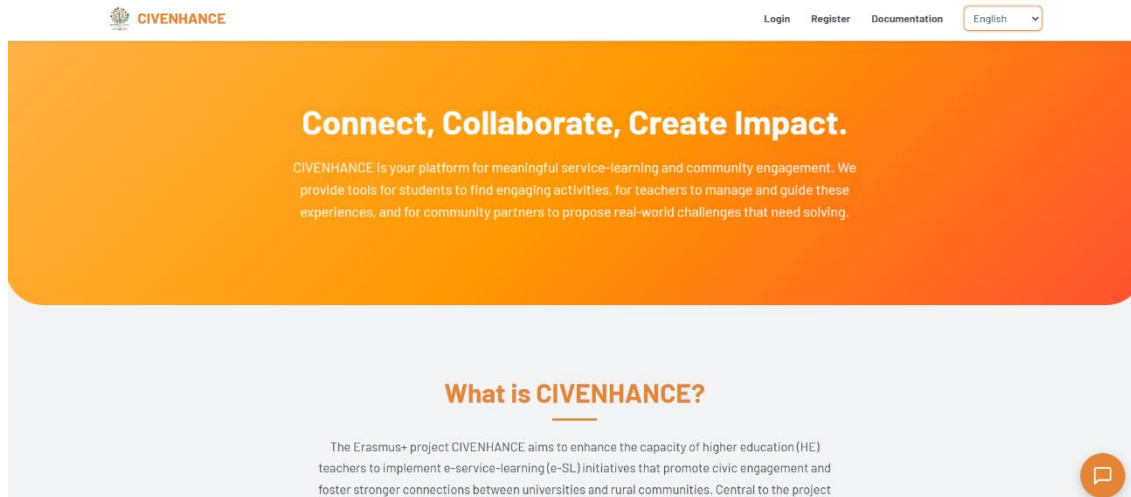


Figure 6. Landing Page - Top Fold.

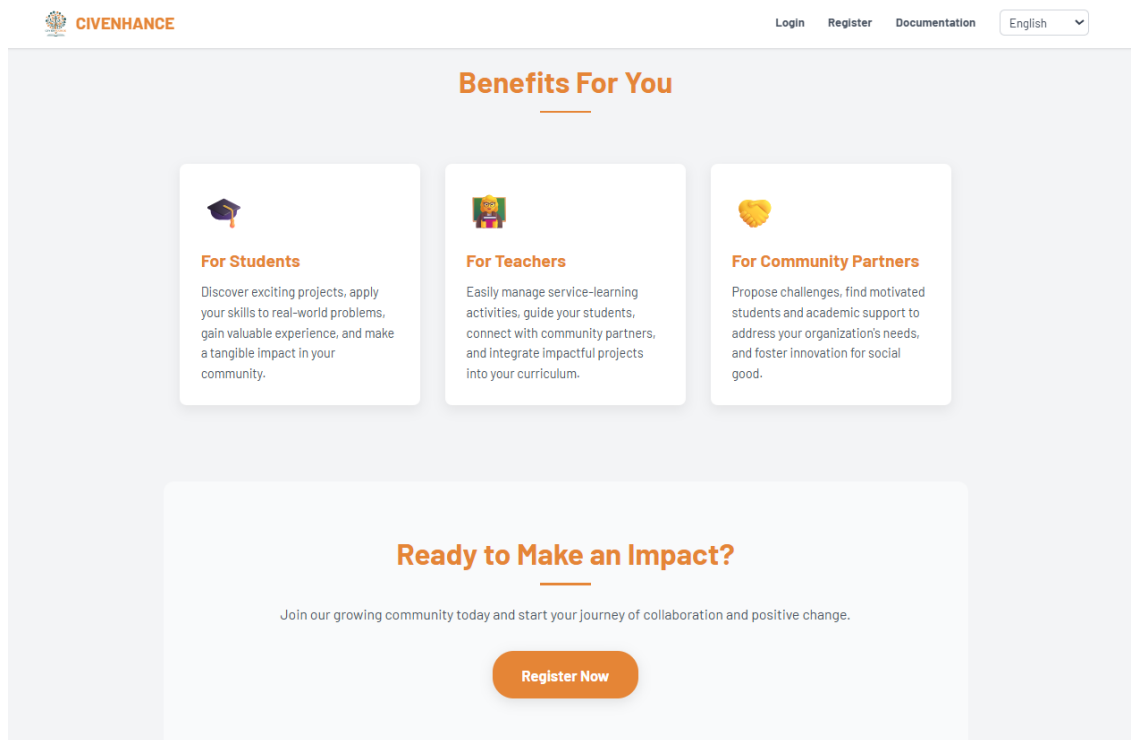


Figure 7. Landing Page - Bottom Section.

The page immediately draws users in with a prominent hero section, featuring the impactful tagline, "Connect, Collaborate, Create Impact." A concise subtitle then elaborates on the platform's fundamental function, swiftly capturing attention and conveying the essence of CIVENHANCE. The visual elements, including a dominant colour scheme of oranges and whites and carefully chosen typography, are all consistently aligned with the main CIVENHANCE project's branding, reinforcing its identity and ensuring a cohesive visual experience. Navigation is intentionally streamlined and accessible in the header, offering direct links to "Login,"

"Register," "Documentation," and a language switcher, all designed to meet immediate user needs.

As users scroll, the page unfolds into clearly defined sections. The "What is CIVENHANCE?" section provides essential context about the project and the platform's integral role within it. This is followed by the "Benefits For You" section, which utilizes a user-friendly card-based layout to distinctly highlight the advantages for various user groups: Students, Teachers, and Community Partners. Each card features a relevant icon and brief, targeted text, allowing different users to quickly grasp how the platform caters to their specific needs. The information is presented in a logical hierarchy, flowing from a general introduction to specific benefits, and culminates in a clear call to action: "Ready to Make an Impact?" This is prominently accompanied by a "Register Now" button, effectively guiding new users towards active engagement.

Ultimately, the Landing Page is designed to quickly orient visitors, clearly communicate the platform's purpose and its tailored benefits for diverse user roles, build trust through its strong alignment with the parent project, and provide intuitive pathways for core actions like registration or login.

4.5.2.2 Documentation Page

The "**Documentation**" page, illustrated in Figure 8, is designed to be a comprehensive resource for users seeking to understand the platform's functionalities in more detail.

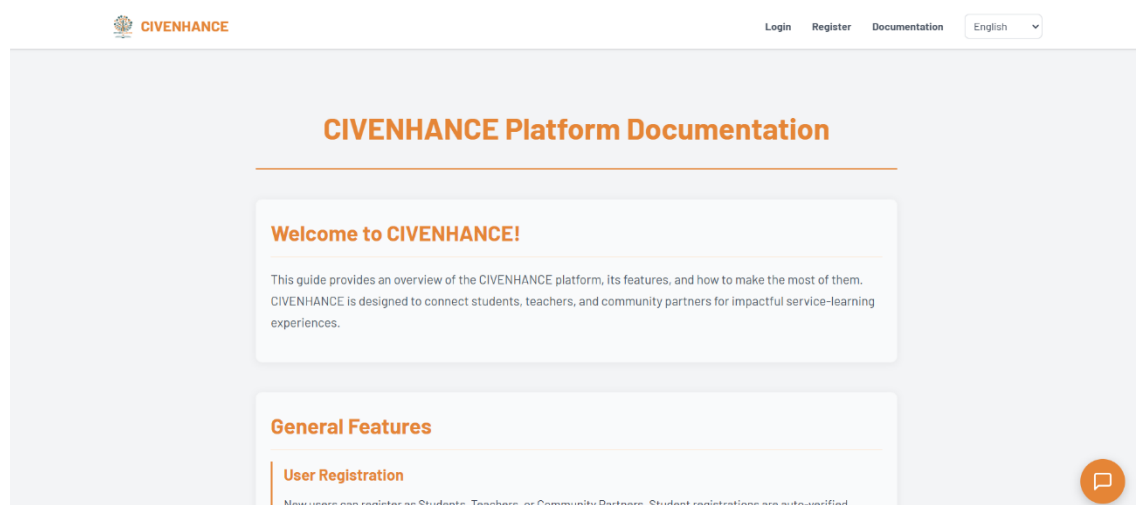


Figure 8. Documentation Page.

Its design prioritizes clear and hierarchical information. The page opens with a straightforward title, "CIVENHANCE Platform Documentation," followed by a welcoming introductory paragraph. Then, the content is thoughtfully organized into logical sections, including "General Features," "For Students," "For Teachers," "For Community Partners," and "Advanced Features." Within each of these sections, specific features are described using a consistent format, typically featuring a clear title and a descriptive paragraph. This modular and categorized approach enables users to easily navigate and find the information most relevant to their particular role or query. Furthermore, the overall layout of the page, including the header with main navigation

links and the language switcher, remains consistent with the rest of the platform, ensuring a familiar and seamless user experience.

This page has a crucial role in user support and guidance. It allows users to independently find information, understand how to perform specific tasks, and explore the full range of the platform's capabilities at their own pace. This self-service approach not only reduces the need for direct support but also empowers users to maximize their engagement with the platform.

4.5.2.3 Registration Page

The "**Registration**" page, illustrated in Figure 9 and Figure 10, is the gateway for new users to join the platform. The design prioritizes a clear, step-by-step process to minimize user friction and gather necessary information effectively.

Figure 9. Registration Page - Part 1.

Figure 10. Registration Page - Part 2.

The form itself is presented within a clean, centered container against a light background, ensuring the user's focus remains squarely on the task at hand. A prominent "Register" title

clearly heads the form. The initial and crucial step involves role selection ("I am a: Student, Teacher, Partner"), a choice that dynamically adjusts the subsequent form fields. Core information such as "Full Name," "Email Address," "Password," and "Confirm Password" are requested first. Additionally, clear labels and helpful placeholder text, like "Min. 8 characters" for passwords, guide user input. For those selecting "Student" or "Teacher," a mandatory "University" selection field becomes visible, populated with a list of participating institutions. An optional "Location" field allows users to specify their geographical area.

A significant and user-friendly aspect of the registration, particularly for students, involves selecting "Interests" and "Skills" using multi-select components. These choices are displayed in a tag-like format for immediate visual feedback, a more intuitive approach than lengthy checkbox lists. The form culminates with a prominent "Register" button. Error messages are designed to appear inline or near the relevant field for maximum clarity, and a success message is displayed upon completion. Moreover, for roles requiring administrator verification (Teachers and Partners), specific information regarding this process is provided. A clear link, "Already have an account? Log in," offers a convenient exit for existing users. To sum up, the overall information hierarchy progresses logically, moving from basic identity and credentials to role-specific details and personalization attributes.

Ultimately, the registration process is designed to be as intuitive as possible, guiding users through the required information based on their selected role. It efficiently captures essential data for account creation and profile personalization, including skills, interests, and university affiliation, all of which are crucial for the platform's matchmaking capabilities. Finally, by displaying conditional fields based on the chosen role, the platform prevents users from encountering irrelevant questions, streamlining their overall experience.

4.5.2.4 Login Page

The "Login" page, illustrated in Figure 11, provides a straightforward access point for registered users.

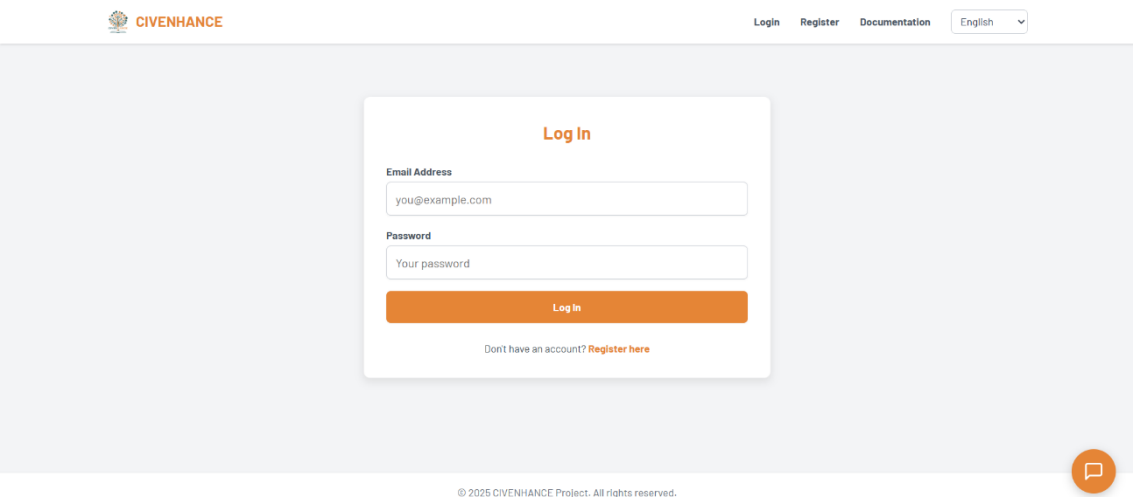


Figure 11. Login Page.

Much like the registration page, the login form is presented within a clean, dedicated container. It features a prominent "Log In" title and asks only for the essential credentials: an "Email Address" and a "Password." The input fields are clearly labeled and include helpful placeholder text. The primary call to action is a prominent "Log In" button. Furthermore, in the case that a new user accidentally lands on this page, a convenient "Don't have an account? Register here" link is provided just below the button, offering a clear path to registration. Moreover, in cases of incorrect credentials or attempts to access with unverified accounts, clear error messages are displayed to inform the user. The overall design is intentionally minimalistic, with its sole focus on the task of authentication to ensure a swift and efficient login process.

In essence, this page properly fulfills the need for existing users to securely and easily access the platform. Its simplicity allows users to quickly authenticate themselves and proceed to their respective dashboards or desired tasks.

4.5.2.5 Dashboard Pages

Upon successful login, users are directed to a personalized dashboard tailored to their specific role. While sharing a common visual language and navigation structure with the rest of the platform, each dashboard prioritizes information and actions most relevant to its user type. A consistent element across student and teacher dashboards is the "Recommended For You" section, which utilizes a carousel to present activity cards. These cards display key information such as title, a brief description, location, duration, required skills, and relevant interests, along with the reason for the match and a "View Activity" button.

The "**Student Dashboard**" page, illustrated in Figure 12 and Figure 13, is designed to be a central hub for students to discover new activities they will probably find interesting and manage their engagement.

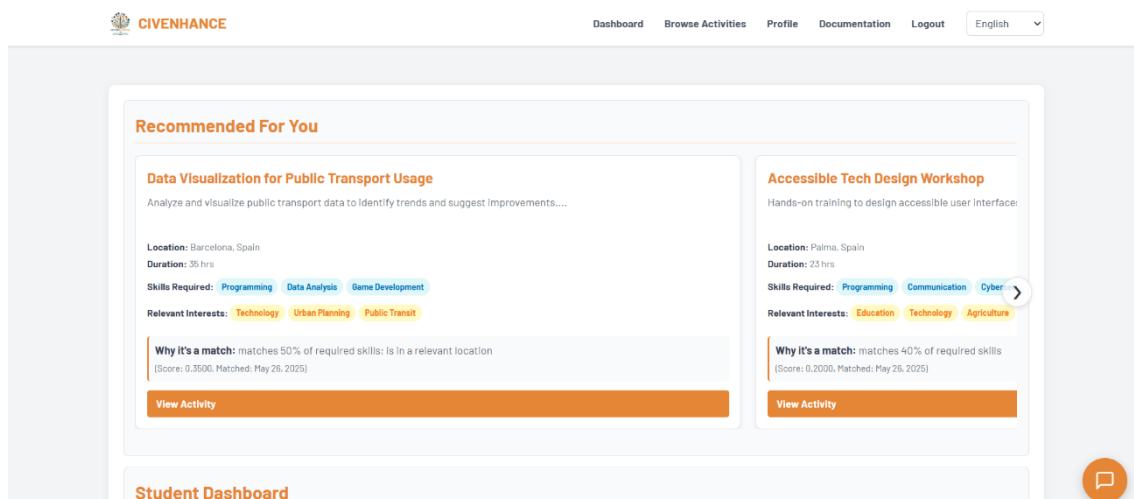


Figure 12. Student Dashboard – Recommendations.

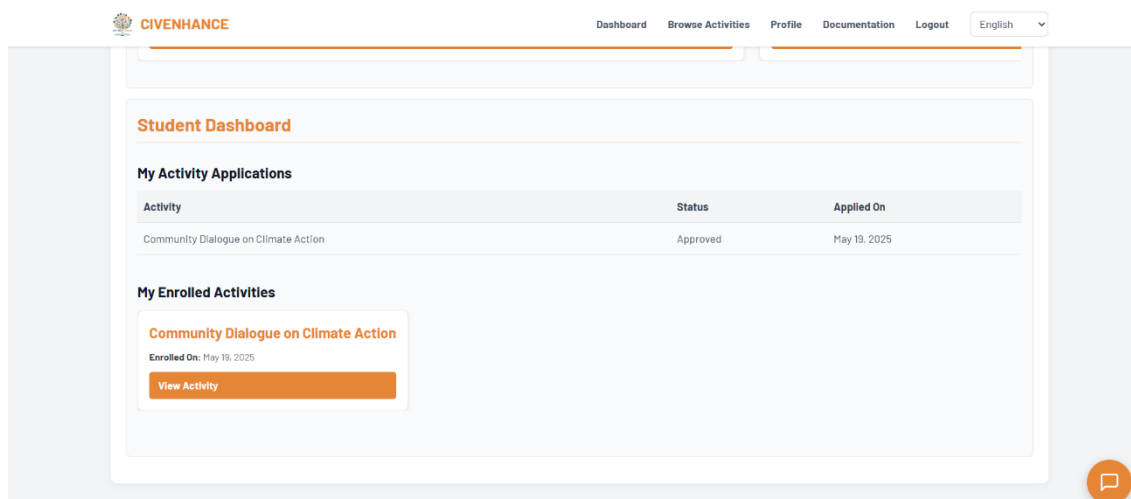


Figure 13. Student Dashboard - Applications & Enrolments.

At the forefront of the dashboard is located the "Recommended For You" carousel, immediately offering personalized suggestions for activities the user might find interesting. Following this, the dedicated sections "My Activity Applications" displays a table detailing the activity title, the current application status (such as "Pending" or "Approved"), and the date the application was submitted. Below, the section "My Enrolled Activities" is located, utilizing a card-based carousel to showcase the activities the student is currently participating in. This section provides the activity title, enrolment date, and a clear link to view more detailed information about the activity. The information on the dashboard is strategically hierarchical, prioritizing new discoveries through recommendations and then providing a clear status tracking for both applications and enrolments, giving students an immediate overview of their current standing and potential next steps.

This dashboard directly addresses the primary needs of students: it helps them find relevant e-SL activities that align with their skills and interests, track the progress of their applications, and easily access details for activities they are already enrolled in. The visual presentation of recommendations is designed to be engaging, while the clear tabular and card-based displays for applications and enrolments ensure clarity.

The "**Teacher Dashboard**" page, illustrated in Figure 14, Figure 15 and Figure 16, serves as a control center for educators to manage e-SL activities, guide their students, and interact with community partners.

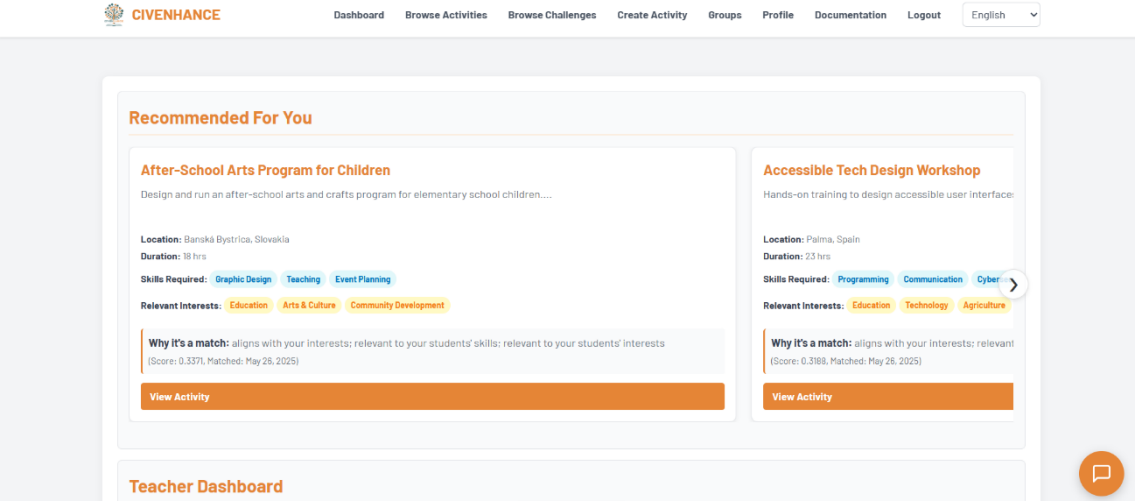


Figure 14. Teacher Dashboard – Recommendations.

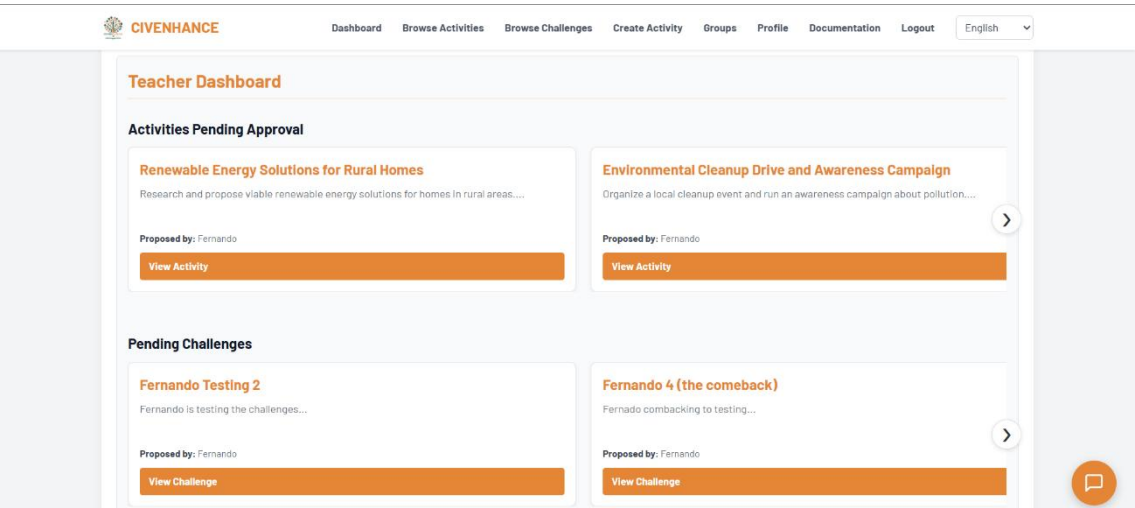


Figure 15. Teacher Dashboard – Approvals.

The screenshot shows the 'Student Engagement in Activities' section of the CIVENHANCE Teacher Dashboard. It displays a table with two columns: 'Activity' and 'Enrolled Students'. The table lists 11 activities and the number of students enrolled in each.

Activity	Enrolled Students
Accessible Tech Design Workshop	1
Community Dialogue on Climate Action	2
Cybersecurity Awareness for Small Businesses	1
Data Visualization for Public Transport Usage	1
Digital Storytelling for Social Impact	1
Ethical Sourcing Guide for Local Businesses	1
Local History Digital Archive Project	1
Promoting Mental Health Awareness on Campus	1
Senior Citizen Storytelling Project	1
Urban Beekeeping Project Setup	1

Figure 16. Teacher Dashboard - Student Engagement.

Similar to the student interface, teachers are greeted with a "Recommended For You" section, which thoughtfully suggests activities pertinent to their courses or student groups. Nevertheless, a significant portion of the teacher dashboard is dedicated to essential administrative and oversight responsibilities. Performed through the "Activities Pending Approval" and "Pending Challenges" sections, which display different cards with relevant information via a dynamic carousel of cards. Each card provides a concise summary of the proposed activity or challenge, including its title, a brief description, and the proposer's name, along with a direct link to either "View Activity" or "View Challenge." This allows for a detailed review and subsequent action, whether it's to approve, reject, or transform a challenge. Another crucial section offers a summary of "Student Activity Engagement", listing activities and the number of enrolled students, providing a clear overview of class participation and valuable insight into which activities are more appealing to students.

In conclusion, the Teacher Dashboard is meticulously tailored to support the teacher's multifaceted role as both a facilitator and an approver. It grants quick access to activities and challenges demanding their attention, streamlines the entire approval workflow, and offers valuable insights into student engagement. The integrated recommendations further empower teachers to discover new activities which might be highly relevant to their students.

The **"Community Partner Dashboard"** page, illustrated in Figure 17, is designed to empower local organizations to propose projects and track their status.

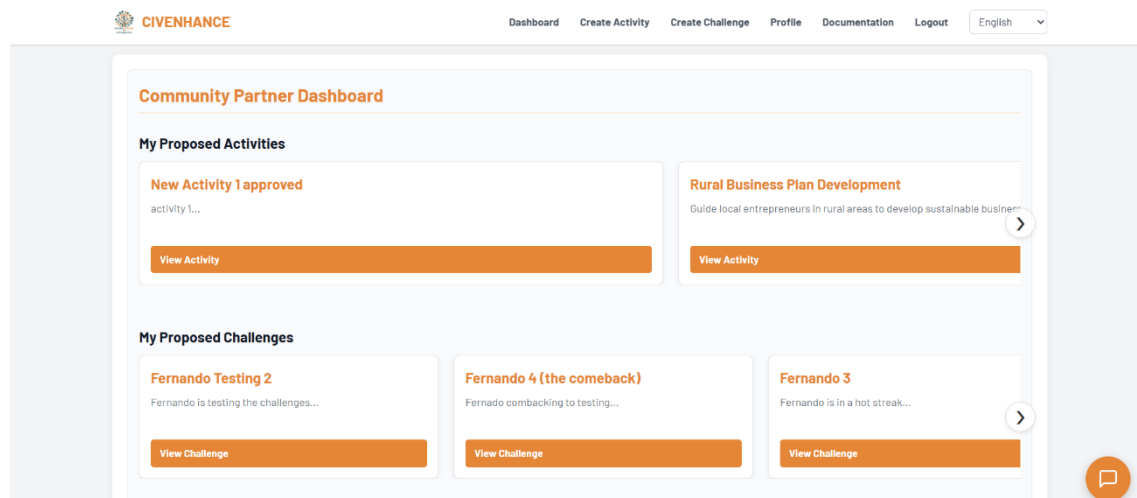


Figure 17. Community Partner Dashboard.

The community partner dashboard's design is centered around the activities and challenges initiated by the partner and their proper tracking and management. Sections like "My Proposed Activities" and "My Proposed Challenges" are essential to this end, showcasing items within a user-friendly card-based carousel. Each card concisely presents the title, a brief snippet of the description, and often the current status, such as "Pending Approval" or "Approved". Additionally, partners can directly access "View Activity" or "View Challenge" links to review or edit their submissions.

Overall, this dashboard enables community partners to effortlessly manage their contributions to the platform, monitor the progress of their proposals, and gain insight into the engagement their initiatives are receiving. It offers a clear overview of both their active and pending projects, facilitating their crucial role in initiating e-SL activities.

In conclusion, the distinct information and functionalities presented across each dashboard, while maintaining a consistent overall design language, ensure that users with different roles can efficiently access the tools and data most pertinent to their specific objectives on the platform.

4.5.2.6 Activity Creation/Editing Page

The "Activity Creation/Editing" page, illustrated in Figure 18 and Figure 19, is a critical component for Teachers and Community Partners, enabling them to define and manage e-SL activities. The form is designed to be comprehensive yet manageable, accommodating the need for detailed information and multilingual content.

The screenshot shows the 'Edit Activity' page in the CIVENHANCE platform. The top navigation bar includes links for Dashboard, Browse Activities, Browse Challenges, Create Activity, Groups, Profile, Documentation, Logout, and a language dropdown set to English. The main form area is titled 'Edit Activity' and features a language selector with options: English (Default), Slovenčina, Hrvatski, Italiano, Español, and Deutsch. The form contains several text input fields: 'Activity Title (English) *' with the value 'Music Therapy Sessions for Seniors in Care Homes', 'Description (English) *' with the value 'Organize and facilitate music therapy sessions for elderly residents.', and 'Expected Outcomes (English) *' with the value 'Improved well-being for seniors, experience in therapeutic interaction.'. Below these are fields for 'Duration' and 'Location'. A small orange speech bubble icon is visible in the bottom right corner of the form area.

Figure 18. Activity Creation/Editing Page - Top Part.

The screenshot shows the bottom part of the 'Edit Activity' page. It includes fields for 'Duration' (value: 12) and 'Location' (value: Zagreb, Croatia). There is a checkbox labeled 'Mark as Pending Approval'. Below these are sections for 'Skills Required' and 'Relevant Interests'. The 'Skills Required' section has three tags: Communication, Music Composition, and Psychological Assessment, followed by a 'Select skills...' dropdown. The 'Relevant Interests' section has three tags: Healthcare, Classical Music, and Elder Care, followed by a 'Select interests...' dropdown. At the bottom right, there are 'Save Changes' and 'Cancel' buttons. A small orange speech bubble icon is visible in the bottom right corner of the form area.

Figure 19. Activity Creation/Editing Page - Bottom Part.

Firstly, the page, as usual, with a clear title, indicating whether the user is creating a new activity ("Create New Activity") or editing an existing one ("Edit Activity"). A key feature for international use is the prominent language switcher, found at the top of the form, offering tabs for all the supported. This thoughtful design allows users to input or edit the activity's title, description, and expected outcomes sequentially in various languages, ensuring that the content is accurately translated and accessible to an international audience. Moreover, English is designated as the default language, and its fields (title, description, expected outcomes) are marked as mandatory to ensure a baseline of information is always present.

Below the language-specific fields, non-translatable activity details are logically grouped together. These include the "Duration" (specified in hours) and "Location." For Teachers who have approval rights, a convenient checkbox like "Mark as Approved" is provided, with "Pending Approval" being the default status for Community Partners. This feature allows for direct control over the activity's visibility status during its creation or editing process. A significant portion of the form is dedicated to associating "Skills Required" and "Relevant Interests" with the activity. This is implemented using the previously seen intuitive multi-select components that display chosen items as tags, making it easy to manage potentially long lists of skills and interests. These selections are vital as they directly feed into the platform's matchmaking algorithm. The form concludes with clear action buttons, such as "Save Changes" (for editing) or "Create Activity" (for new activities), alongside a "Cancel" button. To guide users, error messages are displayed contextually, helping them correct any input issues. In cases where an activity is being created from an existing Challenge, the form will pre-fill certain fields, like the title and description, with the challenge data, streamlining the process for teachers.

Ultimately, this form directly addresses the primary need of Teachers and Community Partners to clearly articulate and define e-SL opportunities. The multilingual input capability is essential for the project, guaranteeing broad accessibility of activity information. Its structured layout, with clear labels and logical grouping of fields, effectively guides users through providing information. The interactive multi-select components for skills and interests significantly enhance usability, while the ability for teachers to directly approve an activity streamlines the workflow. Furthermore, the option to pre-fill information from an existing challenge simplifies the task for teachers who are transforming community needs into activities.

4.5.2.7 Activity Browser Page

The "**Activity Browser**" page, illustrated in Figure 20, is designed to allow users, primarily Students and Teachers, to discover relevant e-SL opportunities from the full catalogue of available activities.

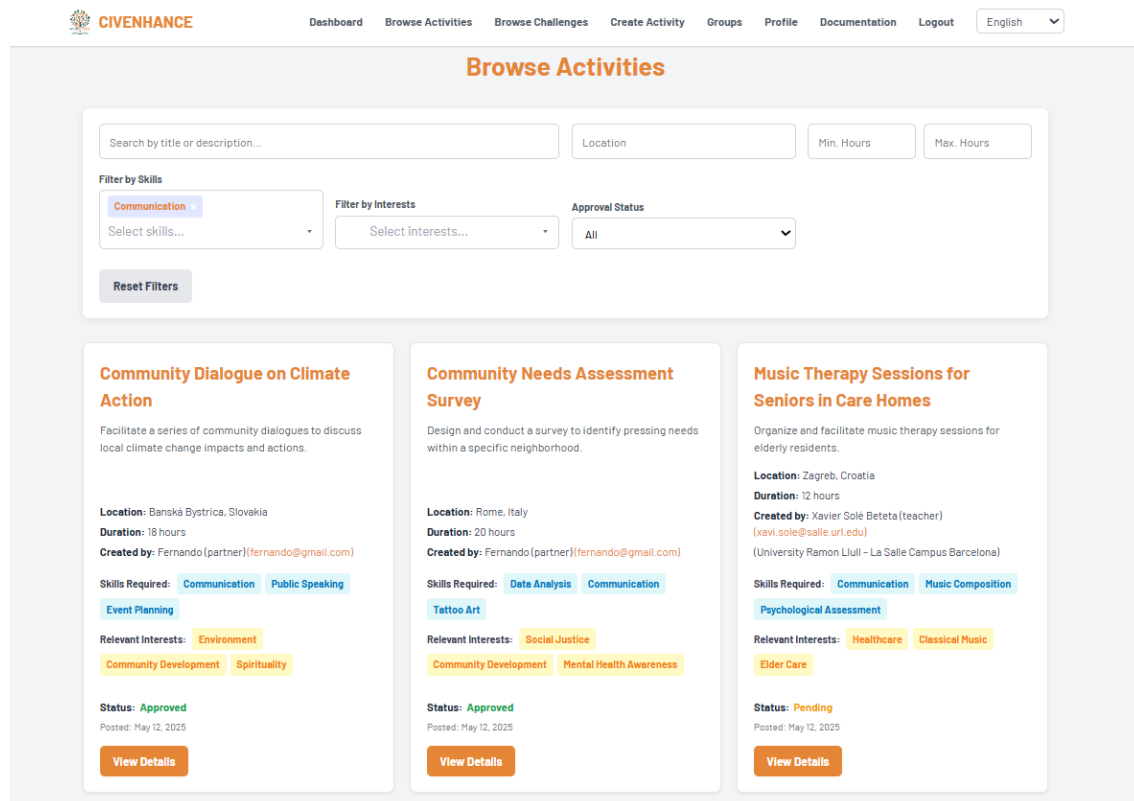


Figure 20. Activity Browser Page.

This page's design prioritizes efficient searching, featuring a prominent and comprehensive filter panel at the top. This allows users to refine their searches using various criteria: a text search field for titles or descriptions, location input, specified minimum and maximum durations in hours, and multi-select options for "Skills" and "Interests." For those with a teacher role, an additional filter for "Approval Status" (such as All, Approved, or Pending Approval) is available, offering more filtering capabilities adapted to their needs. In addition, a convenient "Reset Filters" button ensures users can easily clear their selections and start fresh.

Below the filter panel, activities are intuitively presented in a grid of cards. Each "Activity Card" provides a concise summary, including the title, a snippet of the description, location, duration, and creator information (name, role, email, and university for teachers). Furthermore, required skills and relevant interests are displayed as tags, allowing for a quick visual scanning of each activity summary. In addition, the activity's current status (e.g., "Approved," "Pending") and the date it was posted are also clearly visible. To encourage deeper exploration, a clear "View Details" button is present on each card. Overall, this thoughtful information hierarchy ensures that filtering controls are immediately accessible, followed by a scannable list of results that provide enough information for an initial assessment of relevance.

Ultimately, this page directly addresses the critical need for efficient discovery of e-SL activities. The robust filtering options allows users to quickly narrow down the list to activities that closely match their specific criteria, saving valuable time and effort. The card-based presentation of activities facilitates quick comparison and identification of promising options, while teachers particularly benefit from the ability to view activities based on their approval status, streamlining their management tasks.

4.5.2.8 Activity Details Page

The "Activity Details" page, illustrated in Figure 21, provides an in-depth view of a single e-SL opportunity, presenting all relevant information to help users make informed decisions.

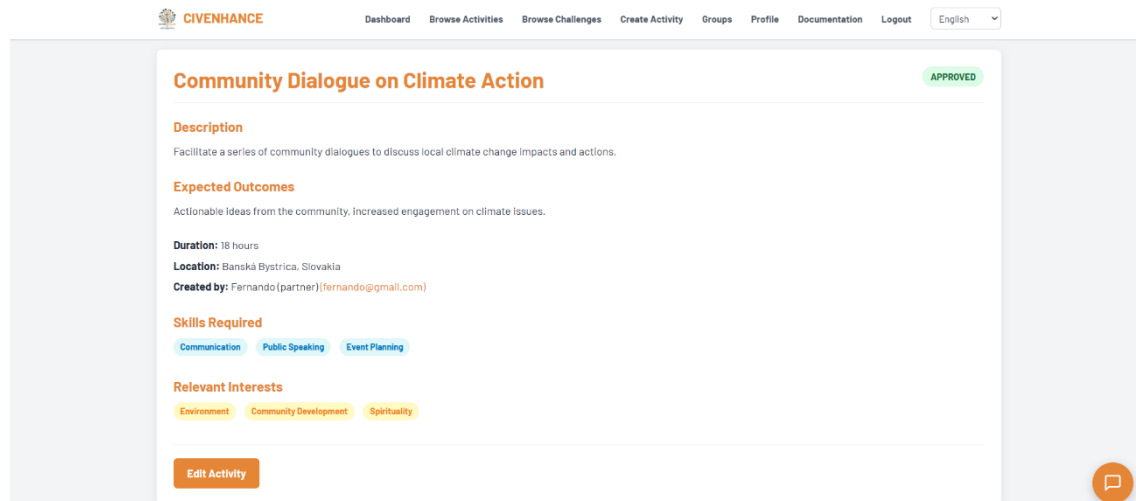


Figure 21. Activity Details Page.

This page immediately highlights the "Activity Title" and a prominent status badge, ensuring instant clarity. The main content is logically segmented into sections like "Description," "Expected Outcomes" and a block containing essential details such as "Duration," "Location," and "Created by" (which includes the creator's role, email, and university, if applicable). Finally, "Skills Required" and "Relevant Interests" are clearly presented using a tag-based format for enhanced readability.

The information hierarchy on this page prioritizes the most crucial details first, allowing users to quickly grasp the activity's essence and current status. More detailed descriptions and logistical information follow next, providing more context. Then, action buttons are positioned at the bottom of the page. For students viewing an approved activity, an "Apply Now" button will be visible. While for authorized users, such as the activity creator or a teacher with the appropriate permissions an "Edit Activity" button will provide access to the modification form previously seen. This role-based visibility ensures that users only see the options relevant to their specific permissions.

In the end, this page effectively addresses the need for comprehensive and full detailed information about a particular activity. It allows students to thoroughly evaluate an opportunity before deciding to apply. Regarding teachers, it offers a complete overview of activities they or others have created. In addition, the clear presentation of all details, including required skills and interests, supports informed decision-making. Furthermore, the role-specific action buttons streamline the user's next steps, whether they're applying for an activity or initiating an edit.

4.5.2.9 Group Management Page

The "**Group Management**" page, illustrated in Figure 22, provides teachers with a dedicated space to oversee and organize their student groups for e-SL participation.

Figure 22. Group Management Page.

The design of this page is logically divided into two primary sections, facilitating a smooth workflow for teachers. At the top, a clearly labelled "Create New Group" section, where teachers will find a straightforward form dedicated to creating new student groups. This form requires only two essential pieces of information: the group name and the number of hours required for the students, which effectively sets expectations for the students' commitment within that particular group. Once these details are entered, a prominent "Create Group" button finalizes the process, bringing the new group into existence.

Below the group creation form lies the "My Groups" section, serving as a clear overview of all existing groups managed by the teacher. Each group is presented as a distinct item or card, displaying crucial information at a glance. Teachers can quickly see the group's name, the defined student SL hours, the current number of students enrolled in the group, and the group's creation date. Moreover, each listed group is equipped with clear action buttons: a "View Details & Manage Students" option, which directs teachers to a dedicated management page for that specific group, and a "Delete Group" button for easy removal. This intentional information hierarchy prioritizes immediate access to group creation, followed by a clear, detailed overview of existing groups, further complimented with direct links for efficient management.

Overall, this page directly addresses a fundamental need for teachers: the ability to efficiently organize their students into manageable groups. Allowing them to easily create new groups tailored to specific courses or projects and providing a centralized dashboard for a quick view of all their active groups and its relevant details. The visibility of the number of students in the group along with the specified SL hours significantly aids teacher in monitoring groups. Furthermore, the "Delete Group" functionality offers essential administrative control, allowing teachers to maintain an organized and up-to-date group roster.

4.5.2.10 Group Details Page

The "Group Details" page, illustrated in Figure 23, allows teachers to oversee and manage the students inside a group.

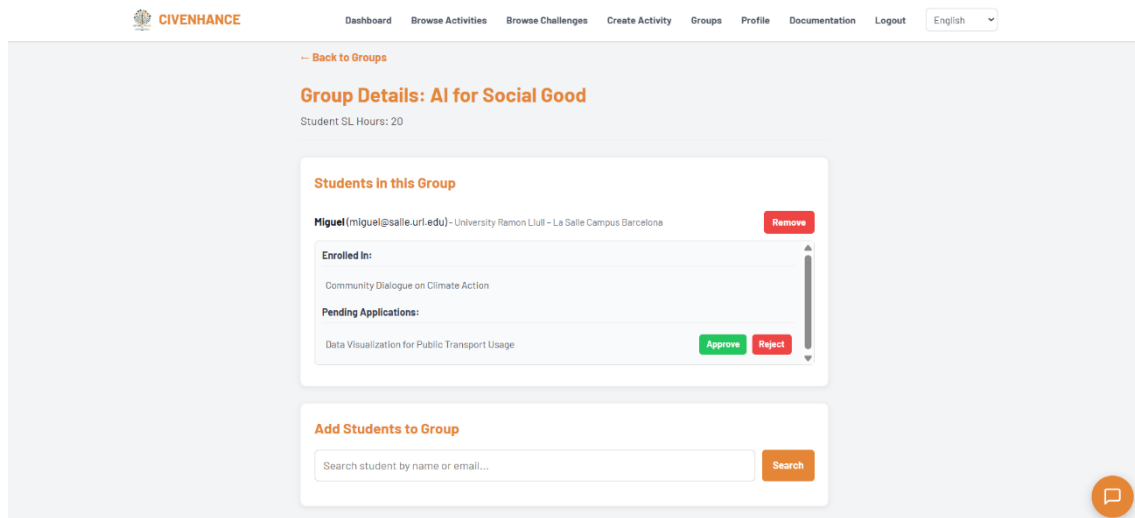


Figure 23. Group Details Page.

At the top of the page, the group name and student SL hours are prominently displayed for immediate identification, with a convenient "Back to Groups" link for easy navigation. Then, the core of the page is organized into distinct, functional blocks. A primary section, "Students in this Group", lists all current members, with each student entry displaying their name, email, and university, alongside a "Remove" button for managing group membership. Moreover, within each student's entry, teachers can also view their activity status, separated into "Enrolled In" (for formally joined activities) and "Pending Applications" (for activity applications awaiting teacher approval). For these pending applications, "Approve" and "Reject" buttons are provided directly inline, allowing teachers to efficiently manage each student applications.

A separate section, "Add Students to Group," provides a search interface where teachers can find and add new students to the current group using an input field for name or email and a "Search" button. Search results then display eligible students not already in the group, each with an "Add to Group" button. In essence, the information hierarchy on this page is clearly task-oriented: first, identify the group, then, manage current members and their activity applications, and finally, add new members.

Ultimately, this page provides teachers with comprehensive control over the students of a group. They can easily see who is in the group, remove students when necessary, and add new ones through a targeted search. A key feature is the integration of student activity application management (approving or rejecting) directly within the group's detail view. This streamlines the teacher's workflow, allowing them to handle applications for students within that group, which is far more efficient than managing all student applications globally.

4.5.2.11 Chatbot Interface

The **chatbot** interface, illustrated in Figure 24, provides users with an alternative, conversational method for seeking information and assistance related to the CIVENHANCE platform. It is designed as a readily accessible pop-up window.

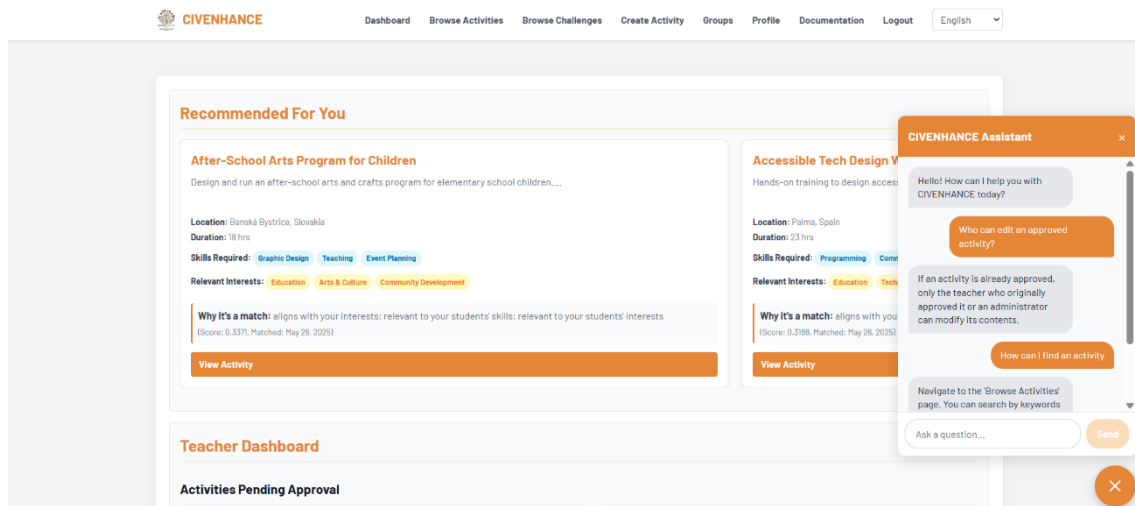


Figure 24. Teachers Dashboard with the Chatbot Pop Up.

The chatbot is activated by a persistent toggle button, a chat bubble icon, present in all the platform pages, fixed in a convenient spot like the bottom right corner. When clicked, a chat window appears as an overlay. This window features a clear "CIVENHANCE Assistant" header and a close button. The main display area shows the conversation history, with user messages and bot responses distinctly separated by alignment and background colour, making the conversation easy to follow due to the visual distinction. For instance, user messages are located on the right side with the platform main orange colour as background, while the bot messages are located on the left with a grey background. Then, at the bottom of the chat window, an input field prompts the user to "Ask a question..." next to a "Send" button. While the bot processes a request, a typing indicator (e.g., "...") appears, letting the user know a response is on its way. Finally, the conversation automatically scrolls to display the latest messages.

Overall, this chatbot interface addresses the user's need for quick answers to common questions and guidance on platform usage, eliminating the need to go through an extensive documentation. Furthermore, its conversational style can be more approachable for some users. Also, the constant availability of the toggle button ensures help is always just a click away, significantly improving user support and potentially reducing frustration.

4.5.3 Accessibility Implementation Details

Ensuring the CIVENHANCE platform is accessible to all users, including those with disabilities, was a guiding principle throughout the design and development process, aligning with the WCAG. To this end, several specific techniques and considerations were implemented:

- **Semantic HTML:** Structuring content using semantic HTML5 elements (e.g., <nav>, <main>, <article>, <aside>, <header>, <footer>, <button>) provides inherent meaning

and structure to the page, which is crucial for assistive technologies like screen readers to interpret and convey information accurately.

- **Keyboard Navigation:** All interactive elements, including links, buttons, form fields, and custom components like carousels and modals, were designed to be fully operable via keyboard-only navigation. This includes ensuring a logical tab order, visible focus indicators for currently selected elements, and that all functionalities can be triggered using standard keyboard commands (e.g., Enter, Spacebar).
- **Text Alternatives for Non-Text Content:** All informative images include descriptive alternative text. Decorative images are given null alt attributes (alt="") so they are ignored by screen readers. Icons used for functionality are accompanied by accessible text label.
- **Sufficient Colour Contrast:** Colour palettes were chosen to ensure adequate contrast between text and background colours, aiding users with low vision or colour vision deficiencies.
- **Responsive Design:** The platform was designed to be responsive, adapting to various screen sizes and orientations, aiding users with visual impairments.
- **Clear and Consistent Layout:** A consistent layout and navigation structure across the platform helps users with cognitive or learning disabilities to understand and predict how the site works more easily.

These techniques implemented reflect a dedicated effort to build an inclusive platform that the widest possible audience can effectively use, thereby fulfilling the accessibility requirements outlined in the SRS.

4.5.4 Multilingual Feature Implementation:

Supporting a diverse European user base required a robust multilingual implementation, allowing users to interact with the platform in their preferred language. This was a core requirement and influenced both UI design and technical architecture.

4.5.4.1 Language Selection and User Experience

Users are provided with a clear and consistently placed language switcher, found in the main navigation header. This is implemented as a dropdown menu that displays the names of the supported languages, such as "English," "Español," and "Slovenčina." Upon selection, the entire user interface, including navigation labels, button text, form field labels, informational messages, and even dynamic content retrieved from the database, instantly updates to the chosen language.

4.5.4.2 Technical Approach to Internationalization

For static UI text like labels, button text, titles, and placeholders, a common “i18n” library approach is used within the Astro and React front-end components. This approach involves storing translation strings in language-specific objects. Within these objects each key corresponds to a piece of text in the UI, and its value is the translation in that particular language (e.g., en: { loginButton: "Log In" }, es: { loginButton: "Iniciar Sesión" }). Then, the front-end application, through different functions dynamically loads and applies the appropriate translation strings based on the user's selected language or browser/system defaults. This ensures that all static interface elements are presented in the user's chosen language.

When it comes to dynamic content stored in the database that requires translation, a different approach is taken. As detailed and explained in the logical schema design, this dynamic content is managed through dedicated translation tables. Each primary entity table is linked to a corresponding translation table, which stores the entity id, a language code, and the translated text for specific fields. When the front-end requests data, for instance, activity details, the API on the back-end is responsible for querying the database and joining the main entity table with the relevant translation table. This join is performed based on the user's currently selected language, which is passed as a parameter in the API request. This ensures that even dynamic content is presented in the user's chosen language. Importantly, English serves as the default or fallback language if a translation isn't available for a specific item in the selected language. As the translations for some dynamic content, like the activities, must be provided by the users creating them.

This dual approach, leveraging client-side “i18n” for static UI text and server-side retrieval of localized dynamic content, ensures a truly comprehensive multilingual experience across the entire platform, effectively adapting to the diverse linguistic needs of the CIVENHANCE project's user base.

4.6 Core Functionality Implementation Details

4.6.1 User Management (FR.1, FR.2, FR.3, FR.4)

The User Management module serves as the heart of the platform's operational integrity, governing the lifecycle of user interaction from initial registration through ongoing profile maintenance. It addresses the functional requirements FR.1 (User Registration), FR.2 (User Verification), FR.3 (Login), and FR.4 (Profile Management) as outlined in the SRS. The implementation of those functional requirements is carried out through a set of RESTful API endpoints, crafted with Node.js and Express.js, which seamlessly communicates with the PostgreSQL database to manage user data.

4.6.1.1 FR.1 User Registration

The journey for any new user begins with the registration process, a critical function handled by the POST /api/auth/register endpoint. This endpoint is designed to receive a comprehensive set of user details within the request body. These details include fundamental information such as

the user's name, email, and password, alongside their designated role on the platform, be it 'student', 'teacher', or 'partner'. In addition, optional data like location, and conditionally required information such as the university for students and teachers, can also be provided. Moreover, to personalize the user experience and facilitate matchmaking, the list of skill tags selected along with the interest tags may be included.

Upon receiving this data, the system initiates a robust validation sequence. It first ensures that all mandatory fields, name, email, password, and role, are present. The provided role is then cross-referenced against a predefined list of valid roles. For users identifying as students or teachers, the presence of the university id is also mandatory. Also, if skill or interest ids are submitted, the system verifies they are structured as arrays. Email integrity is confirmed using a regular expression: `“(/^[^s@]+@[^s@]+\.[^s@]+$/)”`, and password robustness is enforced through another regular expression: `“(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_])[A-Za-z\d\W_]{8,}$/)”`, checking that the password provided at least has eight characters, including uppercase and lowercase letters, a number, and a special character.

Additionally, a key step in maintaining platform integrity is the uniqueness check for the email address. The system queries the users table to verify if the submitted email is already in use; if so, a “409 Conflict status” is returned to prevent duplicate accounts. Security of user credentials is essential and enforced by the SRS, hence the user's password is securely hashed using “bcrypt” with a salt factor of 10 before any storage occurs. The verification status of a new user is determined by their role: students are automatically verified to streamline their onboarding, whereas teachers and partners are initially set as unverified, flagging them for subsequent administrator approval, a process that will be explained later.

Once the validation and security checks are complete, the user's data previously entered: name, email, hashed password, role, location, university ID, and verification status is inserted into the table “users”. If skill ids were provided, only by students, corresponding entries are created in the “user_skills” join table, effectively linking the new user to their selected skills, while any invalid skill IDs are skipped. A similar process occurs for interest ids, populating the “user_interests” join table. Upon the successful creation of the user account and its associated data, the system responds with a “201 Created status”, accompanied by a success message and the new user's details, prudently excluding the password.

To sum up, several key technical decisions were taken in this registration process in order to fulfil the outlined requirements. The choice of “bcrypt” for password hashing reflects a security-first approach, taking advantage of its adaptive nature against common attack vectors. The role-based initial verification mechanism balances user convenience with platform security, allowing students immediate access while ensuring a vetting process for roles with greater platform influence. Furthermore, the normalized storage of skills and interests in dedicated join tables (user_skills, user_interests) facilitates flexible many-to-many relationships and minimizes data redundancy. This implementation directly fulfils FR.1, enabling all user types to register and provide the necessary information for their participation on the platform.

4.6.1.2 FR.2 User Verification

User verification is another critical administrative function, primarily designed for the approval of teacher and community partner registrations. Administrators are equipped with two key endpoints for this purpose. The GET /api/admin/users/unverified endpoint provides a list of all users designated as 'teacher' or 'partner' whose verified status is currently FALSE, allowing administrators a clear view of pending verification requests.

To action these requests, administrators utilize the PATCH /api/admin/users/:userId/verify endpoint. This endpoint expects a Boolean verify field in the request body. If verify is true, the system updates the verified column in the users table to TRUE for the specified user. Conversely, if verify is false, indicating a rejection of the registration, the user's entire record is deleted from the table "users". This decision to delete on rejection was made to maintain a cleaner, more relevant user database. Access to both these verification endpoints is strictly controlled, restricted to administrators through the combined use of authenticateToken and isAdmin middleware, which will be detailed later. This middleware-based security provides a robust layer of protection for these sensitive operations.

This system directly addresses FR.2 by allowing administrators to manage the verification lifecycle for teachers and community partners, either by approving their accounts or removing the rejected applications.

4.6.1.3 FR.3 Login

Once registered and, if necessary, verified, users access the platform via the login functionality, handled by the POST /api/auth/login endpoint. This endpoint expects the user's email and password in the request body. The system first queries the users table to locate a user matching the provided email. If no such user is found, a "401 Unauthorized" status is returned.

For users whose roles are 'teacher' or 'partner', an additional verification check is performed. If the retrieved user record indicates that their verified status is FALSE, a "403 Forbidden" status is returned, informing them that their account is pending administrator approval. If the user is found and verified the system proceeds to compare the submitted plaintext password with the securely hashed password stored in the database, using the "bcrypt.compare()" function. A mismatch at this stage also results in a "401 Unauthorized" response.

Upon successful password verification, a JSON Web Token (JWT) is generated using the "jsonwebtoken" library. This token, crucial for session management, encapsulates the user's id, email, role, and name in its payload. It is signed using a secret key, which is securely stored as an environment variable to keep it separate from the codebase and is configured with an expiration time of one hour. The decision to use JWTs was driven by their stateless nature, making them well-suited for APIs and scalable applications. Finally, a "200 OK" status is returned to the client, along with a success message, the generated JWT, and the authenticated user's details (excluding the password). This implementation effectively fulfils FR.3, providing a secure login mechanism for all registered and verified users.

4.6.1.4 FR.4 Profile Management

The platform also allows users to manage their personal information through dedicated profile management endpoints located under `/api/users`. Authenticated users can retrieve their own profile details via the `GET /api/users/profile` endpoint. Access is secured by the “authenticateToken” middleware, ensuring that only logged-in users can view their data. The user’s id is extracted directly from the JWT payload. This endpoint orchestrates a data retrieval process, querying the users table for core details and performing LEFT JOINS with the universities and its translations tables. This allows the system to fetch the user’s university name and location, thoughtfully translated to the requested language if such a translation exists. Similarly, the user’s associated skills and interests are fetched from their respective join tables (`user_skills`, `user_interests`), again joining with translation tables (`skill_translations`, `interest_translations`) to provide localized names. Critically, the user’s password is explicitly removed from the profile object before it is sent in the “200 OK” response, ensuring sensitive data is not unnecessarily exposed.

For modifying profile information, authenticated users utilize the `PUT /api/users/profile` endpoint, also protected by the “authenticateToken” middleware. The user’s id and role are extracted from the JWT to ensure users only modify their own profiles and that role-specific logic is applied. The endpoint accepts name, location, the id of the university (for students and teachers), skill ids (exclusively for students), and interest ids in the request body. Basic validation, such as ensuring the name field is not empty, is performed. To guarantee data consistency across multiple table updates, the entire update process is wrapped in a database transaction, allowing for a “ROLLBACK” if an error happens or a “COMMIT” if everything went correctly.

Overall, different technical decisions were made for a proper profile management, including the retrieval of related and translated data in a single GET request to optimize front-end loading, and the crucial use of database transactions for updates to maintain atomicity. This implementation robustly addresses FR.4, allowing users to view and modify their registration data as needed.

4.6.2 Activity Lifecycle Management (FR.6, FR.7, FR.8, FR.9, FR.10)

The section addresses the creation, modification, approval, and removal of e-SL activities within the platform. This encompasses both “Challenges,” which are initial project ideas typically proposed by community partners, and “Activities,” which are more formally structured learning engagements. This module addresses functional requirements FR.6 (Create Challenge), FR.7 (Create Activities), FR.8 (Approve Activities), FR.9 (Edit Activities), and FR.10 (Remove Activities) from the SRS.

4.6.2.1 FR.6 Create Challenge

Community partners are allowed to articulate their needs and project ideas through the creation of “Challenges” instead of activities, which are more complex, and community partners might find challenging. This functionality is provided by the `POST /api/challenges` endpoint. Only

authenticated users with the 'partner' role can create challenges, a restriction enforced by middleware checking the user's role from their JWT. This endpoint expects a title and description for the challenge in the default language (English), which are mandatory. Optionally, translations for the title and description in the remaining supported languages can be provided in a structured object.

Like in profile modification request, upon receiving a valid request, the system initiates a database transaction. The core challenge information (title, description, and the id of the creator) is inserted into the challenges table. If translations are provided, the system then iterates through them, inserting or updating corresponding entries in the "challenge_translations" table, linking them to the newly created challenge id and the respective language code. A successful transaction results in a "201 Created" response, returning the details of the new challenge. This process directly fulfils FR.6, allowing community partners to propose challenges by detailing the civic issue, needs, and potential skills involved, which can then be reviewed by teachers to be transformed into formal activities.

4.6.2.2 FR.7 Create Activities

Both teachers and community partners can create and propose "Activities", a process managed by the POST /api/activities endpoint. Authorization checks ensure that only users with 'teacher' or 'partner' roles can access this functionality. In addition, the request body must include essential information such as title and description in the default language. Other details like the expected outcomes, duration, location, associated skill ids, interest ids, and an optional challenge id (if the activity is derived from an existing challenge) can also be provided. Similar to challenges, translations for textual fields can also be supplied.

A critical distinction in the creation process is the approval status: activities created by users with the 'teacher' role are automatically marked as approved. In contrast, activities created by 'partner' roles are never set as approved, requiring the subsequent approval of a teacher. The activity creation also involves a database transaction. First, the main activity details are inserted into the activities table. Then, if provided, translations are stored in "activity_translations" table, and skill and interest associations are recorded in "activity_skills" and "activity_interests" join tables, respectively. Upon successful creation a "201 Created" response with the newly created activity's details, including its translated content for the requested language, is returned. This fulfils FR.7, enabling teachers and community partners to define specific e-SL activities with details like required skills, expected outcomes, and duration. The automatic approval for teacher-created activities streamlines their workflow, while the pending status for partner-created activities ensures an educational oversight and verification process.

4.6.2.3 FR.8 Approve Activities

The approval of activities created by community partners is a crucial step to ensure they align with educational objectives and quality standards. This responsibility falls to users with the 'teacher' role. While there isn't a dedicated "approve" endpoint, this functionality is integrated into the activity editing process (FR.9), to be detailed next. But essentially, when a teacher edits

an activity that is currently unapproved (typically one created by a community partner), they have the option to change its approved status to TRUE through the PUT `/api/activities/:id` endpoint. This makes the activity visible and available for student engagement, directly addresses FR.8.

4.6.2.4 FR.9 Edit Activities

The platform provides robust functionality for editing existing activities through the PUT `/api/activities/:id` endpoint. Nevertheless, activity modification is a critical process and, therefore, authorization for editing is carefully managed by abiding the following rules:

- Administrators can edit any activity.
- Teachers can edit the activities they have created. Furthermore, if an activity is created by a community partner, then any teacher can edit it, which includes the ability to approve it by setting the approved field to TRUE.

The endpoint accepts various fields for modification, including title, description, expected outcomes, duration, location, the approved status, and arrays of skill and interest ids. Additionally, the multilingual content can also be updated via the translations object. Moreover, the update process is transactional, assuring atomicity. The main activities table is updated with the new values and translations (if included) are inserted or updated in the “activity_translations” table. For skills and interests, the common approach is to delete all existing associations for that activity in the join tables and then insert the new set provided in the request, simplifying the update logic. Then, upon successful completion, the endpoint returns the updated activity details. This fulfils FR.9, allowing authorized users to update activity contents.

4.6.2.5 FR.10 Remove Activities

The ability to remove activities is essential for platform maintenance and is a critical action very similar to activity modification. Therefore, abiding by the same rules and following a similar procedure. The functionality is provided by the DELETE `/api/activities/:id` endpoint and, if authorized, the activity and its related data (translations, skill/interest associations, and applications/enrolments) is removed from the database, properly addressing FR.10, allowing teachers (or administrators) to remove existing activities.

4.6.3 Activity Interaction (FR.11, FR.12, FR.13, FR.14)

This section focuses on how users, primarily students and teachers, discover, view, and engage with e-SL activities. It covers the functionalities for browsing available opportunities, inspecting their details, and the processes surrounding student applications and enrolments. These capabilities are central to connecting students with meaningful projects and are addressed by functional requirements FR.11 (Browse Activities), FR.12 (Activity Details), FR.13 (Activity Application Approval), and FR.14 (Activity Student Enrolment) from the SRS.

4.6.3.1 FR.11 Browse Activities

One of the core functionalities of the platform is the activity browsing, which allows users to discover activities. A functionality provided through the GET /api/activities endpoint. This endpoint is accessible to all authenticated users, but the range of visible activities and filtering capabilities can vary based on user roles. Students, for instance, will only see activities that are marked as approved. Teachers, however, have more extensive access, with the ability to view all existing activities regardless of their approval status. Having also the possibility of filtering activities by their approval status.

Regarding filters, the platform provides a rich set of filtering options to enhance the discovery process. Users can perform text-based searches on activity titles and descriptions. Moreover, they can also refine their search by specific skills and interests, minimum or maximum duration (in hours), and location. Then, to get the activities matching the filters selected, the system constructs a dynamic SQL query based on these parameters, ensuring efficient data retrieval. In addition, when results are returned, they include not only the core activity details but also translated content for titles, descriptions, and expected outcomes, based on the user's requested language. Furthermore, associated skills and interests are aggregated and returned with their translated names, providing a rich, localized overview for each activity. This robust browsing and filtering mechanism directly fulfils FR.11, enabling students and teachers to search and filter activities based on various relevant parameters.

4.6.3.2 FR.12 Activity Details

Once an activity of interest is identified through the activity browser or from the recommendation given by the system, users can delve into its specifics using the GET /api/activities/:id endpoint. This endpoint retrieves all the pertinent information of a single activity, allowing for a thorough evaluation. The returned data includes the activity's title, description, expected outcomes, duration, location, approval status, creation date, and details about the creator (name, email, role, and university, including translated university name). Moreover, associated skills and interests are also provided with their translated names.

4.6.3.3 FR.13 Activity Application Approval

The process of a student formally expressing interest in an activity and a teacher subsequently approving or rejecting this application is a key functionality. Students initiate this by applying to an approved activity, a request handled by the POST /api/activities/:id/apply endpoint. This action is restricted to users with the 'student' role. The system first verifies that the target activity exists and is approved. It also checks if the student has already applied for or is enrolled in the activity to prevent duplicate submissions. If all conditions are met, a new entry is created in the "activity_applications" table with a status of 'pending'.

Then, the subsequent approval or rejection of these pending applications is managed by the respective teacher/s of that student through the PATCH /api/activity-

applications/:applicationId/status endpoint. This endpoint requires the id of the student's activity application and a new status ('approved' or 'rejected') in the request body. But before processing the request, authorization is key, as only users with the 'teacher' role can modify an application's status.

Finally, if the authorization check is passed and an application is 'approved', the system updates its status in the "activity_applications" table and a new enrolment record is created and inserted into the "activity_enrolments" table, linking the student to the activity and recording the ID of the teacher who approved the application. This entire status update and enrolment creation process is wrapped in a database transaction to maintain data integrity. This workflow fulfils FR.13, enabling teachers to review student activity applications and either approve or deny them.

4.6.3.4 FR.14 Activity Student Enrolment

The SRS specifies that teachers can enrol students in their class/subject into an approved activity. The primary mechanism for student enrolment, as implemented and explained just now, is a direct consequence of a teacher approving a student's activity application (FR.13). When a teacher approves an application via the PATCH /api/activity-applications/:applicationId/status endpoint, the system automatically creates an entry in the "activity_enrolments" table, effectively enrolling the student.

4.6.4 Group Management for Teachers (FR.5)

A key organizational feature for educators on the platform is the ability to manage student groups. This functionality allows teachers to structure their students into classes or project teams, define specific requirements for these groups, and oversee student participation within this context. This module directly addresses FR.5 (Manage group) as outlined in the SRS.

Teachers are allowed to create new groups through the POST /api/groups endpoint. This action is exclusively available to authenticated users with the 'teacher' role, a constraint enforced by middleware. To create a group, a teacher must provide a name for the group and specify a number of hours, which represents the expected or required number of service-learning hours for students within that group. Both fields undergo validation: the group name must be a non-empty string, and student hours must be a positive number. Upon successful validation, a new record is inserted into the groups table, associating the group with the creating teacher's ID. The system then returns a "201 Created" response, including the details of the newly formed group.

Once groups are created, teachers can get a list of all the groups they manage via the GET /api/groups endpoint. This provides an overview, including each group's id, name, hours, creation date, and the number of students belonging to each group. This allows teachers to quickly assess their various groups.

In addition, for a more detailed oversight, the GET /api/groups/:groupId endpoint allows a teacher to fetch the specifics of a single group. This includes the basic group information and a list of all students belonging to that group. For each student, the response details their id, name,

email, and translated university name. Furthermore, it also provides information about each student's enrolled activities and their pending applications. All this information gives teachers a clear picture of each student's involvement.

Obviously, managing group membership is also a core function. Teachers can add students to a group they own using the POST `/api/groups/:groupId/students` endpoint, which requires the id of the student to be added. The system first verifies that the group exists and is owned by the authenticated teacher. It then checks if the provided student's id corresponds to a valid user with the 'student' role and if the student does not already belong to that group. If both checks pass, an entry is made in the "group_students" join table, linking the student to the group. Furthermore, apart from adding, teachers can remove students from their groups using the DELETE `/api/groups/:groupId/students/:studentId` endpoint. After verifying group ownership and the validity of the ids, the system removes the corresponding entry from the "group_students" table.

Finally, teachers can also delete entire groups they own through the DELETE `/api/groups/:groupId` endpoint. This action, after confirming ownership, removes the group record from the groups table. Associated student memberships in "group_students" are also removed by the database foreign key constraints (ON DELETE CASCADE) to ensure data integrity.

This set of functionalities for creating, listing, detailing, and managing student membership within groups, fulfils FR.5, providing teachers with the necessary tools to organize their students within groups and add or remove students from these groups.

4.6.5 Recommendation System and Chatbot (FR.15, FR.16)

To enhance user engagement and facilitate the discovery of relevant opportunities, the platform incorporates an intelligent matching and recommendation system, complemented by a chatbot for user assistance. These features directly address FR.15 and FR.16 from the SRS.

4.6.5.1 Matching Algorithm (FR.15)

The platform's recommendation engine aims to proactively suggest suitable e-SL activities to both students and teachers. This system operates primarily through a content-based filtering approach, where recommendations are generated by comparing the attributes of users with the attributes of activities. Additionally, to ensure responsiveness, these recommendations are not calculated in real-time upon each user request. Instead, a dedicated service is designed to execute periodically the matching algorithm, pre-calculating and storing matches in the database.

For students, the recommendation process begins by fetching their profile, which includes their declared skill ids, interest ids, and location. Concurrently, the system retrieves all approved activities, each tagged with its required skill ids, relevant interest ids, location, and duration. The core of the matching logic involves calculating a Jaccard Index to quantify the similarity between the student's skills and the activity's required skills, and similarly for interests. The Jaccard Index

is a measure of similarity between two sets, calculated as the size of their intersection divided by the size of their union [15] [16] . For example, if a student has skills {A, B, C} and an activity requires skills {B, C, D}, the Jaccard Index for skills would be the size of {B, C} (2) divided by the size of {A, B, C, D} (4), resulting in a score of 0.5. These similarity scores are then weighted. For instance, skill matches contribute 50% to the overall score, interest matches 40%, and location relevance (a simpler check for whether the activity's location string contains or is contained within the student's location string) the remaining 10%. Activities surpassing a predefined score threshold are considered potential recommendations. The system then compiles a list of these potential matches, each with a calculated score and a reason string summarizing the basis of the match (e.g., "matches 75% of required skills, aligns with 50% of your interests, etc."). Finally, these recommendations are sorted by score, and a top N (e.g., top 10) are selected for each student.

For teachers, the recommendation strategy is slightly different, aiming to suggest activities that might be suitable for their students or align with their interests, which are supposed to be relevant to the students or aligned with their pedagogical focus. The process involves fetching the teacher's own declared interest ids. Additionally, the system aggregates the skill ids and interest ids of all students belonging to the groups managed by that teacher. Each approved activity is then scored based on its alignment with the teacher's personal interests and, more significantly, with the collective skill and interest profile of their students. Weights are applied to these different factors to derive an overall relevance score. Similar to student recommendations, these are then sorted, and the top N are selected.

All generated recommendations, for both students and teachers, are stored in a dedicated matches table in the database. Each record in this table includes the user's id, the matched entity id (which is the activity ID), matched entity type ('activity'), the calculated score, and the textual reason for the recommendation. Then, when a user logs in and views their dashboard recommendation section, the platform retrieves these pre-calculated matches via the GET /api/recommendations endpoint. This endpoint fetches the stored matches for the authenticated user, joins them with activity details (including translated titles and descriptions based on the user's preferred language), and presents them to the user.

While more complex algorithms like collaborative filtering (which considers the behaviour of similar users) or hybrid approaches could be explored in future iterations, the current content-based system was chosen for its effectiveness in taking advantage of the rich attribute data available for both users and activities. It also performs well in "cold-start" scenarios, meaning it can recommend new activities immediately without needing prior interaction data, and it provides transparency in recommendations through the generated "reason" text.

4.6.5.2 The Chatbot (FR.16)

Complementing the proactive recommendation system is an AI-powered chatbot, designed to provide users with on-demand assistance and information. This feature addresses FR.16, aiming to offer an interactive way for users, particularly students, to get answers and potentially discover relevant activities. The chatbot is accessed via a user interface element, and interactions are handled by the POST /api/chatbot/get-response endpoint.

When a user sends a message to the chatbot, the backend first attempts to find a relevant answer within a predefined FAQ dataset. This FAQ dataset is structured by language and contains common questions and their corresponding answers, along with associated keywords. The FAQ matching logic involves normalizing and tokenizing both the user's query and the FAQ keywords (e.g., converting to lowercase, removing punctuation and common stop words). A scoring mechanism then identifies the best match based on keyword overlap and contextual similarity. If a satisfactory answer is found in the FAQ, it is immediately returned to the user. This approach ensures quick responses to common queries and reduces reliance on more computationally intensive processes. However, for queries not adequately addressed by the FAQ, or when a user explicitly asks for activity suggestions or their intent for recommendations is inferred, the system escalates the request to Google's Gemini model. Which is being used due to its free tier layer that enables the platform to make API calls without any cost at all. Nevertheless, if better option arises, it is as easy as changing the API call to change the AI model used. Regarding the API call to the model, a carefully constructed prompt is sent to the Gemini API, providing context that the query pertains to the CIVENHANCE platform and specifying the desired language for the response. The prompt also instructs the AI to be friendly, concise, and to politely decline answering if the question is unrelated to the platform. In addition, for handling recommendations, the prompt includes the user's full profile and the complete, up-to-date list of currently available activities in the platform. Allowing Gemini to analyze the user preferences both from the query and the user's profile, as well as the full list of available activities and their characteristics. Resulting in a recommendation following a different criterion than the recommendation engine, due to the query provided and the enhanced analytical power of LLMs.

Once Gemini processes the request, its generated text is relayed back to the user through the chatbot interface. Regardless of whether the response originates from the FAQ or Gemini, all interactions, including the user's prompt, the bot's response, the user's id, and the source of the response ('faq' or 'gemini'), are logged into a "chatbot_logs" database table. This logging is crucial for monitoring the chatbot's performance, identifying areas for improvement in the FAQ, and understanding user needs.

In conclusion, the chatbot significantly enhances user interaction by providing on-demand assistance, information, and personalized activity recommendations, fully addressing FR.16.

4.6.6 User Dashboards (FR.17)

The user dashboards are essential to provide each user with a concise, and relevant overview of their engagement and responsibilities within the CIVENHANCE platform. This functionality directly addresses FR.17 (User Dashboard) from the SRS, which mandates that users have a personalized page displaying pertinent information in an accessible manner. The GET /api/dashboard endpoint is the primary mechanism for fetching this role-specific data, always requiring user authentication to ensure data privacy and relevance.

Upon successful authentication, the system identifies the user's role (student, teacher, partner, or admin) and dynamically queries the database to gather the specific information sets pertinent

to that role. All textual data, such as activity titles or challenge descriptions, is retrieved with appropriate translations based on the user's preferred language, defaulting to English if a specific translation is not available.

For students, their dashboard is designed to be a hub for discovery and tracking. It prominently features a list of their submitted activity applications, displaying the translated activity title, the current status of each application (e.g., 'pending', 'approved', 'rejected'), and the application date. Alongside this, students can also view their enrolments, which lists all the activities they are formally part of, showing the translated activity title and the date in which they were enrolled. Moreover, activity recommendations are a key part of the student experience and are visually integrated into their dashboard, these are fetched via the separate `/api/recommendations` endpoint as described in section 3.6.5.

Regarding the teacher dashboard, it serves as a control center for managing e-SL initiatives and student participation. It includes a section for proposed activities, listing activities created by community partners that are awaiting approval. Each item showing the translated title and description, the name of the partner who created it, and its creation date. Another vital component of the dashboard is the student engagement section, which provides an overview of activities in which at least one of their students (from the groups they manage) is enrolled, displaying the translated activity title and the number of students enrolled in that activity. Furthermore, teachers can see a list of pending challenges, those proposed by community partners that have not yet been converted into formal activities, showing translated titles, descriptions, and creator details. Furthermore, as the student dashboard, this one also has a recommendation section integrated, whose items are also fetched through the `"/api/recommendations"` endpoint.

Community partners receive a dashboard focused on the lifecycle of their contributions. The partner activities section lists all activities they have proposed, showing the translated title, description, current approval status, and creation date. Similarly, partner challenges list all challenges they have initiated, with their translated titles, descriptions, and creation dates. This allows partners to easily track the progress and status of their initiatives.

Finally, for administrators, the dashboard lists all users registered with 'teacher' or 'partner' roles whose accounts are still pending verification, showing their name, email, role, associated university name (if applicable), and registration date, enabling administrators to efficiently manage the verification queue.

By tailoring the information presented to the specific needs and responsibilities of each user role, the dashboard system effectively provides a personalized and efficient entry point into the platform's functionalities. This ensures that users can quickly access the most relevant data and tasks, thereby fulfilling the requirements of FR.17.

4.7 Security Implementation

Ensuring the security and integrity of the CIVENHANCE platform and its user data was a top priority. A multi-layered approach to security has been implemented, addressing authentication,

authorization, data protection, and the prevention of common web vulnerabilities. This aligns with the security requirements outlined in the SRS, particularly concerning data encryption, access control, and user verification processes.

4.7.1.1 Authentication and Authorization Mechanisms

The primary mechanism for user authentication is token-based, utilizing JWT. Upon successful login, the system generates a JWT signed with a secret key. This token contains a payload with essential user information, including the user id, email, role, and name. The token is then sent to the client, which includes it in the Authorization header (as a Bearer token) for all subsequent requests to protected API endpoints.

The “authenticateToken” middleware, previously mentioned, is applied to all routes requiring user authentication. This middleware extracts the token from the request header, verifies its signature against the server's secret key, and checks for expiration. If the token is valid and not expired, the decoded user payload is attached to the “req.user” object, making user information available to subsequent route handlers. If the token is missing, invalid, or expired, an appropriate HTTP error status (401 Unauthorized or 403 Forbidden) is returned, preventing unauthorized access.

Regarding authorization, once a user is authenticated, authorization mechanisms determine what actions they are permitted to perform. This is primarily role-based. The user's role (student, teacher, partner, admin), embedded within the JWT payload, is inspected by various API endpoints to control access to specific functionalities or data.

For instance, sensitive administrative functions, such as verifying teacher and partner accounts or viewing lists of unverified users, are protected by an additional “isAdmin” middleware. This middleware, as deduced by its name, explicitly checks if the user role is 'admin'. If not, a “403 Forbidden” response is issued, even if the user has a valid authentication token. Similar role-based checks are implemented directly within route handlers for other functionalities. For example, creating challenges is restricted to 'partner' roles, creating groups to 'teacher' roles, and applying for activities to 'student' roles. Editing or deleting resources like activities also involves checks to ensure the user is the creator, an authorized teacher, or an administrator, depending on the specific business rules.

4.7.1.2 Data Encryption

As mentioned during the registration functionality, user passwords are never stored in plaintext. Each user's password is securely hashed using “bcrypt” with a salt factor of 10. “Bcrypt” is a strong, adaptive hashing algorithm designed to be resistant to brute-force attacks and rainbow table attacks. When a user attempts to log in, the provided password is then compared with the stored hash using `bcrypt.compare()`.

While the specific deployment configuration is part of the deployment environment, the platform is designed to operate exclusively over HTTPS in a production setting. This ensures that all data transmitted between the client and the server, including credentials, JWTs, and personal

information, is encrypted in transit, protecting it from eavesdropping and man-in-the-middle attacks, which aligns with the communication interface requirements in the SRS.

4.7.1.3 Input Validation and Sanitization

In addition, to be protected against common web vulnerabilities such as Cross-Site Scripting (XSS) and SQL injection, the platform implements input validation and, where appropriate, sanitization.

All data received from clients via API request bodies or query parameters undergoes validation on the backend before being processed or stored. Including:

- **Type Checking:** Ensuring data is of the expected type (e.g., string, number, boolean, etc.).
- **Format Validation:** Checking formats for specific fields like email addresses (using regular expressions) and password complexity.
- **Range and Value Checks:** Ensuring numerical values are within expected ranges or that string values for roles match predefined allowed values.
- **Presence Checks:** Verifying that all mandatory fields are provided.

If validation fails, the API typically returns a “400 Bad Request” status with an error message indicating the specific issue, preventing malformed or malicious data from being processed further.

Regarding SQL Injection, the backend primarily interacts with the PostgreSQL database using parameterized queries (also known as prepared statements) facilitated by the “pg” library. By using parameterized queries, user-supplied input is treated as data rather than executable SQL code, effectively neutralizing the risk of SQL injection attacks. Dynamic SQL construction is avoided wherever possible, and when absolutely necessary for complex filtering (like in activity browsing), parameters are carefully managed.

Finally, for XSS Prevention, when displaying user-generated content, especially rich text or descriptions, the platform would ensure proper encoding or sanitization on the front-end before rendering or rely on the framework's built-in mechanisms. On the backend, while API responses are primarily JSON, which is less susceptible to direct XSS injection into HTML, the principle of validating and being cautious with any data that might eventually be rendered as HTML is maintained.

5 Preliminary Evaluation and Testing

5.1 Introduction

This chapter presents results from the preliminary evaluation of the CIVENHANCE e-SL matchmaking platform, focusing on usability, perceived usefulness, and system logic as per the SRS and research objectives. Due to the project timeline, evaluation used simulated data and scenario-based testing, not live user data. The comprehensive testing included unit, integration, and system tests, with findings detailed below to show functional robustness, user experience, and goal alignment.

5.2 Testing Setup

Preliminary evaluation occurred in a controlled development environment.

- **Environment:** A development server mirroring the planned setup was used, with testing across modern web browsers (Chrome, Firefox, Safari).
- **Simulated Data:** A synthetic dataset was generated to test functionalities like the recommendation engine and activity browsing. This included diverse user profiles and a catalogue of simulated e-SL activities and challenges with multilingual details.
- **Testing Tools:** Standard developer tools, browser consoles, API testing tools (Postman), and backend log monitoring were used.

5.3 Unit Testing

Unit tests verified individual platform components and modules:

- **User Management (FR.1-FR.4):** Registration showed robust validation, secure password hashing, and correct role assignment. Verification endpoints functioned, and login successfully authenticated users with JWT generation. Finally, profile management allowed secure data updates.
- **Activity Lifecycle (FR.6-FR.10):** Challenge and activity creation (including multilingual content and role-based approval) were successful. Editing and removal by authorized users performed as expected.
- **Group Management (FR.5):** Endpoints for group creation and student management by teachers functioned correctly.
- **API Endpoint Handlers:** Showed fast response times, with most operations taking a value in between 5ms and 55ms.
- **Frontend Components:** Key React components rendered and managed state appropriately.

No critical bugs impeding core functionality were found and minor input validation issues were rectified.

5.4 Integration Testing

Integration tests verified interactions between platform components:

- **Frontend-Backend API Communication:** User registration, activity creation/editing (with multilingual data), and activity browsing/filtering flows were successful, with correct data handling and UI updates.
- **API-Database Interaction:** CRUD operations were verified, with correct data insertion into multiple related tables and transactional integrity.
- **Role-Based Access Control:** Middleware and authorization checks correctly protected API endpoints.
- **Chatbot Integration:** Frontend queries were successfully sent to the backend, FAQ/Gemini logic was tested, and responses were relayed and logged.
- **Recommendation Data Flow:** Pre-calculated recommendations were correctly retrieved and displayed.

During the process, minor multilingual data formatting discrepancies were found and resolved. Overall, components showed robust integration with low API response times.

5.5 System Testing

System testing was conducted to evaluate the CIVENHANCE platform as an integrated whole, ensuring all components work together as specified in the SRS and that the overall system logic was sound. This phase involved executing end-to-end scenario-based tests using the Cypress testing framework to simulate real user workflows and validate overall system behaviour and user experience.

End-to-end workflows for each functionality of each user role (Student, Teacher, Community Partner) were meticulously executed. These tests covered the full spectrum of interactions, from initial registration to task completion and information retrieval.

As an example, a typical test scenario executed for students via Cypress involved:

1. Navigating to the registration page and successfully creating a new student account, including inputting personal details, selecting a university, and adding skills and interests.
2. Logging into the platform using the newly created credentials.

3. Browse the "Activity Browser" page, applying various filters (e.g., by skill, by interest, by location), and verifying that the displayed activities matched the filter criteria.
4. Selecting an activity and navigating to its "Activity Details" page to review the information.
5. Successfully applying for an e-SL activity.
6. Navigate back to the dashboard to verify the new application and its status is "Pending".

The successful execution of such student lifecycle scenario is illustrated in Figure 25, which shows the Cypress system test results.

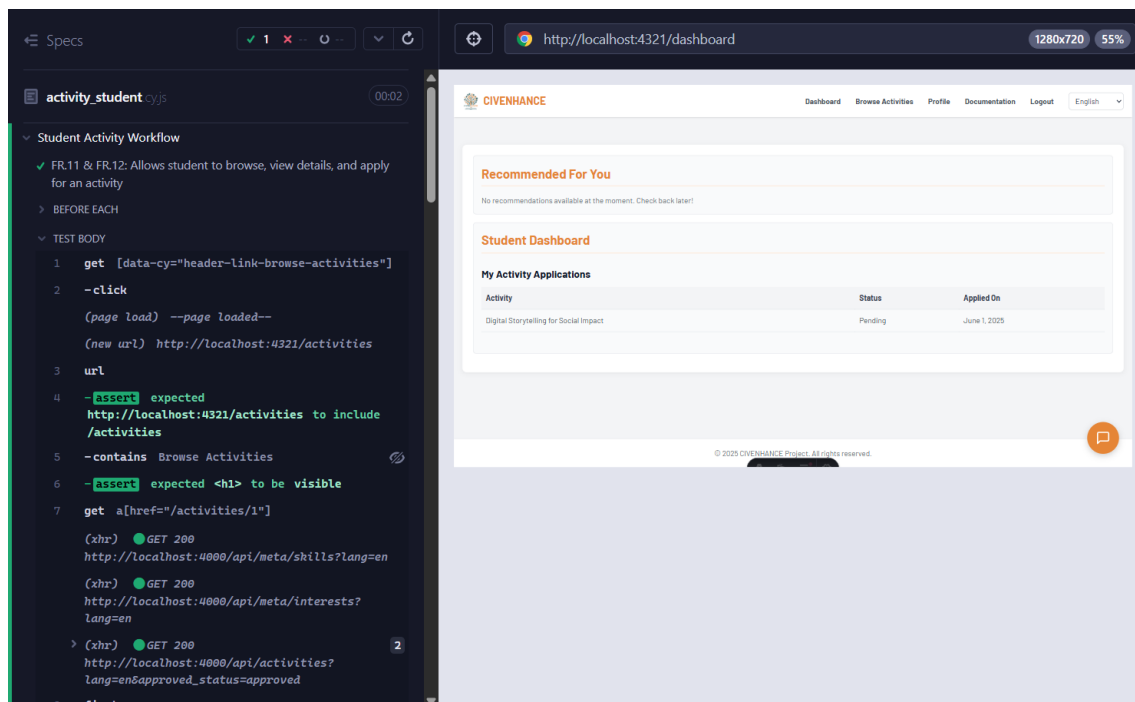


Figure 25. Results of Student Cypress Test.

Another example of a system test would be the one testing some of the key functionalities of both teacher and partner users. Specifically, this test included the following steps:

1. Logging in as a Partner user.
2. Navigating to "Create Challenge" and submitting a new challenge with a title and description.
3. Checking their dashboard to see the status of the proposed challenge.
4. Log out and log in as a teacher user.
5. Navigating to the "Challenges" page.

6. Look for the new challenge created by the partner and start its conversion into an activity.
7. Verify the form already includes the data from the challenge.
8. Add the necessary information (outcomes, skill/interest tags...) in the activity form.
9. Save the new Activity.
10. Navigate to the activity browser and check the new activity appears by searching for it.

Figure 26 provides a visual confirmation of the test execution for the combined teacher and partner workflow detailed above, demonstrating the seamless interaction between these roles as verified by Cypress.

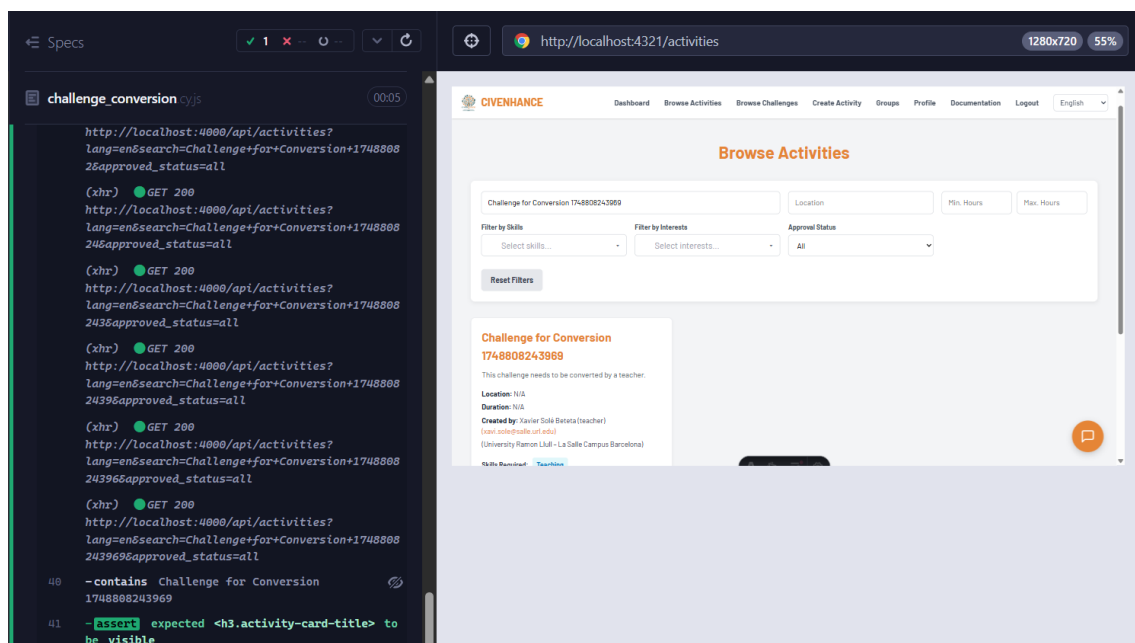


Figure 26. Results of Partner and Teacher Cypress Test.

Overall, the system tests confirmed that core functionalities operated cohesively and as a unified system, aligning with the SRS requirements. The system correctly guided users through complex processes, enforced permissions as designed, and maintained data integrity across various interactions. Backend API response times, observed via Cypress command logs and browser developer tools during these end-to-end tests, remained consistently low, and no major deviations from expected behaviour or critical bugs were observed.

5.6 Simulated Data Tests

5.6.1 Recommendation System Evaluation

The recommendation system, a key component for enhancing user engagement, was evaluated using simulated data due to the lack of live user interactions. The primary goal of this evaluation was to assess the logic and potential effectiveness of the content-based matching algorithm in generating personalized and relevant suggestions for both students and teachers.

Firstly, a synthetic dataset was meticulously constructed to serve as the basis for this evaluation. This dataset included a variety of student profiles, each with a unique combination of declared skills, and geographical locations. For instance, one simulated student profile might be "Ana Silva", a Computer Science student from "Barcelona, Spain" with skills in "Python", "Machine Learning", and "Web Development", and interests in "Artificial Intelligence" and "Social Impact". While another might be "Markus Weber", an Arts student from "Vienna, Austria" skilled in "Graphic Design" and "Video Editing", interested in "Cultural Heritage" and "Community Arts". In addition, regarding the activities, more than 50 distinct e-SL activities were simulated, each with a detailed description, defined required skills, associated relevant interests, specific locations, and varying durations. For example, one activity, "Youth Mentorship & Tutoring Program," might be located in "Palma, Spain", require skills like "Communication", "Teaching", and "Fundraising", target interests such as "Education", "Social Justice" and "Mental Health Awareness", and have a duration of "18 hours". While another, "Community Health Data Mapping", could be based in "Rome, Italy", need skills like "Data Analysis" and "Research", appeal to interests in "Healthcare" and "Community Development", and have a duration of "35 hours".

This dataset was then processed by the content-based matching algorithm, which, as detailed in before, primarily utilizes the Jaccard Index to quantify the similarity between a user's attributes (skills, interests) and an activity's attributes, combined with location relevance. The algorithm successfully generated personalized recommendations for the simulated student and teacher profiles. Then it was observed that activities exhibiting a high degree of skill and/or interest overlap with a user's profile were consistently ranked higher in the recommendation lists. Moreover, the weighting system employed by the algorithm (skills: 50%, interests: 40%, location: 10%) appeared balanced in its influence on the final rankings during these simulations. Activities that matched well on highly weighted criteria (like skills) but were geographically distant might still be recommended, but local activities with a reasonable (though perhaps slightly lower) skill/interest match could also feature prominently, preventing an over-reliance on a single factor. This suggests that the intended balance, where multiple facets of user preference and activity characteristics contribute to the match score, was being achieved.

Furthermore, the "reason for match" feature, which provides a textual summary explaining why an activity was recommended, was deemed particularly useful for transparency. This feature allows users to understand the basis of the suggestions, potentially increasing trust and engagement with the recommendations.

Nevertheless, it is crucial to reiterate that these findings are indicative and based on a controlled, simulated environment. The true effectiveness of the recommendation system, including the

optimal tuning of weights and thresholds, can only be assessed through iterative testing and refinement with real user data and explicit feedback on the quality and relevance of the recommendations.

5.6.2 Chatbot Evaluation

The chatbot was evaluated on its FAQ answering efficacy and its ability to provide relevant responses and recommendations through the Gemini model escalation. The FAQ matching algorithm achieved around 84% accuracy on a test set of 50 common questions, remaining queries either had no match or resulted in incorrect retrieval due to ambiguity or limitations in keyword matching. Moreover, queries not matched in the FAQ were correctly escalated to the Gemini API. For informational requests, Gemini provided relevant, coherent responses in the requested language, adhering to the contextual instructions given in the prompt. For simulated activity recommendation requests, the Gemini API was provided with the simulated user profile, the query, and a list of around 50 diverse, simulated activities. In these test scenarios, Gemini successfully processed the input and returned a set of pertinent activity suggestions. In addition, the justifications for these suggestions, when provided by the model, generally aligned with the user's profile and query. While a proper evaluation of recommendation accuracy via the chatbot necessitates extensive real user testing, these initial tests demonstrated promising relevance in the activities suggested. Overall, the FAQ mechanism showed high accuracy for common informational queries, even though it can be further improved, and the Gemini integration proved capable of handling both advanced informational support and generating personalized activity recommendations.

5.7 Limitations of the Evaluation

This preliminary evaluation was limited due to various factors, like the absence of live user data, which impacted the assessment of features like the recommendation engine and chatbot. As the recommendation engine testing with synthetic data doesn't reflect real user complexities. In addition, the chatbot evaluation is best measured by actual user interactions and their feedback. Finally, due to the time constraints of both the CIVENHANCE project and this thesis, a longitudinal impact assessment was not conducted. Consequently, the long-term effects on student engagement or learning outcomes could not be assessed.

6 Discussion

6.1.1.1 Introduction

The primary goal of this thesis was to design and develop a functional prototype of a web platform to facilitate the discovery, recommendation, and coordination of e-SL activities, thereby addressing the challenges in connecting students, teachers, and community organizations. The following sections will evaluate how the project outcomes align with this goal and the specific research questions posed.

6.1.1.2 Answering the Research Questions

RQ1: What are the key functional, technical, and user experience requirements for a web-based platform aimed at promoting e-SL engagement among European students?

The process of requirements elicitation, involving stakeholder consultation and culminating in the SRS, directly addressed this question. The key functional requirements identified included robust user management with distinct roles (Student, Teacher, Community Partner, Administrator), comprehensive activity and challenge lifecycle management, group management for teachers, a recommendation system, and an AI-powered chatbot recommender. Technical requirements emphasized a scalable web architecture, API-driven communication, database integrity, multilingual support, and adherence to W3C Web Accessibility Initiative standards. User experience requirements focused on intuitive navigation, role-specific dashboards, and clear information presentation.

The successful implementation and testing of these functional requirements demonstrate that the platform incorporates these core needs. For instance, the distinct user registration flows, role-based dashboards, and activity creation/management processes directly reflect the elicited requirements. The chosen technology stack (Astro/React for frontend, Node.js/Express.js for backend, PostgreSQL for database) was selected to meet these technical demands, and the preliminary performance results (e.g., API response times typically between 3ms-55ms) suggest the architecture is capable of supporting the initial operational demands.

RQ2: How can user-centered design principles be applied to create an intuitive and accessible interface for browsing, managing, and participating in e-SL activities?

User-centered design guided the platform's design, as described in Chapter 4 (Section 4.5). This included:

- **Role-Specific Interfaces:** Dashboards and navigation menus were tailored to the specific needs and tasks of each user role, aiming to reduce cognitive load and present relevant information upfront.
- **Clear Information Architecture:** The platform was structured to allow logical progression through tasks, such as activity creation (Figure 18 and Figure 19) or browsing (Figure 20).

- **Consistent Design Language:** A consistent visual style, typography, and component behaviour were maintained across the platform to enhance predictability and learnability.
- **Accessibility Considerations:** Adherence to WCAG principles was a stated goal, influencing choices in semantic HTML, keyboard navigability considerations, and aiming for sufficient colour contrast (even though formal accessibility audits were outside the scope of this preliminary phase).
- **Multilingual Support:** The interface was designed to seamlessly switch between multiple languages, a critical aspect for user experience in a European context.

The system testing scenarios (Chapter 5, Section 5.5), which simulated user workflows, indicated that users could navigate and complete core tasks logically. While formal usability testing was not conducted, the design choices were made with the intention of creating an intuitive and accessible experience. The positive functionality observed during system testing suggests a solid foundation in user-centered design.

RQ3: What recommendation strategies are most suitable for matching students with appropriate e-SL opportunities based on their preferences, skills, and academic goals?

Given the pre-deployment stage and the absence of historical user interaction data, a content-based filtering approach was implemented for the recommendation system. This matches users to activities based on explicit attributes. In addition, the matching algorithm utilizes the Jaccard Index to quantify similarity between user and activity attributes (skills and interests), combined with location relevance, and weighted to produce a match score. Complementing this main recommendation engine, the AI-powered chatbot offers an alternative, conversational recommendation strategy. By using the Gemini API to analyze a user's profile, their stated preferences within the chat, and the full activity pool, it provides on-demand, personalized suggestions.

The evaluation of this system using simulated data indicated that this dual-approach could generate relevant recommendations while addressing different user preferences for interaction and discovery. For instance, student profiles with specific skill/interest combinations were consistently matched with activities requiring similar attributes.

Overall, this dual-component strategy was chosen as it directly addresses the "cold start" problem (by not requiring past user behaviour) and instead takes advantage of the rich descriptive data available for both users and activities. It also offers multiple sources for recommendation (through a structured, attribute-driven engine and a flexible, conversational AI) thereby catering to different user interaction styles. Moreover, this approach is computationally feasible for the initial scale of the platform. However, its true effectiveness requires real user data for refinement and the potential integration of complementary recommendation techniques.

RQ4: How effective is the prototype in terms of its designed usability, inferred perceived usefulness, and system logic, based on preliminary evaluations using synthetic data and scenario-based testing?

- **Designed Usability:** Scenario-based system testing (Chapter 5, Section 5.5) showed good functional usability. Core workflows were successfully executed, supported by clear navigation, role-specific dashboards, and responsive backend performance.
- **Inferred Perceived Usefulness:** While direct user feedback is pending, the platform's functionality addresses the core need for a centralized e-SL connection system. Features like activity management and multilingual support suggest it would be perceived as useful.
- **System Logic:** Unit, integration, and system testing (Chapter 5, Sections 5.3-5.5) confirmed the robustness and correctness of the system logic, including authentication, permissions, approval workflows, and data integrity, aligning with the SRS.

In summary, the preliminary evaluations suggest the prototype is effective in its core design and logic. It successfully implements the required functionalities and provides a user interface structure that supports key tasks, laying a strong foundation for future real-world testing.

RQ5: What technical, ethical, and practical challenges arise in the design of a recommender-based educational platform, and how can they inform future iterations and full deployment?

- **Technical Challenges:**
 - **Recommendation Algorithm Refinement:** Requires ongoing tuning with real user data to ensure relevance and avoid filter bubbles.
 - **Data Sparsity:** Initial limited data could affect recommendation quality; strategies to encourage detailed profiles and diverse activity posting are needed.
 - **Scalability:** Significant user growth would require performance monitoring and potential scaling strategies.
- **Ethical Challenges:**
 - **Bias in Recommendations:** Must consider fairness and bias detection/mitigation.
 - **Data Privacy:** Ensuring GDPR compliance and transparent data usage is crucial.
 - **Transparency of Recommendations:** Users should understand data influence and have preference control.
- **Practical Challenges:**
 - **User Adoption and Engagement:** Active participation from all stakeholders is crucial.

- **Content Quality Control:** May require moderation or guidelines, teacher approval for partner activities partially addresses this.
- **Maintenance and Sustainability:** Long-term support beyond initial funding is a key consideration.

These challenges highlight the iterative nature of developing such a platform. Nevertheless, the current prototype provides a strong base, and addressing these points will be key to its successful evolution and impact.

6.1.1.3 Reflection on Design and Development Choices

The technology stack chosen (Astro/React, Node.js/Express.js, PostgreSQL) proved to be suitable for developing a modern, API-driven web application.

The combination of Astro and React allowed for efficient static site generation for content-heavy pages and dynamic interactivity where needed. This hybrid approach aimed to balance performance with rich user interface capabilities.

Regarding Node.js and Express.js, it provided a lightweight and scalable backend for API development, with JavaScript consistency across the stack simplifying the development process for a single developer. In addition, the observed API performance supports this choice.

Finally, the choice of PostgreSQL proved to be key for the platform's development. As its relational integrity features were crucial for managing the complex relationships between users, activities, groups, and translations.

7 Conclusions and Future Work

7.1 Summary of Key Findings and Conclusions

The primary objective of this thesis was to design and develop a functional web platform to facilitate the discovery, recommendation, and coordination e-SL activities for students across European HE institutions, acting as a centralized marketplace with personalized recommender systems, and focusing on the pre-implementation phase. The research and development process detailed in the preceding chapters has led to key findings and conclusions demonstrating that this primary objective, along with its specific sub-objectives, was successfully achieved:

1. **Requirement Analysis and SRS Creation:** The functional and user requirements for the e-SL matchmaking platform were systematically identified and analyzed through engagement with relevant stakeholders. This process culminated in the creation of SRS, which guided the subsequent design and development. The developed prototype's adherence to these documented requirements confirms this objective's fulfilment.
2. **Platform Architecture and User Interface Design:** The architecture and user interface of the web-based platform were designed ensuring usability, accessibility, and alignment with the needs of students, teachers, and rural community partners. In addition, the technical feasibility and initial performance evaluations of the chosen technology stack, together with core implemented features such as multilingual support and role-specific dashboards, demonstrate a sound design and architecture responsive to stakeholder needs.
3. **Recommendation Engine Implementation:** A recommendation engine capable of suggesting personalized e-SL activities based on user preferences (skills, interests) and activity characteristics was successfully designed and implemented using a content-based filtering approach. Preliminary evaluations using simulated data indicated its potential to provide relevant suggestions to users. Nevertheless, proper testing with real user's data must be made in order to fine-tune it.
4. **AI-Powered Chatbot Implementation:** An AI-powered chatbot was implemented, capable of gathering user preferences through interaction and providing information or suggesting e-SL activities. The chatbot was found to provide a functional initial layer of user support and guidance, as well as useful recommendation made using a different criterion than the recommendation engine.
5. **Functional Prototype Development:** A functional prototype that includes the core functionalities of the matchmaking platform—such as the activity marketplace, user management with distinct roles, activity and challenge lifecycle management, group management tools for teachers, and the integrated recommender system and chatbot—was successfully developed. This prototype serves as a tangible system ready for further refinement and pilot testing.

6. **Preliminary Evaluation:** Preliminary evaluations of the platform's usability, functionality, and recommendation logic were conducted using simulated data and scenario-based testing, covering unit, integration, and system tests. These tests confirmed good functional robustness, sound system logic, and efficient backend API performance, with most common operations completing within a 3ms to 55ms range.
7. **Documentation and Groundwork for Future Work:** The development process, challenges, and technical considerations have been thoroughly documented throughout this thesis. This documentation establishes the groundwork for future deployment and iterative development within the broader CIVENHANCE European project.

In summary, this thesis concludes that it is feasible to design and implement a comprehensive web platform that addresses the multifaceted needs of e-SL engagement. The project successfully translated the defined objectives into a working prototype, validating the concept of a centralized digital tool to enhance the coordination and accessibility of e-SL initiatives. This provides a solid technical and functional foundation for the subsequent phases of the CIVENHANCE project.

7.2 Contributions of the Thesis

This thesis makes several notable contributions:

- **A Functional e-SL Matchmaking Platform Prototype:** The most significant contribution is the delivery of a working software prototype. This platform serves as a foundational asset for the CIVENHANCE European project, moving the concept from ideation to a demonstrable system.
- **Validated Requirements and Design for e-SL Systems:** The detailed requirements elicitation process and the subsequent design of the platform's architecture, database, and user interface contribute to a better understanding of what such systems entail, which can inform future projects in the e-SL technology domain.
- **Demonstration of a Recommender-Assisted Approach:** The integration of a content-based recommendation system for proactive suggestions, alongside an AI-powered chatbot, together showcase practical approach to personalizing the discovery of e-SL opportunities, addressing a key challenge in matching user skills and interests with community needs.
- **Practical Implementation of Multilingual Support:** The thesis demonstrates a viable strategy for implementing multilingual capabilities in both static UI elements and dynamic, user-generated content within a web platform, a critical aspect for European or international projects.

7.3 Limitations of the Work

It is essential to acknowledge the limitations of this study to provide a balanced perspective on the conclusions. The most significant limitation is the absence of live user data and real-world evaluation, which constrains the ability to definitively assess real-world usability, user satisfaction, the true effectiveness of the recommendation engine, and the practical utility of the chatbot. Furthermore, the simulated evaluation of key features, such as the recommendation system and aspects of the chatbot's performance, utilized synthetic data and simulated queries that may not capture the full spectrum of real user behaviours, preferences, or the nuances of their information needs. Additionally, no formal acceptance testing with diverse end-users was conducted within the timeframe of this thesis, despite the platform being designed based on initial stakeholder input. Finally, the study lacks a longitudinal impact assessment, making it unable to measure the long-term effects of the platform on student engagement, learning outcomes, the development of civic skills, or the success rate of e-SL collaborations. These limitations underscore that the current work represents a foundational stage, primarily validating technical feasibility and functional completeness against initial specifications.

7.4 Temporal Costs

The completion of this thesis and the development of the CIVENHANCE platform prototype involved a significant investment of time across various distinct phases. These tasks, undertaken to achieve the objectives outlined in first chapter, can be grouped into the following categories, with an estimated total of 480 hours dedicated to the project.

- **Research and Investigation (Estimated: 40 hours):** This initial phase involved a comprehensive review of existing literature on e-Service-Learning, civic engagement platforms, recommendation systems in educational contexts, and relevant web technologies. Tasks included understanding the CIVENHANCE project's broader goals, investigating potential technology stacks, and researching best practices for multilingual and accessible web application development. This foundational work was crucial for informing the subsequent design and development decisions.
- **Definition (Requirements and Design) (Estimated: 70 hours):** This category encompasses all activities related to defining the platform's scope, functionalities, and architecture. Key tasks included preparing for and conducting stakeholder consultations, analyzing feedback, consolidating requirements, and formally documenting them in the SRS. It also involved designing the system architecture, database schema (ERD), and user interface mockups.
- **Development (Estimated: 220 hours):** This was the most time-intensive phase, involving the actual coding and implementation of the CIVENHANCE platform. Tasks included setting up the development environment, developing the backend API (Node.js/Express.js), implementing database interactions (PostgreSQL), building the frontend user interface and components (Astro/React), integrating the

recommendation logic, and developing the chatbot functionality. Basically, this phase required translating the design specifications into a working software system.

- **Testing (Estimated: 60 hours):** This phase focused on ensuring the quality, reliability, and correctness of the developed platform. Tasks included conducting all the test previously mentioned: unit tests for individual modules, integration tests to verify interactions between components (frontend-backend, API-database), and system tests using scenario-based approaches to validate end-to-end workflows. Creating simulated data for testing the recommendation engine and chatbot also fell into this category.
- **Documentation (Thesis and Project) (Estimated: 90 hours):** This category includes the time spent on creating all written deliverables associated with the project. The primary task was the writing of this thesis document, detailing the entire process from problem statement to conclusions and future work. It also includes time spent on commenting code, and preparing user documentation for the platform.

The distribution of these hours is summarized in Table 3 and visually represented in Figure 27.

Table 3. Estimated Temporal Cost Distribution by Category.

Category	Estimated Hours	Percentage of Total
Research and Investigation	40	8.3%
Definition (Requirements & Design)	70	14.6%
Development	220	45.8%
Testing	60	12.5%
Documentation (Thesis & Project)	90	18.8%
Total	480	100%

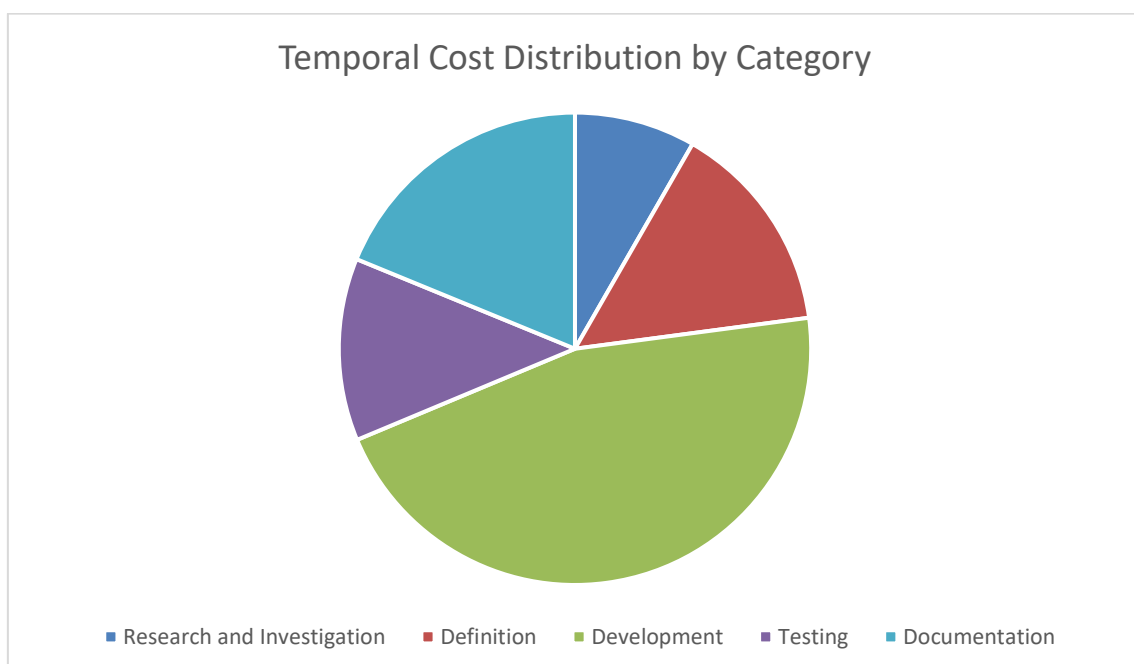


Figure 27. Temporal Cost Distribution Pie Chart.

Overall, this temporal breakdown reflects a typical software development lifecycle within an academic research context, with a significant emphasis on the development phase, followed by thorough documentation and definition.

7.5 Future Work and Recommendations

The development of the CIVENHANCE platform is an iterative process. The current prototype lays a strong foundation, but significant work remains to realize its full potential. The following are key recommendations for future work:

- **Pilot Testing and Iterative Refinement:** The immediate priority is to deploy the platform in controlled pilot programs with partner institutions involved in the Civenhance project. This deployment should involve actively gathering qualitative feedback through interviews, focus groups, and usability testing sessions, as well as quantitative data via surveys and platform analytics, from real students, teachers, and community partners. This feedback will be crucial for iterative development, helping to identify pain points, refine existing features, improve the UI/UX, and prioritize the development of new functionalities.
- **Recommendation System Enhancements:** Future work should focus on refining the recommendation system. This includes live data refinement, where the content-based algorithm's weights and Jaccard Index thresholds are tuned based on real user interaction data and feedback on recommendation quality. As the dataset of users and activities grows, there should be an exploration of advanced algorithms, potentially integrating collaborative filtering techniques or hybrid models. Moreover, natural language processing for richer content analysis can be utilized to extract deeper meaning from activity and challenges descriptions to improve match accuracy. Finally, bias detection and mitigation strategies should be proactively researched and implemented to ensure fair exposure for different types of activities or user groups.
- **Chatbot Development and Improvement:** The chatbot's capabilities can be significantly improved. This involves FAQ expansion, by continuously refining the FAQ dataset based on actual user queries logged during pilot testing to improve the efficiency and accuracy of informational responses, reducing unnecessary LLM calls. Analyzing user interactions and feedback on chatbot-provided activity recommendations to iteratively fine-tune the prompts sent to the Gemini API. This includes optimizing how user profile data, conversational cues, user preferences, and activity details are structured and presented to the LLM to improve the relevance, diversity, and justification quality of suggested activities.
- **Platform Feature Expansion:** Several new features could enhance the platform's utility. Implementing a notification system would alert users about relevant events like new activity matches or application updates. Based on user feedback, community and collaboration features like discussion forums or secure direct messaging could be

considered. Lastly, implementing feedback mechanisms for activities would allow students and teachers to rate completed e-SL activities, benefiting future participants and the recommendation engine.

- **Technical Scalability, Maintenance, and Robustness:** For long-term viability, focus should be placed on performance monitoring and optimization, implementing tools to track API response times and server load, and proactively optimizing bottlenecks. Additionally, a clear long-term maintenance plan for ongoing security updates and bug fixes is essential. Finally, implementing robust DevOps practices for automated testing and Continuous Integration/Continuous Delivery will streamline future development and ensure platform stability

To facilitate future development and onboarding of new contributors, a comprehensive guide to setting up the development environment, initializing the project, and managing core configurations is provided in Appendix A.

7.6 Concluding Remarks

This thesis has successfully navigated the complex process of designing, developing, and conducting a preliminary evaluation of the CIVENHANCE e-Service-Learning matchmaking platform. The outcome is a functional and promising prototype that directly addresses the identified need for a centralized digital tool to foster connections and collaborations within the e-SL ecosystem. The platform's architecture is sound, its core features are operational, and it is prepared to effectively address the objectives of the CIVENHANCE European project.

While the journey from a prototype to a fully deployed and impactful platform involves further challenges and iterative development, the work presented in this document provides a critical and validated foundation. On top of that, by following the future work recommendations the CIVENHANCE platform has the potential to significantly enhance the efficiency, reach, and impact of e-SL initiatives, ultimately contributing to stronger civic engagement and more resilient communities across Europe.

8 References

- [1] Waldner, L., McGorry, S., & Widener, M. (2010). Extreme E-Service Learning (XE-SL): E-Service Learning in the 100% online course. *MERLOT Journal of Online Learning and Teaching*, 6(4), 839-851. https://jolt.merlot.org/vol6no4/waldner_1210.pdf
- [2] Figuccio, M. J. (2020). Examining the Efficacy of E-Service-Learning. *Frontiers in Education*, 5. <https://doi.org/10.3389/feduc.2020.606451>
- [3] Kuh, G. D. (2008). *High-impact educational practices: What they are, who has access to them, and why they matter*. Association of American Colleges and Universities. <https://navigate.utah.edu/resources/documents/hips-kuh-2008.pdf>
- [4] GivePulse.com. (n.d.). *EService-Learning | GivePulse*. GivePulse. <https://www.givepulse.com/eservicelearning>
- [5] Faulconer, E. (2020). eService-Learning: A Decade of Research in Undergraduate Online Service-Learning. *American Journal of Distance Education*, 35:2, 100-117. https://www.usm.edu/community-engagement/eservice_learning_a_decade_of_research.pdf
- [6] Waldner, L. S., McGorry, S. Y., & Widener, M. C. (2012). E-Service-Learning: The evolution of service-learning to engage a growing online student population. *Journal of Higher Education Outreach and Engagement*, 16(2), 123. https://www.researchgate.net/publication/277129268_E-Service_Learning_The_Evolution_of_Service-Learning_to_Engage_a_Growing_Online_Student_Population
- [7] Ngai, G., Lau, K. H., & Kwan, K. P. (2023). A Large-Scale Study of Students' E-Service-Learning Experiences and Outcomes During the Pandemic. *The Journal of Experiential Education*. <https://journals.sagepub.com/doi/10.1177/10538259231171852>
- [8] Ugarte, N., Larrañaga, M., & Arruarte, A. (2022). The Use of Recommender Systems in Formal Learning. A Systematic Literature Mapping. *JUCS - Journal of Universal Computer Science*, 28(4), 414-442. <https://doi.org/10.3897/jucs.69711>
- [9] Davar, N. F., Dewan, M. a. A., & Zhang, X. (2025). AI chatbots in education: challenges and opportunities. *Information*, 16(3), 235. <https://doi.org/10.3390/info16030235>
- [10] NASPA. (2025, February 13). *The Impact of Generative AI Educational Chatbots on the Academic Support Experiences of Students in U.S. Research Universities*. NASPA. <https://www.naspa.org/blog/the-impact-of-generative-ai-educational-chatbots-on-the-academic-support-experiences-of-students-in-u-s-research-universities>
- [11] CIVENHANCE. (n.d.). <https://civenhance.eu/>
- [12] Hivebrite. (2025, May 6). *The best in class online community engagement platform | Hivebrite*. <https://hivebrite.io/>
- [13] Bettermode. (n.d.). *BetterMode: All-in-One Customer Community Platform | Engage, support & grow*. <https://bettermode.com/>
- [14] Online Engagement Software. (2025, May 9). *Online Engagement Software | Community Engagement platform*. Engagement Hub Website. <https://engagementhub.com.au/>
- [15] Fletcher, S., & Islam, M. Z. (2018). Comparing sets of patterns with the Jaccard index. *Australasian Journal of Information Systems*, 22.

https://www.researchgate.net/publication/323624308_Comparing_sets_of_patterns_with_the_Jaccard_index

- [16] ScienceDirect. (n.d.). *Jaccard similarity*. Retrieved May 31, 2025, from <https://www.sciencedirect.com/topics/computer-science/jaccard-similarity>

9 Appendices

9.1 Appendix A: Guide to set-up the environment and initialize the project.

9.1.1 Introduction

This appendix serves as a practical guide for developers undertaking further work or maintenance on the CIVENHANCE matchmaking platform. Its primary purpose is to provide a clear and concise set of instructions for setting up the necessary development environment, successfully initializing and running the project, and understanding how to manage key configurations and data entities within the system.

The platform is built upon a modern technology stack, chosen to ensure performance, scalability, and maintainability. As detailed in Chapter 4, Section 4.2 of this thesis, the frontend is developed using Astro and React, the backend API is built with Node.js and Express.js, and the database is managed by PostgreSQL. Familiarity with these core technologies will be beneficial for any developer working on this project. This guide aims to facilitate the onboarding process, enabling new team members to become productive quickly and efficiently.

9.1.2 Prerequisites

Before you begin setting up the CIVENHANCE platform, ensure you have the following software installed and any necessary accounts configured.

9.1.2.1 Software Installations

The following software is required for development:

- **Node.js and npm:** The backend and frontend are built using Node.js. Npm comes bundled with Node.js and is used for managing project dependencies (Note that npm is not mandatory, feel free to use any other package manager compatible with Node.js like yarn, bun...).
 - *Recommended Version:* LTS (Long Term Support) version of Node.js.
 - *Download:* <https://nodejs.org/>
- **PostgreSQL:** The relational database system used to store all platform data.
 - *Recommended Version:* Version 12 or higher.
 - *Download:* <https://www.postgresql.org/download/>
- **pgAdmin:** A popular administration and development platform for PostgreSQL. This is recommended for managing the database.
 - *Download:* <https://www.pgadmin.org/download/>

- **Git:** The version control system used for managing the project's source code.
 - *Download:* <https://git-scm.com/downloads>
- **Visual Studio Code (IDE):** A lightweight but powerful source code editor. This was the IDE used during the initial development.
 - *Download:* <https://code.visualstudio.com/>
 - *Recommended Extensions (Optional):*
 - ESLint
 - Prettier - Code formatter
 - Astro

9.1.2.2 Account Requirements

Google Account (for Gemini API): The platform's chatbot functionality utilizes the Google Gemini API. It will be required for the one providing the API key. However, if it is already set up, you would not need to have a google account and follow any guide related to creating the API key. For further information: <https://ai.google.dev/>

9.1.3 Environment Setup

This section outlines the steps to get the project code, configure the backend and frontend environments, and set up the database.

9.1.3.1 Cloning the Repository

Firstly, the repository containing the whole code of the platform will be needed. If you already have it or if you already know how to get it feel free to skip this part. To get it, you should go to the directory where you want to store the project and open a terminal or the other way around. Then clone the repository by using the git command:

```
git clone <repository_url>
```

Replacing `<repository_url>` with the actual URL of the repository.

9.1.3.2 Backend Setup

1. **Install Dependencies:** Once the repository is downloaded into your machine, move to the root of the project and from there to the backend directory. Once inside, the command: "npm install" must be executed in order to install all the dependencies used by the backend.

2. **Configure Environment Variables:** This is a critical part, the backend requires a “.env” file in its root directory to store sensitive configuration data. Create a file named “.env” and populate it with the following variables seen in Figure 28, replacing the placeholder values with your actual configuration:

```
DATABASE_URL=postgresql://<db_user>:<db_password>@<db_host>:<db_port>/<db_name>
PORT=<backend_port_number>
JWT_SECRET=<your_strong_jwt_secret_key>
GEMINI_API_KEY=<your_google_gemini_api_key>
```

Figure 28. Contents of the .env file.

- **DATABASE_URL:** The connection string for your PostgreSQL database.
 - **PORT:** The port on which the backend server will listen.
 - **JWT_SECRET:** A long, random, and strong secret key used for signing JSON Web Tokens.
 - **GEMINI_API_KEY:** Your API key for accessing the Google Gemini service for the chatbot.
3. **Database Setup:** The database schema and initial data are defined in SQL files (located under the sql folder in the root directory). You will need to execute these using a PostgreSQL client like pgAdmin. To do so, follow the next steps:
 - a. Connect to your PostgreSQL server using pgAdmin or another client.
 - b. Create the database if it doesn't already exist. Ensure the user specified in your DATABASE_URL has the necessary permissions and that all the values specified in your DATABASE_URL match with your database configuration.
 - c. Execute schema.sql (located within the sql folder) by opening and executing it through pgAdmin or your preferred tool. This script will create all the necessary tables, relationships, and also insert initial data for skills, interests, and universities. If do not want to add the default skills and interests in the schema, feel free to remove that part.
 - d. Execute interest_skills_translations.sql (located also in the sql folder) if you inserted the interests and skills appearing in the schema, also execute this file. This script adds the translations for the initially inserted skills and interests into their respective translation tables for all supported languages.

9.1.3.3 Frontend Setup

As with the backend, here you will also need to install the dependencies. To do so, navigate to the frontend directory under the project root directory. Once inside, the command: “npm install” must be executed in order to install all the dependencies used by the frontend.

9.1.4 Running the Platform

To run the platform, you will need to run the back-end and the front-end separately. To run the back-end, under the backend directory execute the command: “node server.js”. In your terminal you should see output: “Server listening on port <your_port>”, indicating that the server has started and is listening on the configured port. You can also test an API endpoint using a tool like Postman to ensure it's responsive.

Regarding the front-end, under the frontend directory you will need to execute the command: “npm run dev”. Then, among other messages, the terminal will display a local URL (e.g., <http://localhost:4321/>). Open this URL in your web browser to access the CIVENHANCE platform.

Moreover, if no administrator user is already created. To create one, register a new user in the platform with the desired mail and password for the administrator. Then through the database, update the role of that user to “admin” and set verified to true if it is not true already.

9.1.5 Managing Translations

Translations are managed differently for static UI text and dynamic content retrieved from the database.

9.1.5.1 Static Content Translations

Static text elements in the user interface (e.g., button labels, navigation links, form field labels, instructional text) are managed through a dedicated translation file in the frontend application. This file is translations.js, located in the src/i18n/ directory of the frontend project. As seen in Figure 29, this file is structured as a large object where each top-level key is a language code (e.g., en, es). The value for each language code is another object containing key-value pairs. The keys are unique identifiers for text snippets (e.g., loginButton, registerTitle), and the values are the translated strings for that language.

```
1  export const availableLanguages = {
2    en: 'English',
3    es: 'Español',
4    // ... other languages
5  };
6
7  export const translations = {
8    en: {
9      loginNav: "Login",
10     registerNav: "Register",
11     // ... other English translations
12   },
13   es: {
14     loginNav: "Iniciar Sesión",
15     registerNav: "Registrarse",
16     // ... other Spanish translations
17   },
18   // ... other language blocks
19 };
20
```

Figure 29. Translation.js structure.

Therefore, to add a new language you will need to add the new language code and its display name to the `availableLanguages` object. Then create a new top-level key in the `translations` object with the new language code and copy the structure from an existing language and translate all the string values. If a new translatable string must be added, simply define a new unique key for that string and add it to each language block within `translations.js` with its proper translation. Finally, the frontend application uses helper functions (like `getTranslations()`) to load and display the correct strings based on the user's selected language.

9.1.5.2 Dynamic Content Translations (Database)

Dynamic content, such as the names of skills, interests, universities, and the titles and descriptions of activities and challenges, is stored in the PostgreSQL database and requires a different approach for translation. The basic mechanism that applies to all the dynamic content in the system and is used to translate it is the presence of a translation table for each main table requiring translations. The basic structure of these tables is an id that serves as the primary key for the entry, a foreign key linking back to the main entity's id, a language code to specify the language of the translation, and one column for each of the different columns in the main entity that need a translation. In addition, it is important to note that the translation tables are used to store the data translated into a language different than English. As the data in English, which is the default language, is stored in the main table.

9.1.6 Managing Core Data Entities

This section details how to add new core data entities like skills, interests, and universities to the platform. These entities often require an entry in a main table and corresponding entries in a translation table, as explained before.

9.1.6.1 Adding Skills/Interests and Their Translations

When needing to add a new skill or interest to the system, an insert must be made into its respective table in the database (interests or skills). These tables basically have two columns: id and name. Therefore, to insert a new entry only the name must be inserted (in English, as explained before) as the id will be auto-generated. So, an insert to any of these tables looks like this: `“INSERT INTO skills (name) VALUES ('New Skill Name in English');”`.

After having the new entry inserted, you will need to add its translations into the corresponding translation table (`skill_translations` or `interest_translations`). To do so, you will need the id of the newly inserted entry. Once known, then you will need to insert this id as the foreign key to the main entry, the language code, and the translation of the interest or skill. For example, if `N` was the new entry id, then an insert of a translation will look like this: `“INSERT INTO skill_translations (skill_id, language_code, translated_name) VALUES (N, 'es', 'Nuevo Nombre de Habilidad en Español');”`. Note that the `schema.sql` file already populates a base set of skills and interests, and the `interest_skills_translations.sql` file provides their initial translations. You can refer to these files for more examples.

9.1.6.2 Adding Universities

Adding a new university follows a similar two-step process: adding the main entry and then its translations. Firstly, insert the new university into the universities table (the name and location columns, as always in a main entity store the English version). The insert will look like this: "INSERT INTO universities (name, location) VALUES ('New University Name', 'City, Country');".

Again, as before, with the new entry inserted in the main entity, the id will have to be retrieved in order to insert it as the foreign key in the translation table to the main entity, together with the translated name and location of the university. Resulting in an insert like this: "INSERT INTO university_translations (university_id, language_code, translated_name, translated_location) VALUES (U, 'es', 'Nuevo Nombre de Universidad', 'Ciudad, País');"

9.1.7 Recommendation Service Configuration

The recommendation service, implemented in the recommendationService.js file, is responsible for generating personalized activity suggestions for students and teachers. Its execution and scoring logic are key aspects to understand. As well as its periodic execution.

9.1.7.1 Execution and Scheduling

The execution of the recommendation engine service is handled in the server.js file. As seen in Figure 30, the runRecommendationEngine() function is currently invoked immediately when the backend server starts. However, this is primarily for development and testing purposes to ensure recommendations are populated when the server initializes. For a production environment, this should be disabled.

```
(async () => {  
  console.log("Running recommendation engine on server start (for testing)...");  
  await runRecommendationEngine();  
})();
```

Figure 30. Recommendation Service Execution on Initialization.

For a production environment, the snippet of code shown in Figure 31 which is currently commented, should be uncommented. This code is for scheduling the recommendation engine to run periodically using node-cron.

```
// Schedule the recommendation engine to run (For now, daily at 2 AM)  
cron.schedule('0 2 * * *', () => {  
  console.log('Running scheduled recommendation engine job...');  
  runRecommendationEngine().catch(err => console.error('Scheduled recommendation job failed'));  
});
```

Figure 31. Recommendation Service Scheduling.

To enable and configure this for a production environment. Firstly, as said, it must be uncommented. Then, the cron expression: "0 2 * * *" should be modified according to the desired frequency for updating recommendations. The cron expression consists of five fields

representing: minute, hour, day of month, month, and day of week. Therefore, the expression just seen schedules a daily execution at 2 AM.

9.1.7.2 Scoring, Weights, and Thresholds

Now focusing on how the algorithm works, the relevance of an activity for a user is determined by a scoring mechanism within `recommendationService.js`. This score is a weighted sum of similarities across different attributes. The Jaccard Index is used to calculate the similarity between sets of attributes (e.g., a student's skills vs. an activity's required skills). This is used not only for skills but also for interest. But this gives two values when we only need 1. Therefore, a weight is applied to each of them, a 50 % weight is applied to the skills, 40 % to the interest, and the remaining 10 % is for a simple verification that checks if the activity's location is the same as the student's location. Finally, there is a final threshold applied to filter out the activities whose score is not high enough for the recommendation to be considered relevant. This threshold is currently set at 0.1, but as explained during the thesis, based on testing with actual user data this value might need to be re-adjusted. Moreover, a final limit is set so only the 10 recommendations with the highest score are stored.

As has been said, these weights and thresholds can be adjusted within the `recommendationService.js` file to fine-tune the recommendation engine's behaviour and the types of matches it prioritizes.

9.1.8 Chatbot Configuration

As explained during the thesis, the platform's chatbot provides on-demand assistance to users. It first attempts to answer queries using a predefined FAQ dataset and can escalate to the Google Gemini API for more complex questions or when direct FAQ matches are not found.

9.1.8.1 Managing FAQ Dataset

The FAQ data is the primary source for the chatbot's responses to common queries. It is located in the file `faqData.js`, under the `data` folder directory in the backend project root. This file exports an object which has top-level keys representing the language codes (as with `translations.js`). Each language object then contains further nested objects, where each key is a unique identifier for a FAQ item. Finally, each FAQ item contains:

- **keywords:** A string of space-separated keywords relevant to the question. These are used by the matching logic in `chatbot.js`.
- **question:** The question in that specific language.
- **answer:** The predefined answer to the question in that language.

For a clearer view of how this file is structured, refer to the actual file or Figure 32.

```
1  const faqData = {
2    en: {
3      whatIsCivenhance: {
4        keywords: "what is civenhance platform meaning purpose",
5        question: "What is CIVENHANCE?",
6        answer: "CIVENHANCE is a platform connecting students, teachers, and community partners for service-learning projec
7      },
8      // ... other English FAQs
9    },
10   es: {
11     whatIsCivenhance: {
12       keywords: "que es civenhance plataforma significado proposito",
13       question: "¿Qué es CIVENHANCE?",
14       answer: "CIVENHANCE es una plataforma que conecta a estudiantes, profesores y socios comunitarios para proyectos de
15     },
16     // ... other Spanish FAQs
17   },
18   // ... other language blocks
19 };
20
21 module.exports = {
22   faqData
23 };
```

Figure 32. Structure of faqData.js file.

9.1.8.2 Configuring Gemini API Access

When a user's query is not found in the FAQ dataset or the system detects a user intent for asking recommendations, the chatbot escalates the request to the Google Gemini API. For the calls to the API to work, an API key is needed. Which as explained before in section 9.1.3.2, this key must be set as an environment variable in the .env file. In addition, if the model to which the system makes the call is changed, the API key will have to be changed as well as the endpoint to where the call is made.

Finally, all interactions, whether answered by the FAQ or Gemini, are logged into the chatbot_logs database table for monitoring and future analysis.